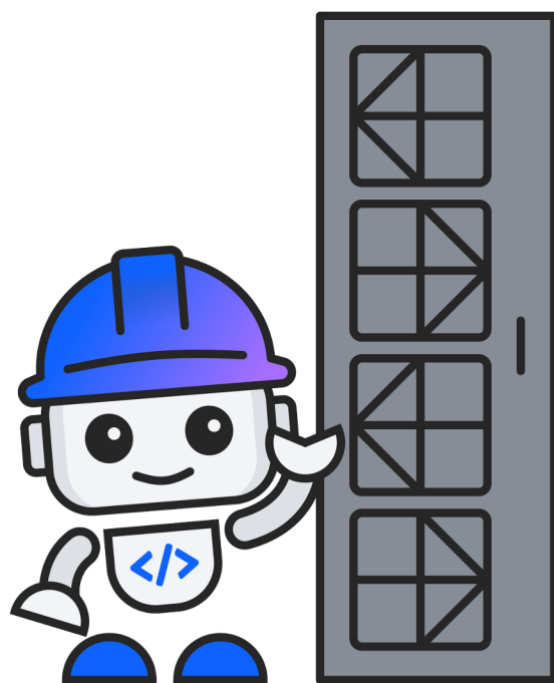




BOB Premium Package for Z

Bobathon Lab Guide

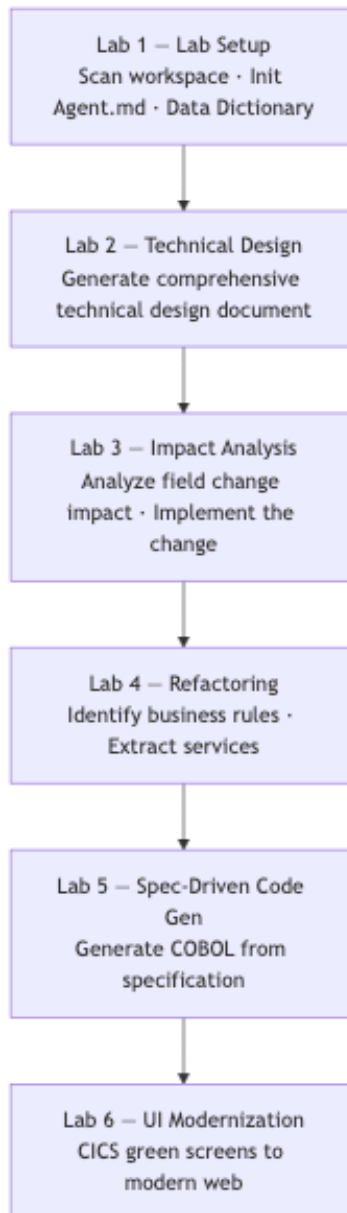
IBM Client Engineering • 2026

Bob Premium Package for Z

Created by Sophie Harrison & Renate Hamrick — Application Modernization for Z team

Overview

This guide walks through six hands-on labs using IBM Bob Premium Package for Z inside VS Code. Complete the labs in order — each one builds on the setup completed in the previous section.



Lab 1: Getting Started with IBM Bob Premium Package for Z

This lab introduces Bob, walks through the interface, and completes the three setup steps that every use case depends on.

Part 1: What Is IBM Bob Premium Package for Z?

IBM Bob is an AI-powered coding assistant built into VS Code. Bob Premium Package for Z is the edition purpose-built for mainframe development — it understands COBOL, PL/I, HLASM, JCL, CICS, IMS, MQ, and DB2 natively.

Bob Is a Multi-Agent System

Bob is not a single AI model answering questions. It is a **multi-agent system** — a team of specialized AI agents that coordinate to complete tasks.

When you send Bob a prompt, here is what happens behind the scenes:

1. A **planning agent** reads your request and breaks it into a series of steps
2. **Specialist agents** (Z Architect, Z Code, etc.) each handle the parts they are best suited for
3. Agents call **tools** — `scan_program`, `get_control_flow`, `resource_mapper`, and others — to read your code, query the metadata database, and take actions
4. Each tool call requires your approval (unless you turn on Auto Approve) — this keeps you in control
5. Agents share what they discover and hand off work to each other as the task progresses

This is why Bob produces results that feel like a team of experts worked on your code — because, architecturally, a team did.

What Makes Bob Different for Z

Capability	What It Means for You
Workspace-level understanding	Bob scans your entire codebase, not just the file you have open. It understands program relationships, copybook dependencies, and DB2 table mappings.
Z-native modes	Z Architect mode handles analysis, documentation, and planning. Z Code mode handles code changes, generation, and modernization. Each mode uses agents tuned for that type of work.
Persistent context	Agent.md and the Data Dictionary follow Bob across every session. You set them up once; they guide Bob from that point forward.
Tool-driven actions	Bob doesn't just describe what to do — it does it. It reads files, edits code, generates documents,

Capability

What It Means for You

Approval-based workflow

and runs analysis queries directly in your workspace.

Every action Bob wants to take appears as a tool request you must approve. You always see what Bob is about to do before it does it.

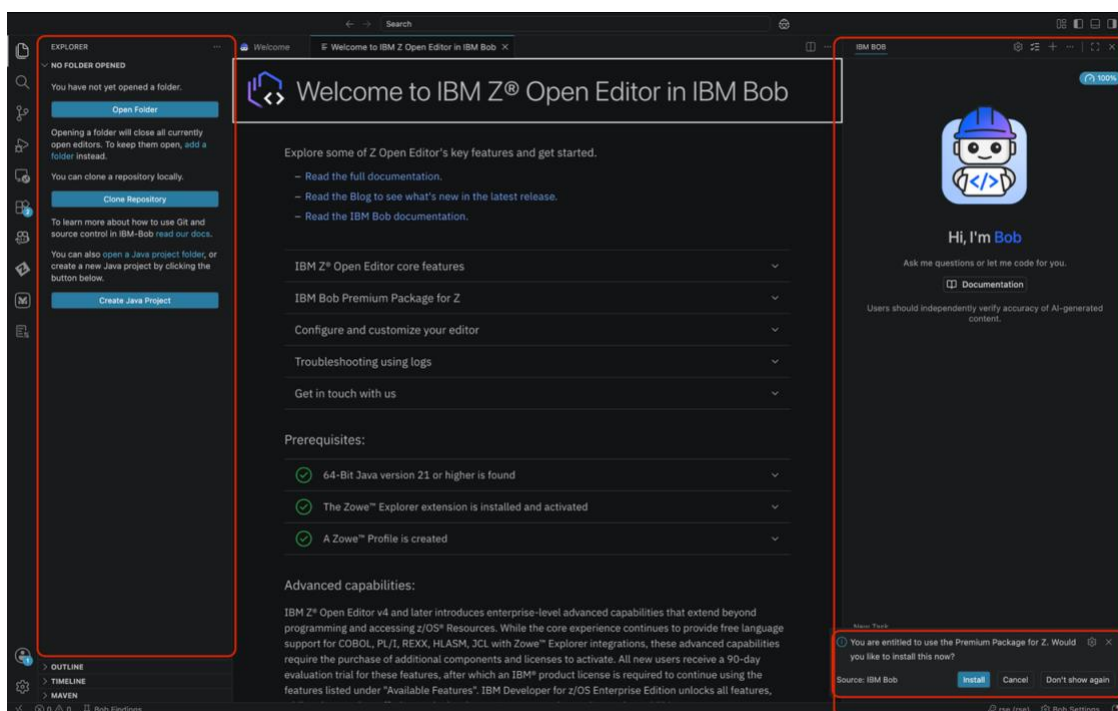
Part 2: The Bob Interface

Before running any prompts, take a moment to orient yourself to the Bob interface inside VS Code.

The VS Code Layout with Bob

Left panel — File Explorer This is where your code lives. When Bob generates documentation, architecture diagrams, data dictionaries, or new programs, they appear here as files you can open.

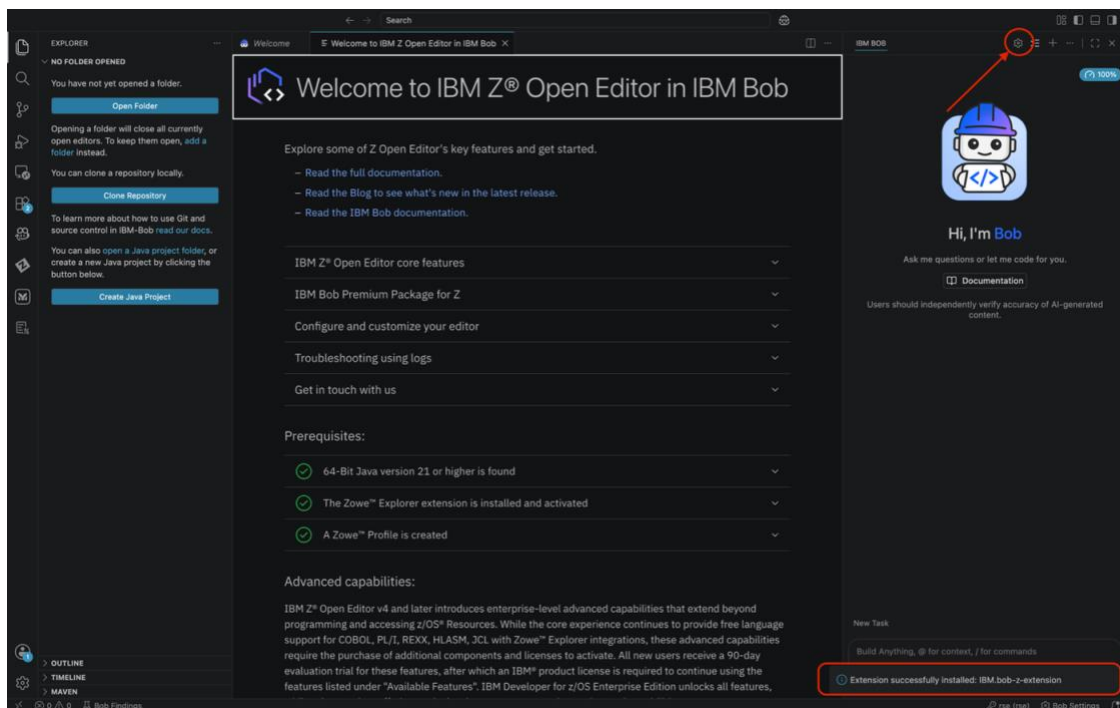
Right panel — Chat Window This is where you interact with Bob. Type prompts, review responses, and approve or decline tool requests. When Bob wants to read a file or run an analysis tool, you'll see a tool request card appear in the chat.



The VS Code layout with Bob

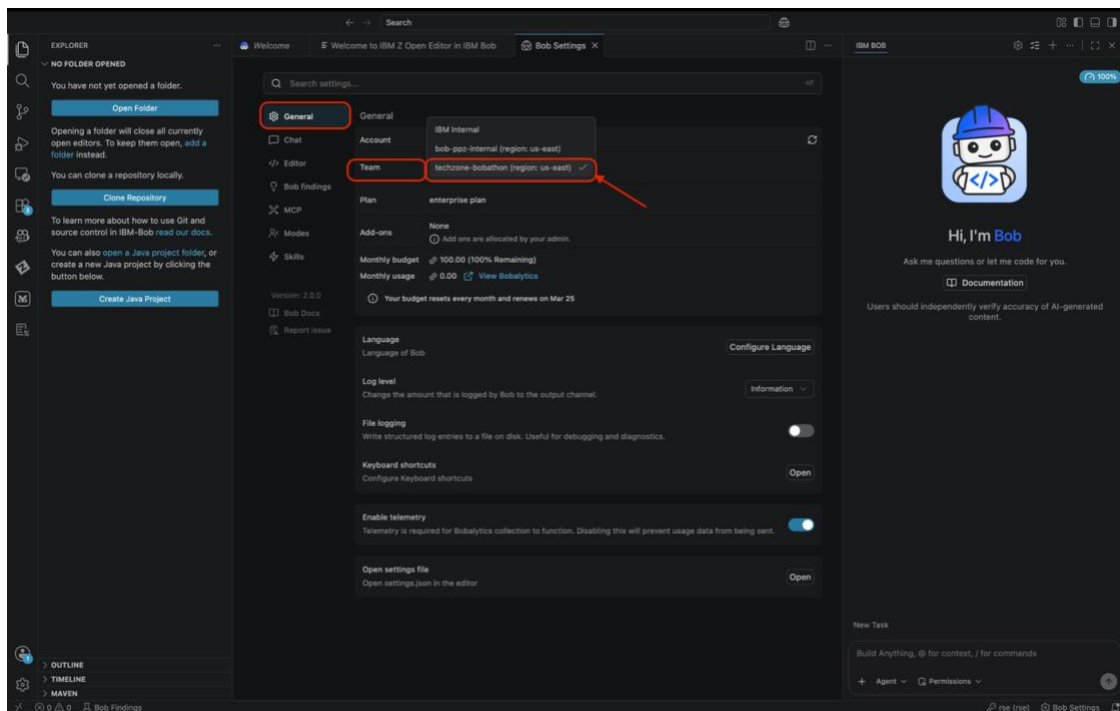
Ensure You Have Access to Bob Premium Package for Z

ACTION: Under bob settings, locate the general settings.



Bob general settings

ACTION: Select the team's name from the drop down, for instance, in the screenshot **techzone-bobathon**. Please ask your instructor for the team's name for this bobathon.



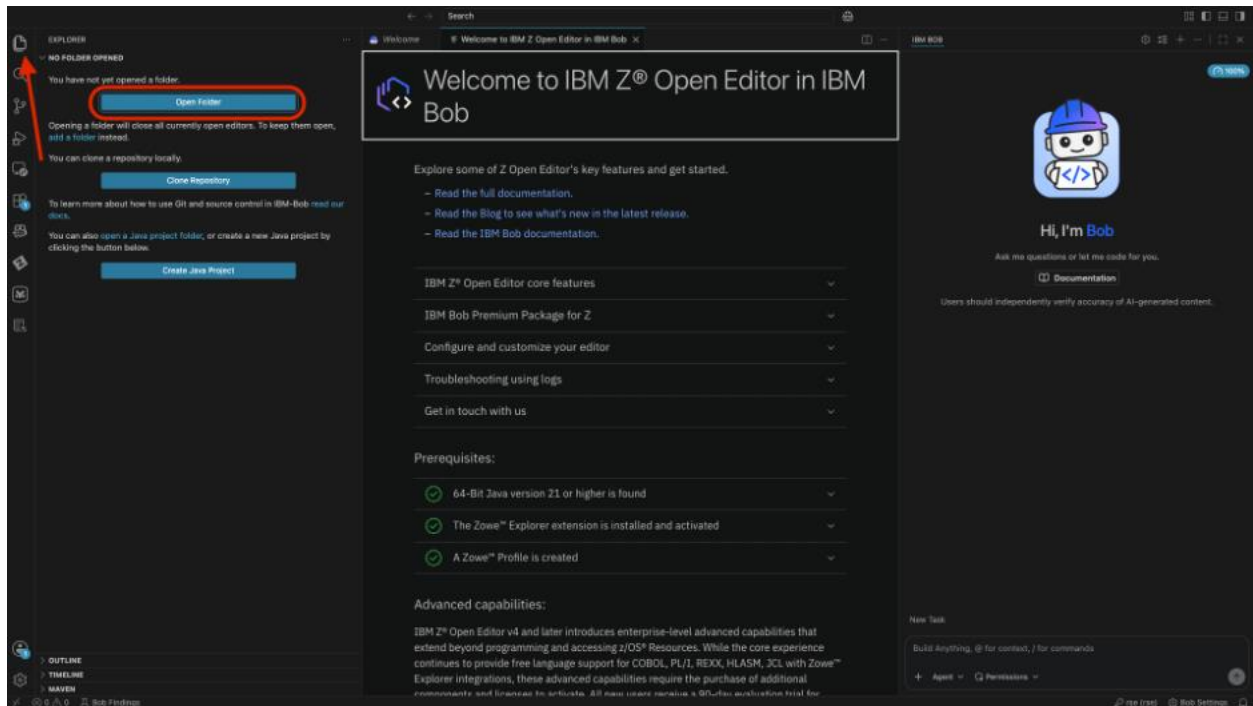
Select team name

Pulling the Sample Application Code to Use with Bob

ACTION: In Bob and in the Explorer section select **Open Folder**

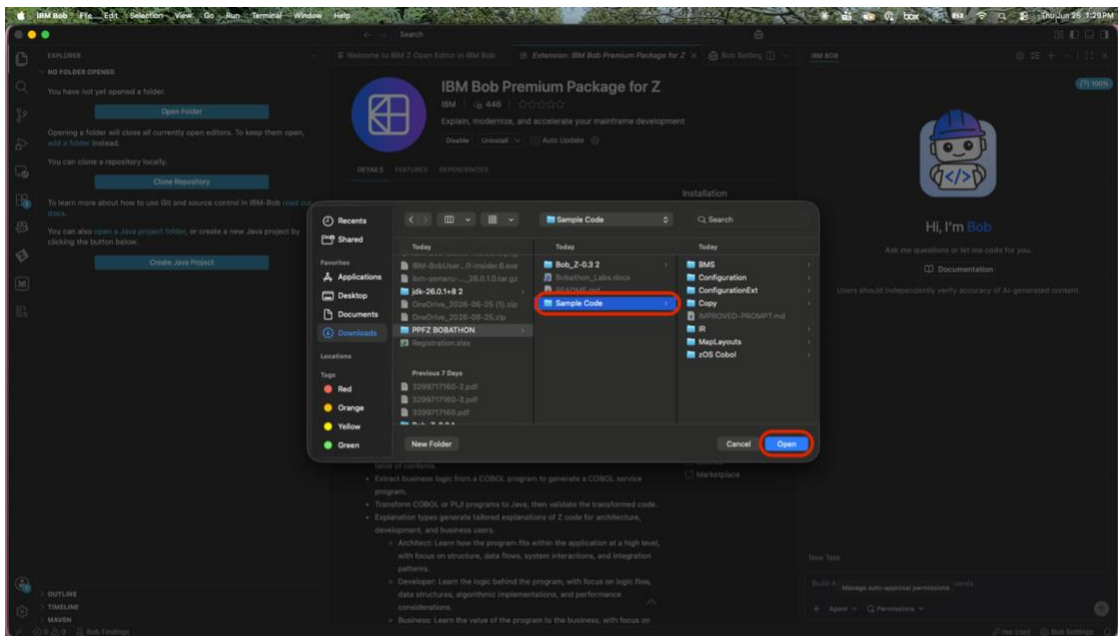
Tip

You can also use the keyboard shortcut **Ctrl + K, Ctrl + O** to open a folder or select **File → Open Folder** from the menu bar.



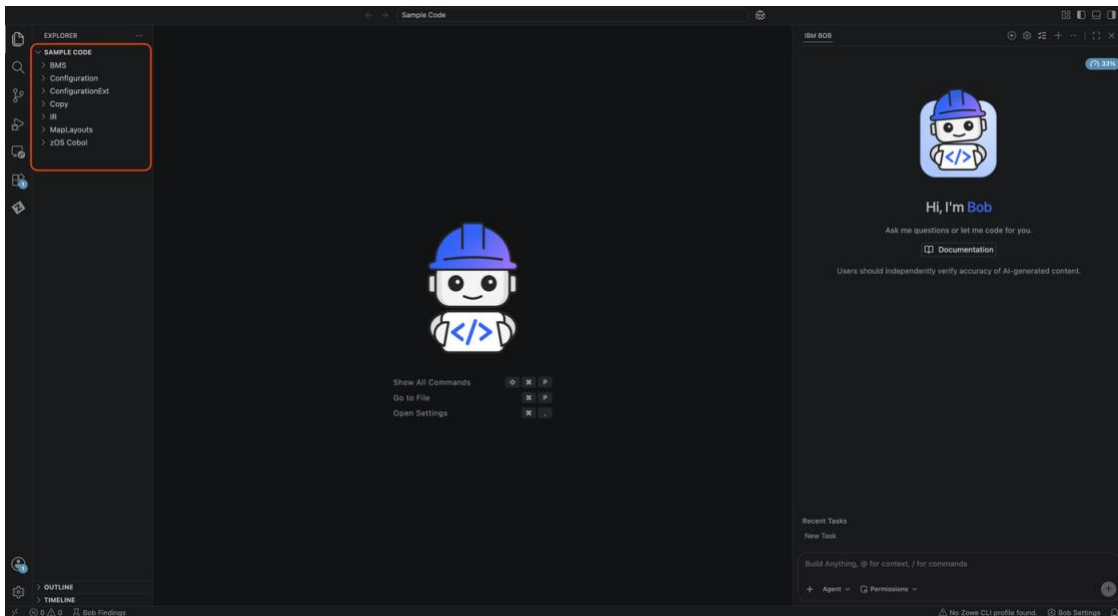
Open Folder in VS Code Explorer

ACTION: Please select the Sample Code Folder you downloaded and click the **Open** button.



Select and open the Sample Code folder

Once complete you should see the sample code in the Explorer section of VS Code.



Sample code loaded in Explorer

Switching Modes

The **mode selector** is at the bottom of the chat window. Click it to see available modes and switch between them.

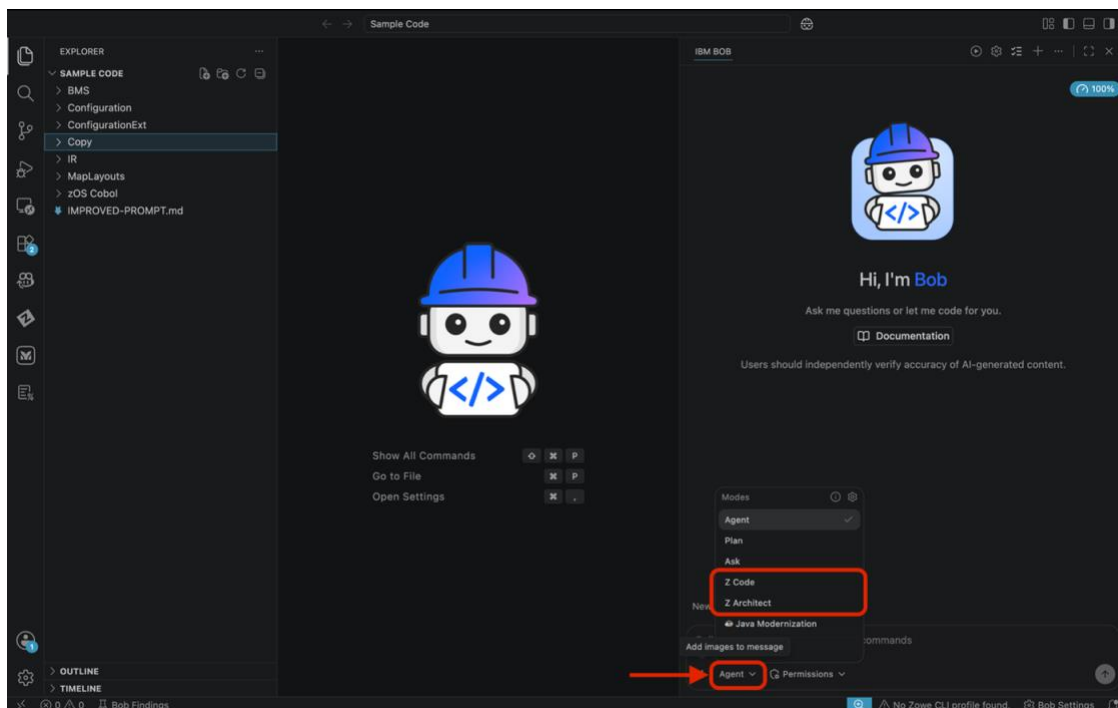
Mode

When to Use

Mode	When to Use
Z Architect	Analysis, documentation, impact analysis, architecture diagrams, data dictionaries, Agent.md setup
Z Code	Code changes, service extraction, UI modernization, code generation, COBOL-to-Java

Mode Matters

Bob uses different agents depending on the active mode. If a prompt isn't getting the result you expect, check that you are in the right mode first.



Mode selector at the bottom of the chat window

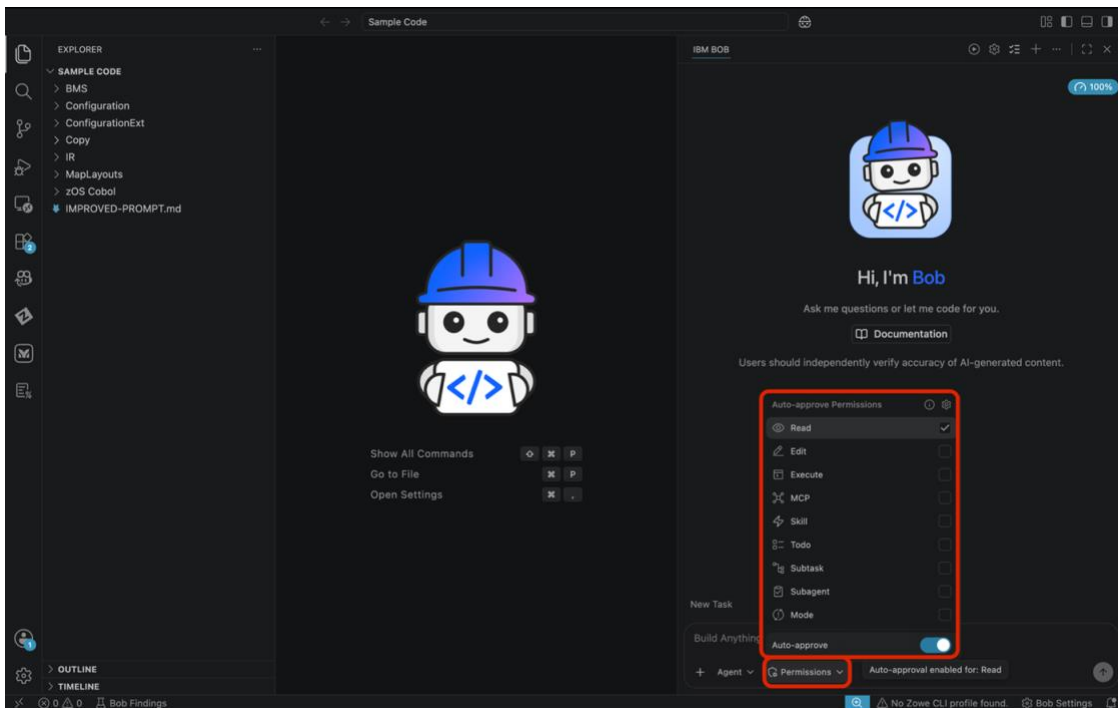
Tool Request Approvals

When Bob needs to take an action, a tool request card appears in the chat:

- Click **Approve** to let Bob proceed
- Click **Reject** to stop that specific action
- You can also enable **Auto Approve** (**Permissions** dropdown near the mode selector) to approve all requests automatically — useful for long-running workflows

Tip

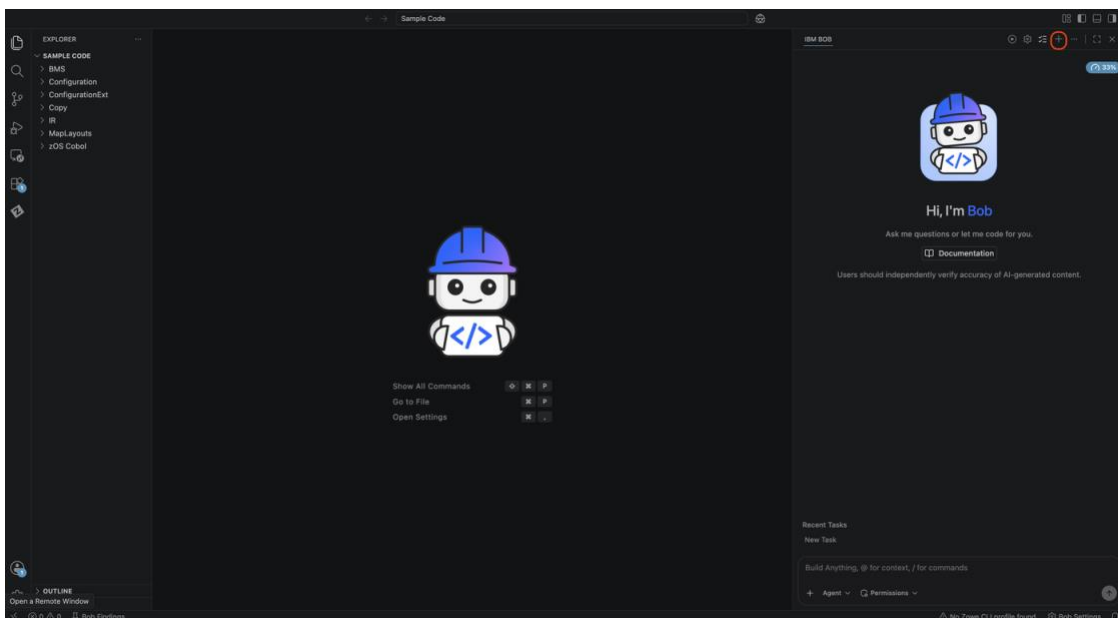
You may be asked to approve several requests in sequence for a single prompt. This is normal — each approval allows Bob to complete one step before moving to the next.



Tool request approval card in the chat

Starting a New Chat

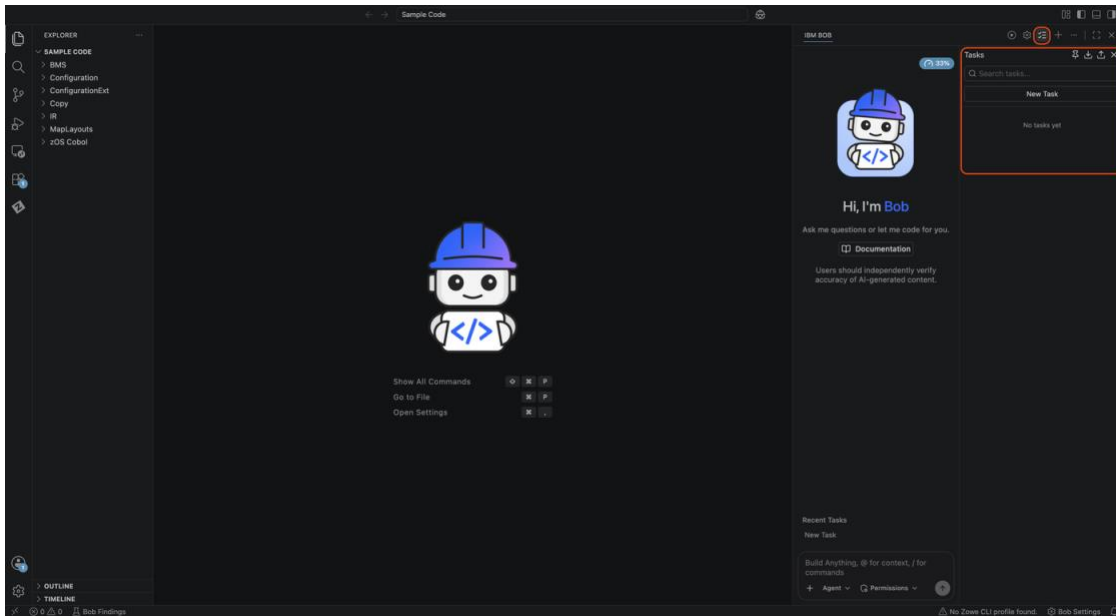
Click the + button at the top of the chat panel to start a fresh conversation. Your chat history is preserved — you can browse previous sessions using the history icon. A fresh chat is recommended at the start of each new use case to avoid context from a prior session carrying over.



New chat button at the top of the chat panel

Tasks

You can also run multiple tasks at once and here is where you may toggle between them.



Tasks panel for toggling between multiple tasks

Part 3: Scan the Workspace

What This Does

The workspace scan builds a local metadata database that Bob uses to understand your entire application — program inventory, copybook dependencies, DB2 table mappings, call graphs, and program-to-program relationships. This database is what allows Bob to answer questions like “what programs use this field?” or “what does this program call?” without you having to tell it anything.

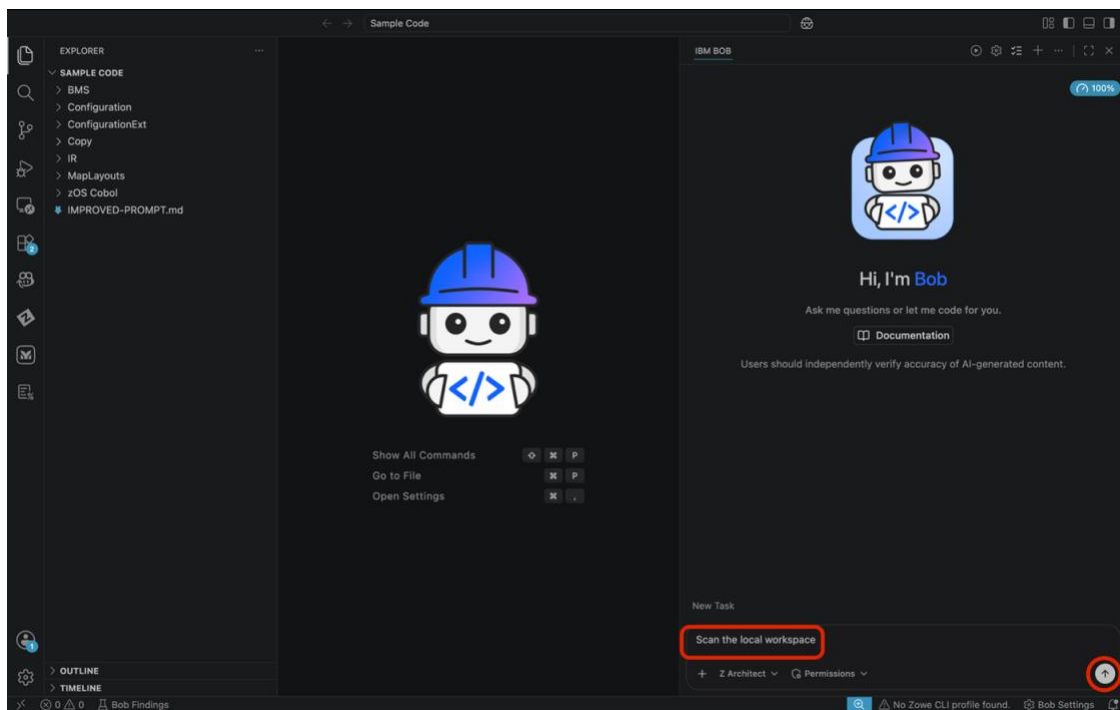
Without the scan: Bob works only from files you explicitly share or mention.

With the scan: Bob proactively builds out and has a deep understanding of the whole application.

Actions

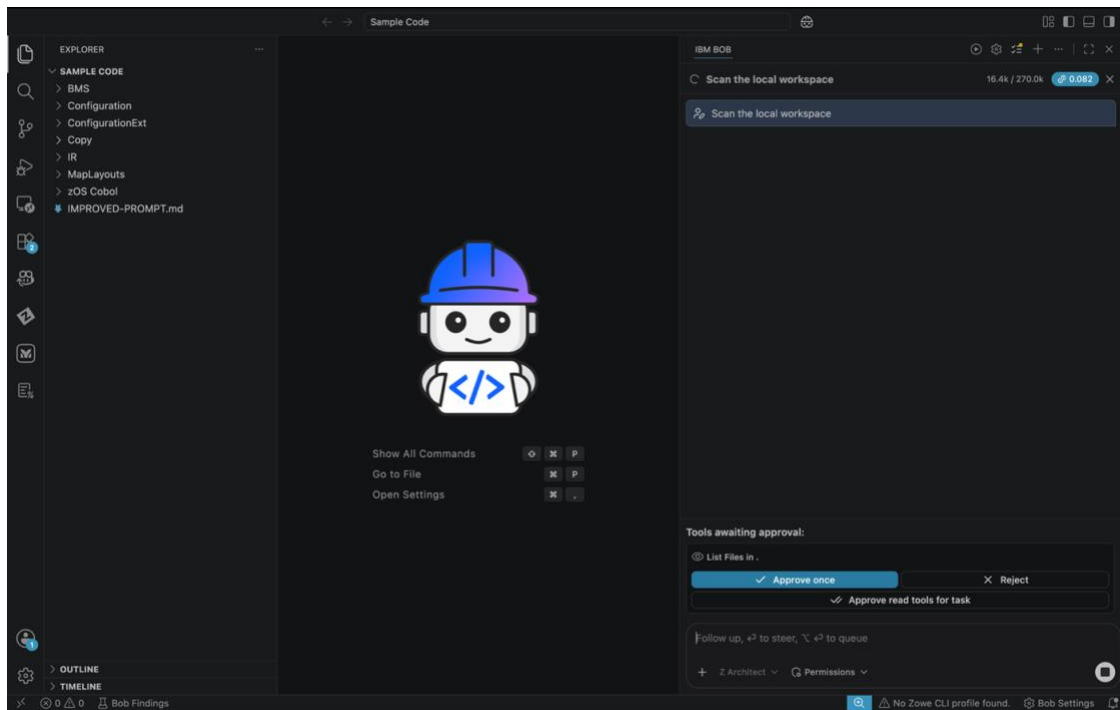
1. Ensure you are in **Z Architect** mode. In the chat window, please paste/type the following prompt:

Scan the local workspace



Scan the local workspace prompt in Z Architect mode

2. When Bob needs to take an action, a tool request card appears in the chat (example below):



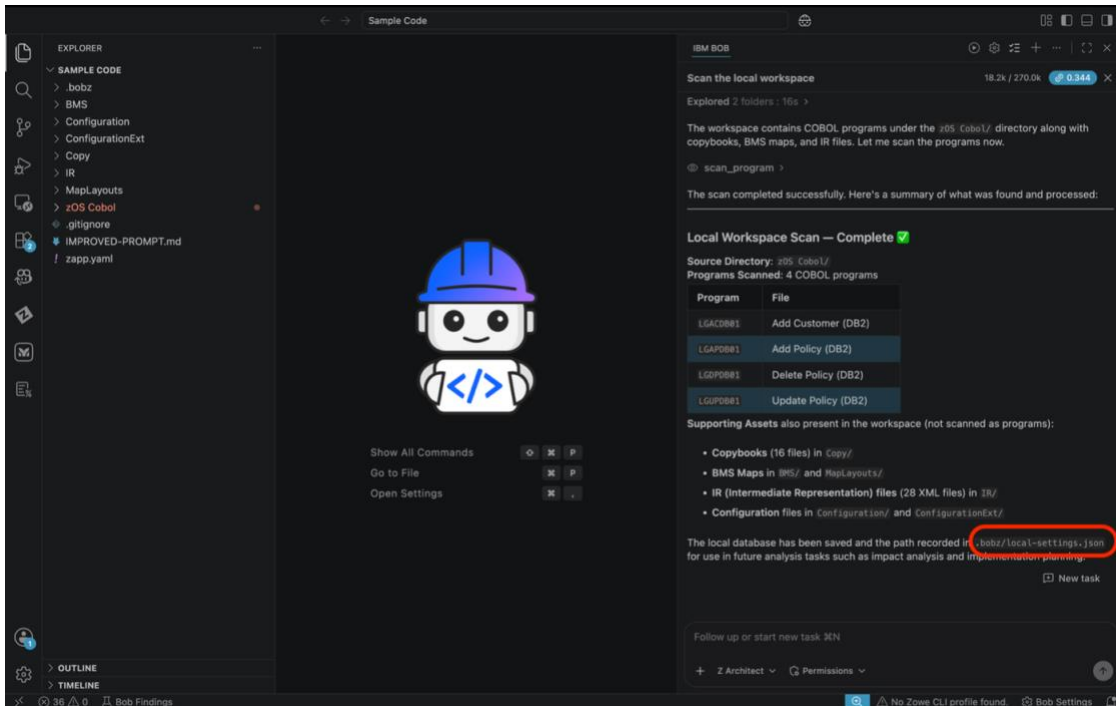
Tool request card during workspace scan

- Additional approval requests may follow — click **Approve** for each or you may turn your auto approvals on in **Permissions**.

Do Not Close VS Code During the Scan

The scan must complete fully before Bob can run analysis queries against it.

- The scan typically takes 2–5 minutes depending on workspace size. When complete, Bob will confirm the scan finished and show a summary. You will note the location of your database in the `.bobz` folder under `local-settings.json`. The local database is now available for architecture analysis tasks.



Completed workspace scan confirmation

Expected Results

- ✓ Local metadata database created
- ✓ Program inventory populated
- ✓ Bob ready for application-level analysis

Part 4: Initialize Agent.md

What This Does

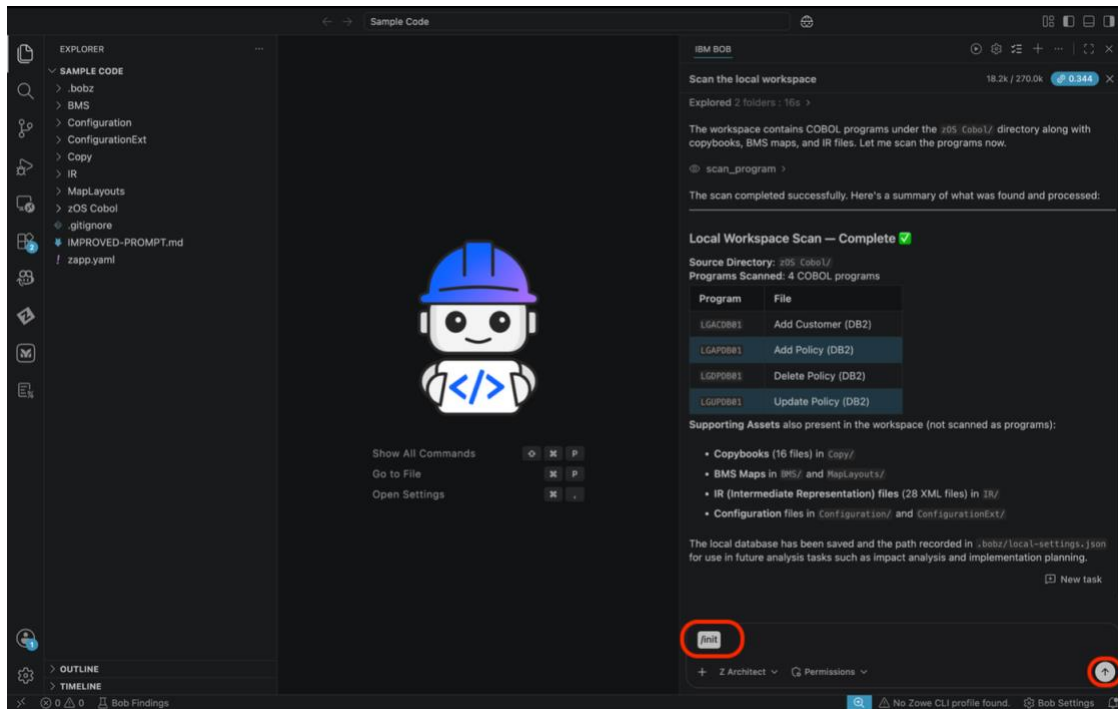
Agent.md is Bob's persistent context document for your project. It creates a central reference for the agents, containing key artifacts such as file locations and user preferences, which guides the agents in analyzing COBOL programs by capturing details such as workspace type, repository structure, data dictionary location, and other relevant configuration information.

Bob reads Agent.md automatically at the start of every session — giving Bob project-aware responses from the first prompt.

Actions

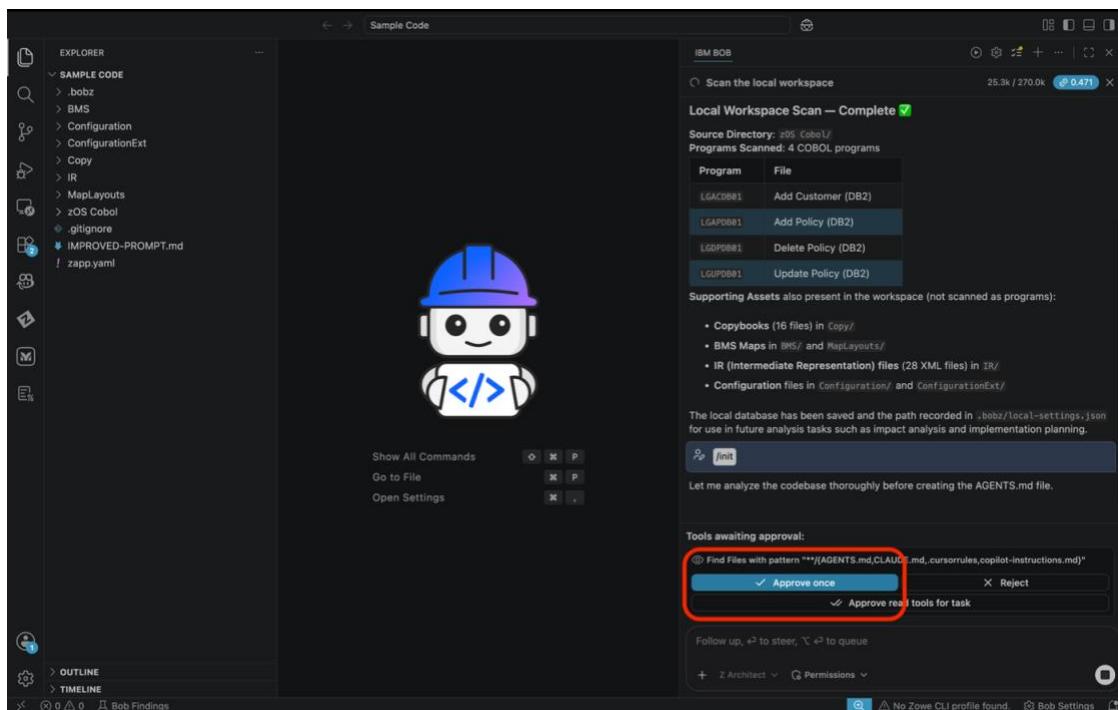
1. Ensure you are in **Z Architect** mode. In the chat window, please paste/type the following prompt:

/init



/init prompt in Z Architect mode

2. You will notice Bob starts by reading through your code base and reviewing the assets of the .bobz folder then he will begin to write the AGENTS.md. Click **Approve** for each step as prompted.

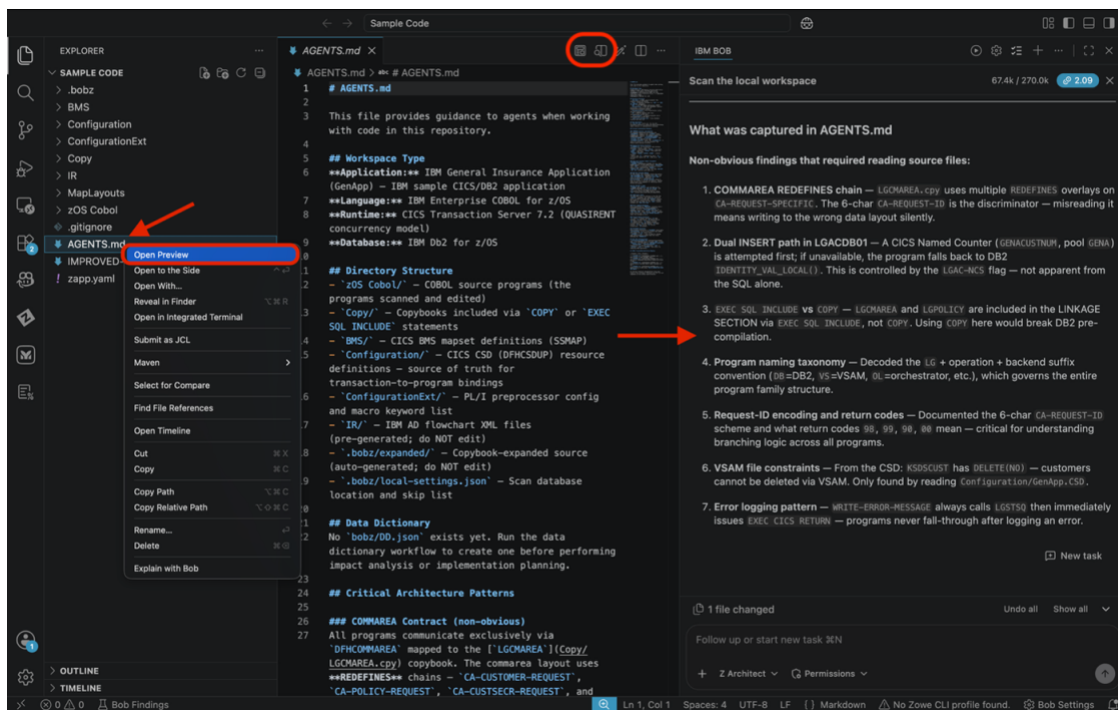


Bob writing AGENTS.md with approval prompts

- When complete, an AGENTS.md file will appear in the file explorer. Click **Preview** to review it.

Tip

If the AGENTS.md doesn't automatically open, locate it on the left explorer panel. You can preview by right clicking the file and selecting **Open Preview**, or you can select the file then in the top of your viewer select the icon that allows you to preview.



AGENTS.md file in the Explorer panel

Expected Results

- ☒ AGENTS.md created at the workspace root
- ☒ Workspace type and structure documented
- ☒ Key directories (COBOL source, copybooks, BMS, JCL) captured
- ☒ Program naming conventions with call patterns
- ☒ Location of scan database documented

Part 5: Generate a Data Dictionary

What This Does

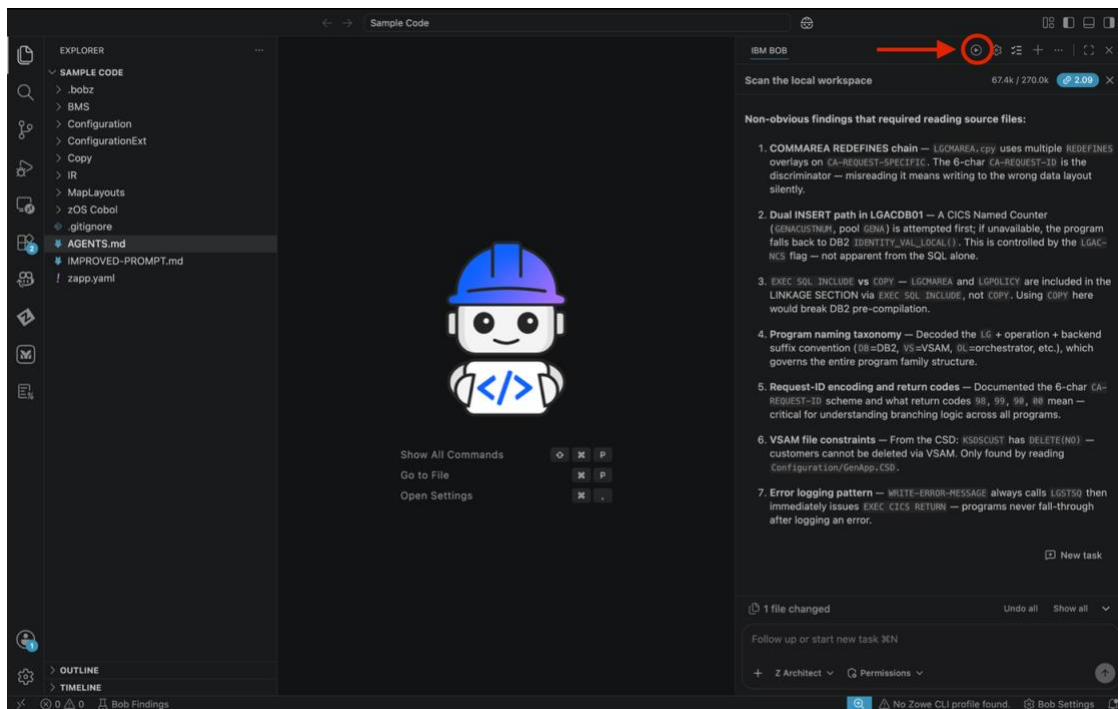
The data dictionary maps COBOL variable names to their business-language meanings. COBOL field names like CA-CUSTOMER-NUM or DB2-LASTNAME are technical identifiers — the data dictionary gives them human-readable descriptions and business context.

Bob uses the data dictionary throughout your session to explain code in meaningful terms to your enterprise, identify relationships between fields, and reason about business logic rather than raw field names. It is stored as bobz/DD.json and referenced by Agent.md.

A workflow will be used for this task. Workflows guide you through complex tasks step by step with built-in AI prompts.

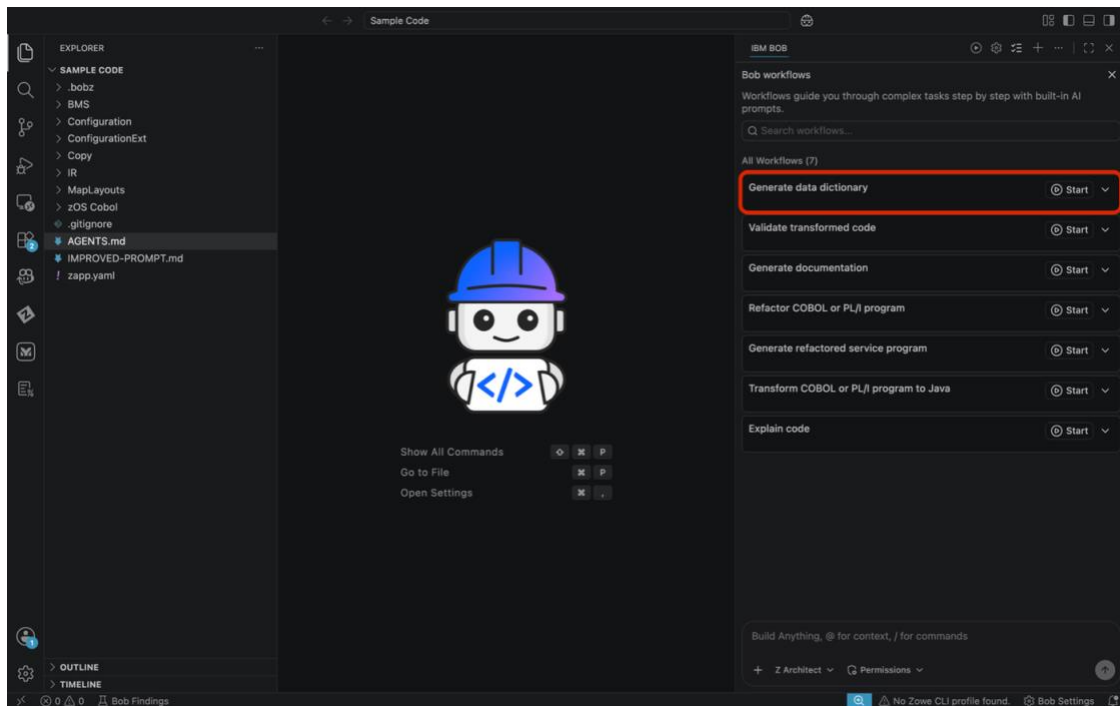
Actions

1. Select the **Workflows** icon.



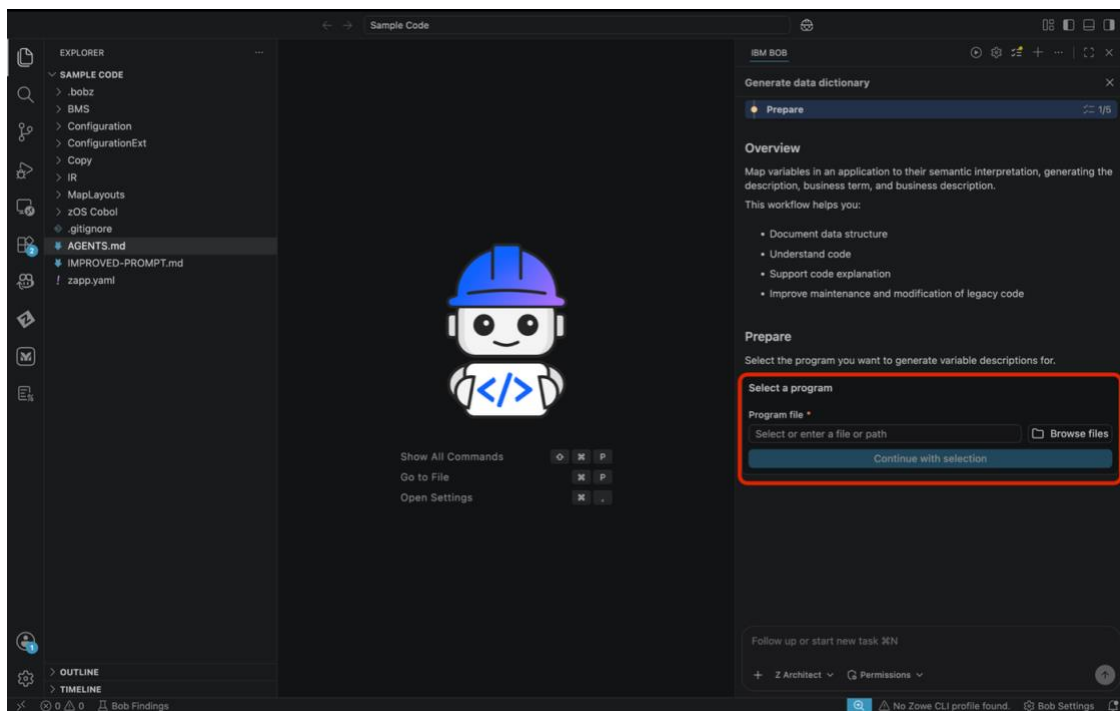
Workflows icon in the Bob interface

2. Select the workflow titled **Generate Data Dictionary**. You can use the dropdown to review what the workflow is used for.



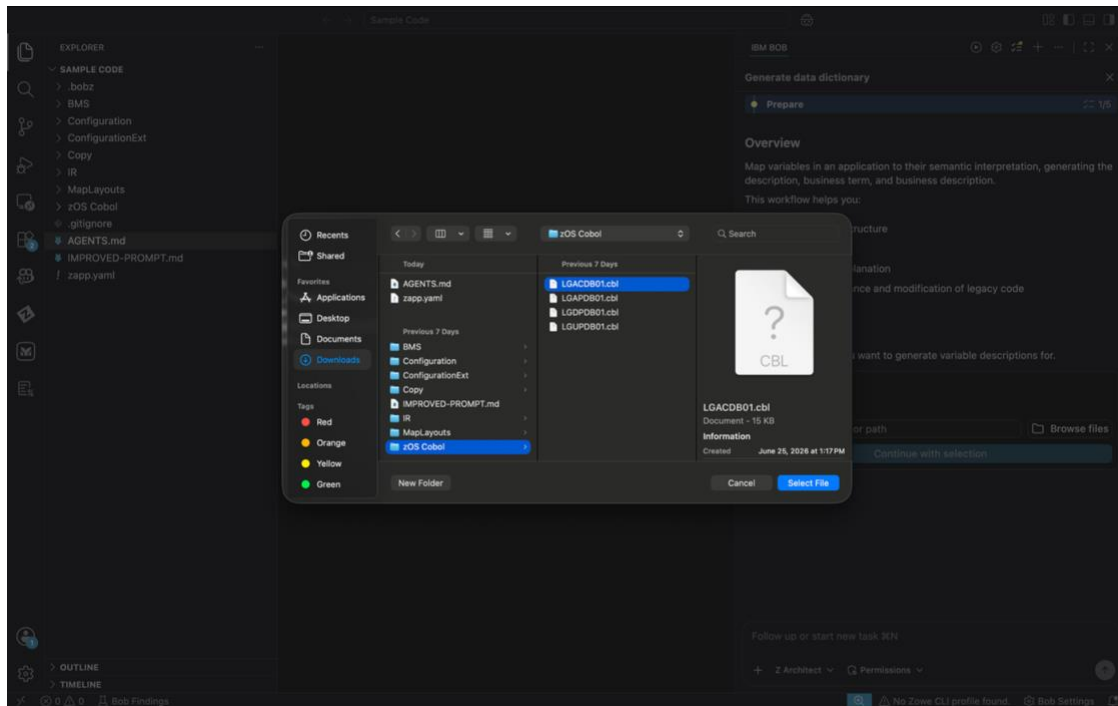
Generate Data Dictionary workflow selection

3. Bob will have you select a program. You can input a path or click **Browse Files** to select a code file. Then click **Continue with selection**.



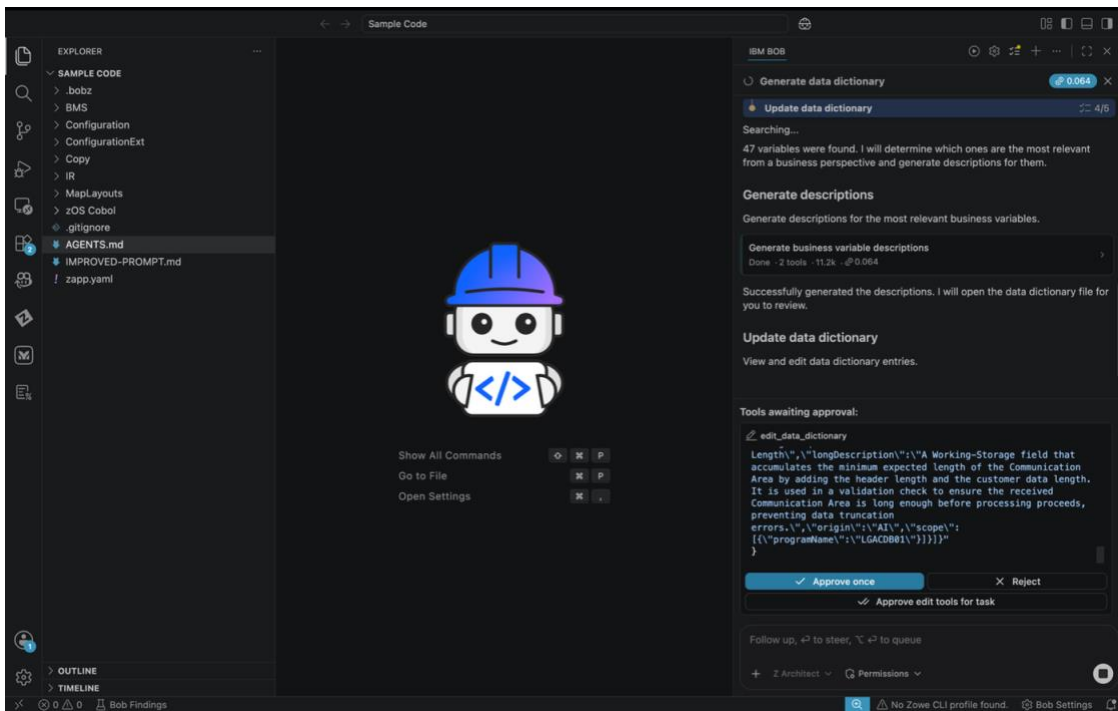
Program file selection screen

ACTION: Select **Browse files** → **zOS Cobol** → **LGACDB01.cbl** → select file → **Continue with selection**.



Browse files to select LGACDB01.cbl

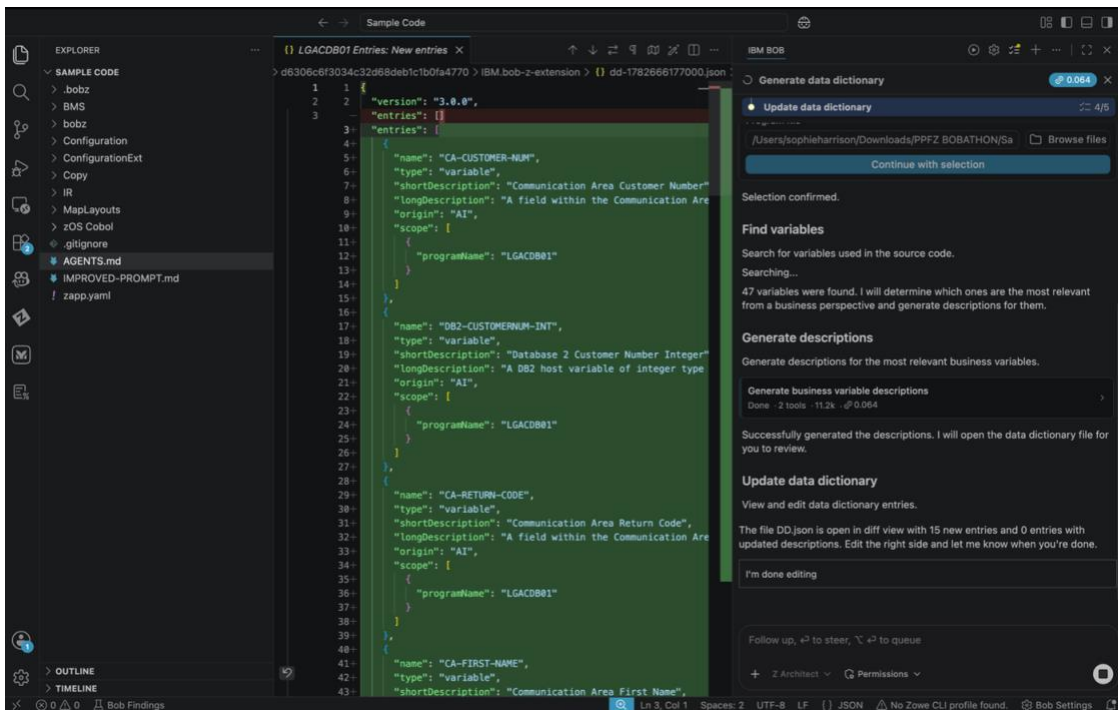
4. Bob may ask to use various skills to create the data dictionary. If auto approve is off, please select **Approve**. If prompted for various other approvals, please select approve each time unless noted.



Skill approval request during Data Dictionary generation

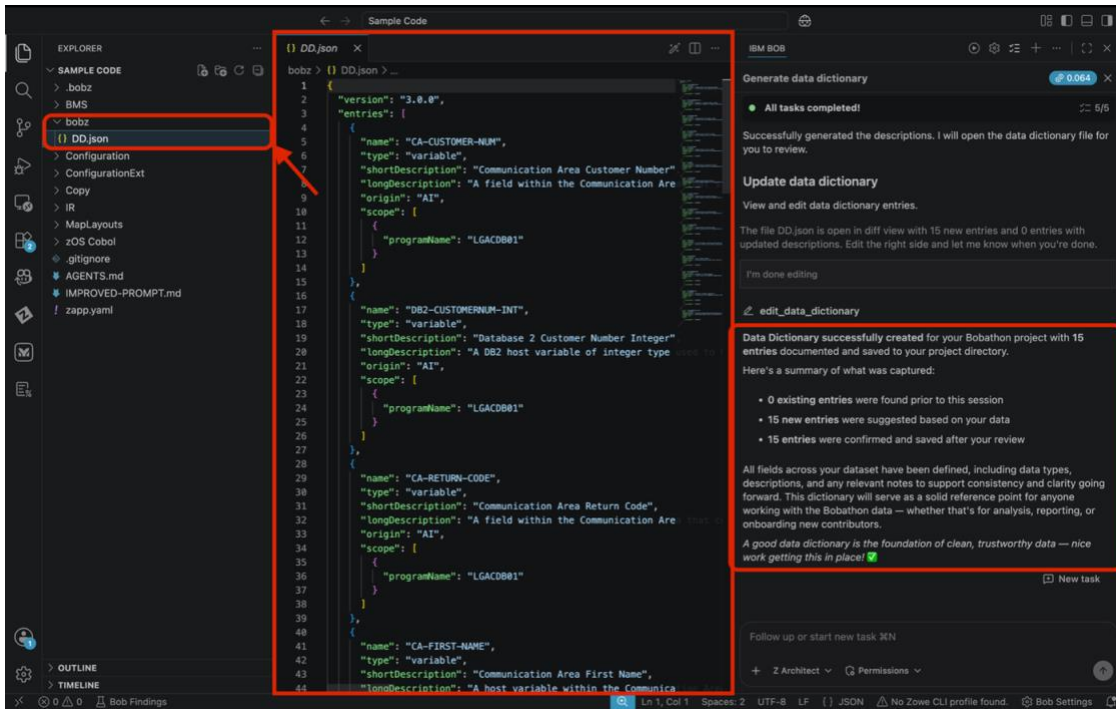
5. You will then be asked to review or edit entries:

- This is optional but valuable — you can correct interpretations, add business context, or clarify variable meanings at this point



Review and edit Data Dictionary entries

- When complete, a `DD.json` file will appear in the file explorer under the `bobz` folder. Click to review it.



DD.json file in the bobz folder

Expected Results

- ✓ bobz/DD.json created with variable descriptions
- ✓ Each entry includes a short description and detailed business context

✓ Lab 1 Setup Complete

Bob now has everything it needs to work deeply with your codebase:

What Was Built

Metadata database (from scan)

Agent.md

Data Dictionary

What It Does for Bob

Answers questions about the whole application, not just open files

Provides project context at the start of every session

Translates field names into business language

Start a New Chat

Please select the + sign at the top of the chat window to start a new session before moving to the next lab.

Lab 2: Technical Design Document

Prerequisites: Complete Lab 1 (Lab Setup) before starting this lab

Code Set: Any COBOL workspace — recommended code: `Sample Code`

Duration: 10–15 minutes

Difficulty: Beginner

Overview

Generate a comprehensive technical design document for your COBOL application — covering program inventory, processing logic with business rules, code snippets, and system architecture with visual Mermaid diagrams. The document is produced from the metadata built during lab setup.

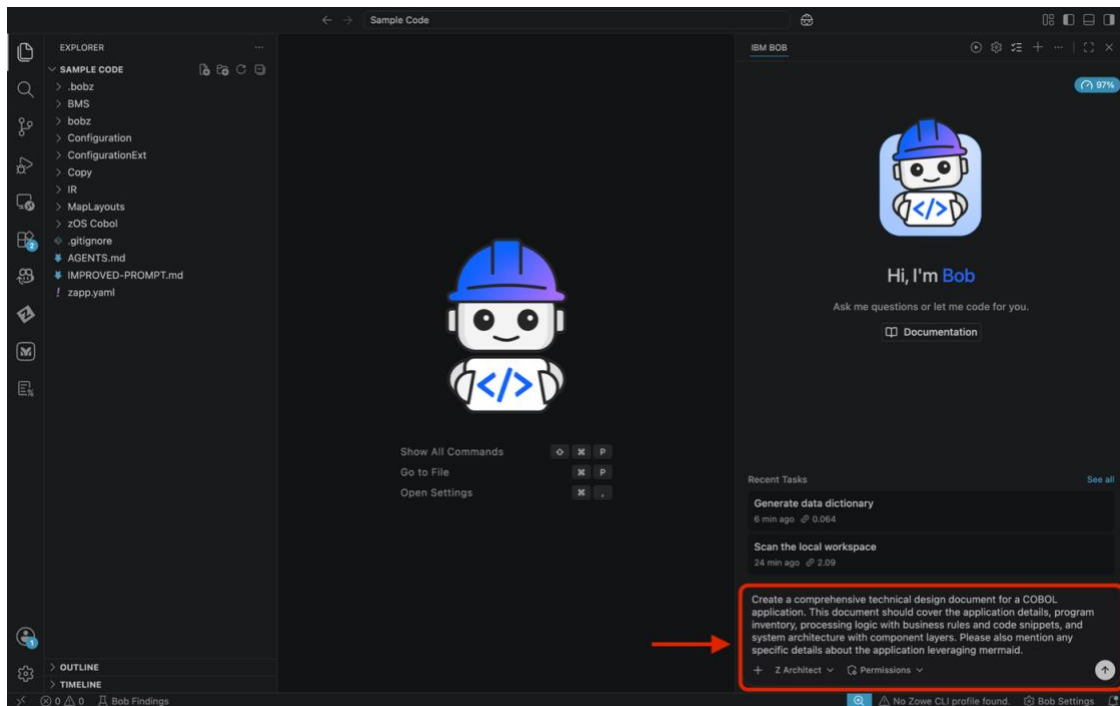
Learning Objectives

- Generate a complete technical design document from an existing COBOL codebase
 - Understand how Bob extracts business rules and explains them in plain language
 - Produce architecture diagrams automatically from code analysis
 - Create a document suitable for technical and non-technical stakeholders
-

Actions

1. Ensure you are in **Z Architect** mode and in the chat, paste the following prompt:

Create a comprehensive technical design document for a COBOL application. This document should cover the application details, program inventory, processing logic with business rules and code snippets, and system architecture with component layers. Please also mention any specific details about the application leveraging mermaid.



Technical design prompt in Z Architect mode

2. If auto-approvals are off, click **Approve** for each step or turn some/all approvals on.

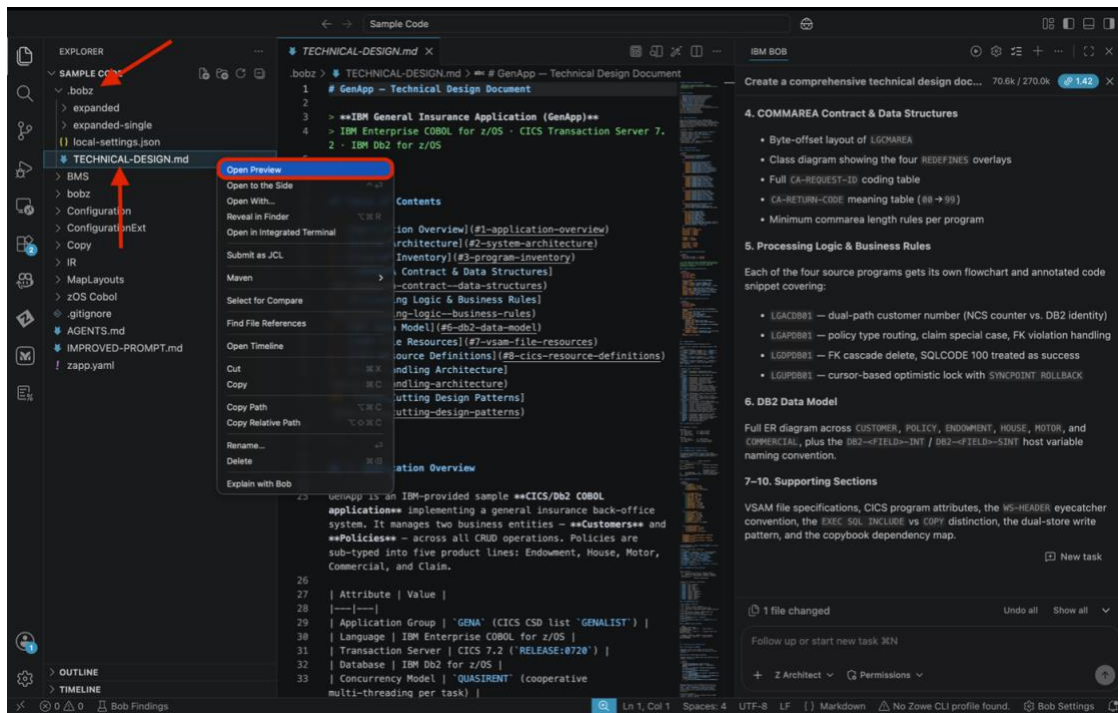
Tip

During generation, you may see that Bob creates a to-do list where it will create an action plan of what it's going to do.

3. Generation typically takes 3–5 minutes. **Please note**, the name of your document may not be the exact same and may take a few minutes to complete. This document is fully editable, so you can make any changes in the chat at any time. When complete, use the **Preview** option to view the generated markdown document.

Tip

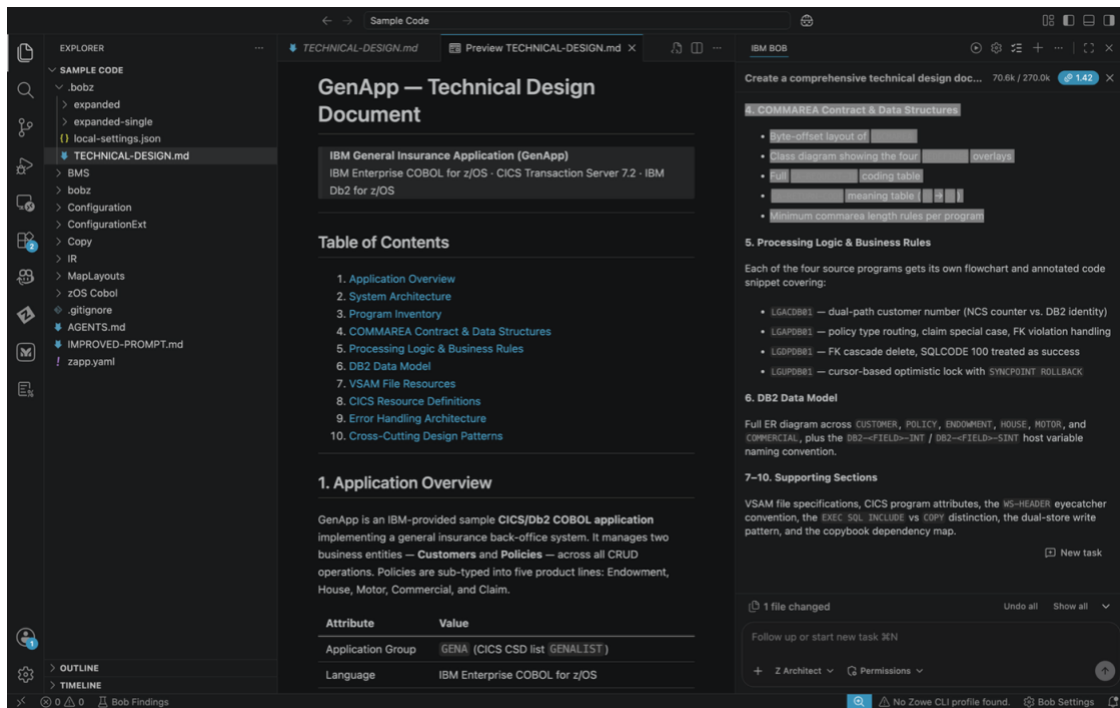
The generated file will be located in `.bobz/<File Name>`. In the following example the document is named `Technical-Design.md` — your file may have a different name.



Generated Technical Design markdown file in the Explorer panel

Expected Results

- ✓ Comprehensive technical design document created
- ✓ All programs listed with descriptions
- ✓ Business rules extracted and explained in plain English
- ✓ Architecture diagrams generated (Mermaid)
- ✓ Code snippets included for key logic
- ✓ Component relationships visualized



Preview of the generated technical design document

Key Takeaways

- **Literate Coding:** Bob explains COBOL in natural language — accessible to non-technical stakeholders
- **Visual Documentation:** Mermaid diagrams generated automatically from code analysis
- **Multi-Mode Intelligence:** Z Architect mode specializes in analysis and documentation tasks

Start a New Chat

Please select the + sign at the top of the chat window to start a new session before moving to the next lab.

Lab 3: Impact Analysis

Prerequisites: Complete Lab 1 (Lab Setup) before starting this lab

Code Set: Any COBOL workspace — recommended code: Sample Code

Duration: 30 minutes

Difficulty: Beginner

Overview

In this lab, you'll perform comprehensive impact analysis when changing copybook fields or program logic — understanding the ripple effects of a change across your entire mainframe application landscape.

Learning Objectives

By the end of this lab, you will be able to:

- Analyze the impact of changing copybook fields
 - Generate comprehensive impact analysis documentation
 - Understand change propagation across programs
 - Identify all affected programs and modules
 - Document implementation approaches for changes
 - Plan safe change rollout strategies
-

Exercise 1: Analyze Field Change Impact

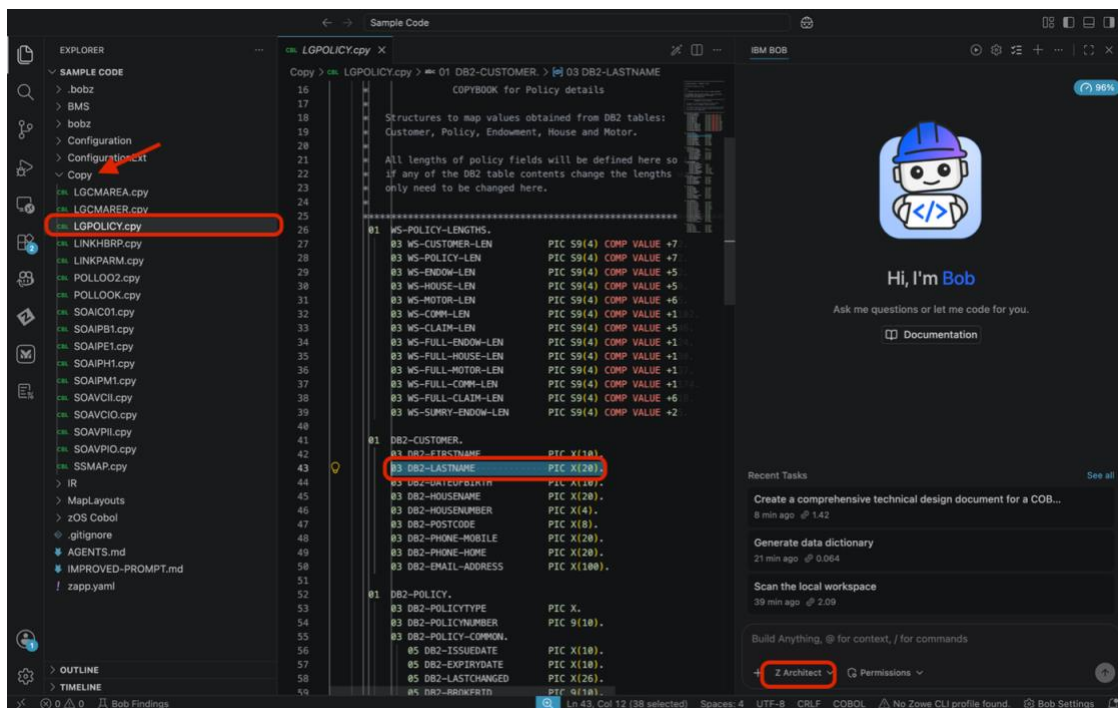
Before You Begin

Ensure you are in **Z Architect** mode before starting this exercise.

1. Locate the Target Copybook

In the file explorer, navigate to the Copy folder. Open the LGPOLICY copybook (double-click). This copybook typically contains policy-related data structures. Locate the DB2-LASTNAME field — this field stores customer last names with a maximum of 20 characters.

```
03 DB2-LASTNAME      PIC X(20) .
```



LGPOLICY copybook with DB2-LASTNAME field highlighted

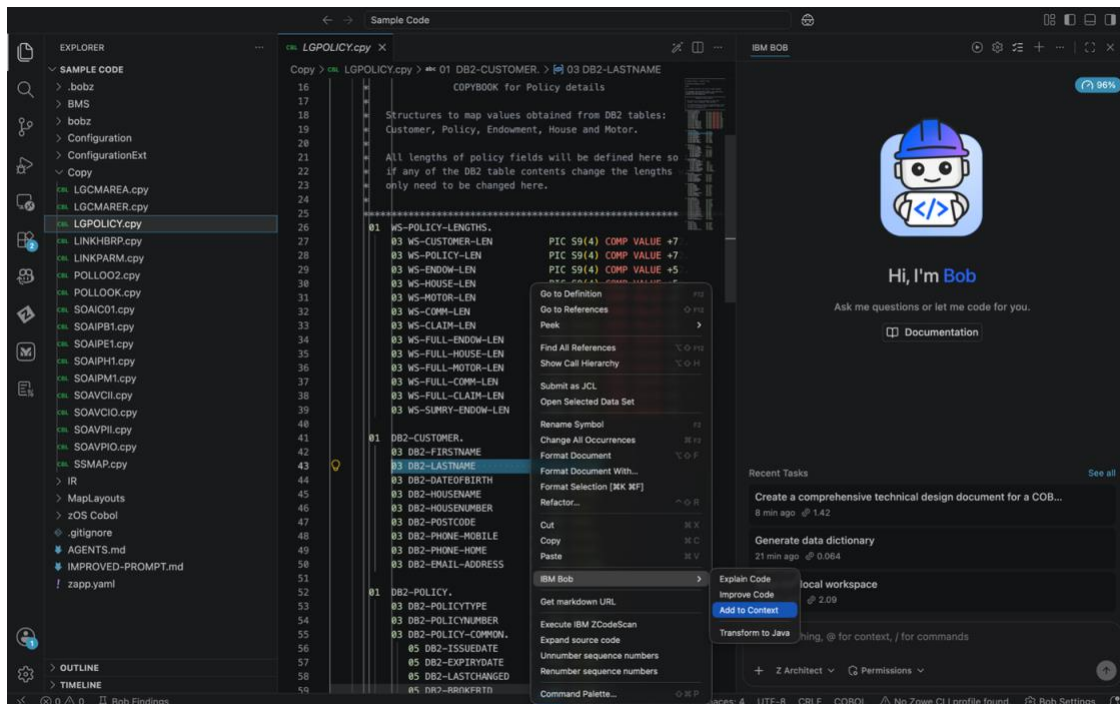
2. Add Field to Context

- Highlight the field definition line
- Right-click and select **IBM Bob** → **Add to Context**
- Verify the field appears in the chat context window
- You should see something like:

LGPOLICY.cpy:43

Tip

Please ensure you only highlight `03 DB2-FIRSTNAME PIC X(10)`. Do not include any spaces before or after.

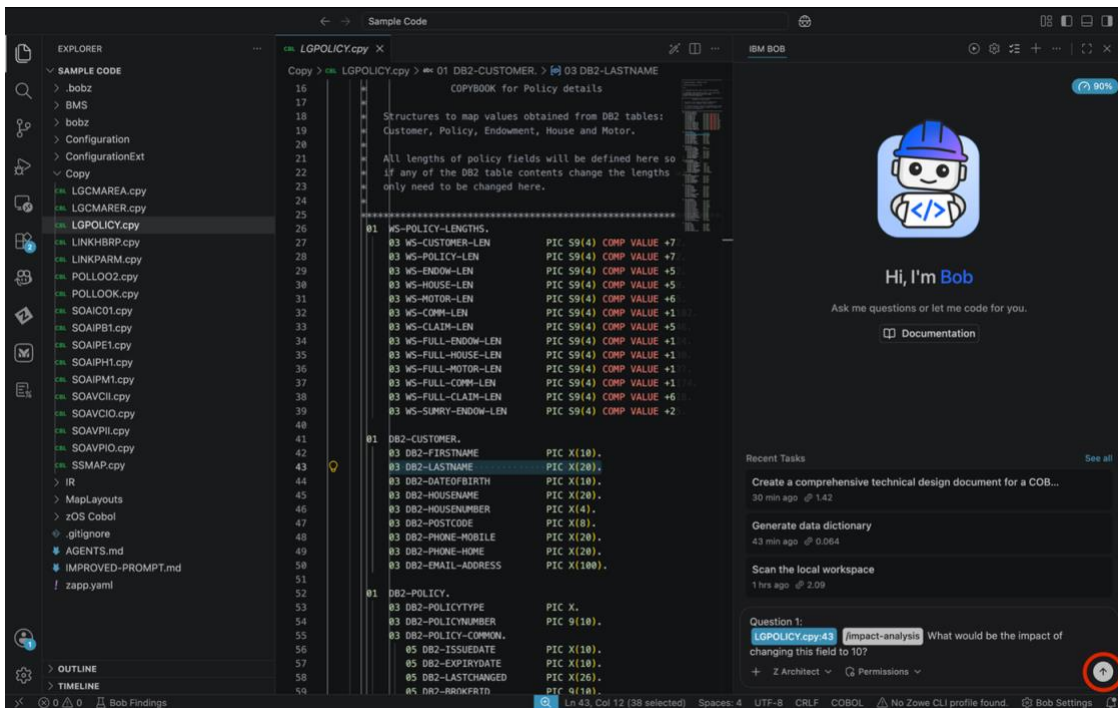


Field added to context in the chat window

3. Use / Commands

Bob is loaded with / commands that allow you to leverage prebuilt prompts. In this case, you will use the `/impact-analysis` command to generate a comprehensive impact analysis report for the field you added to context. In the chat, type:

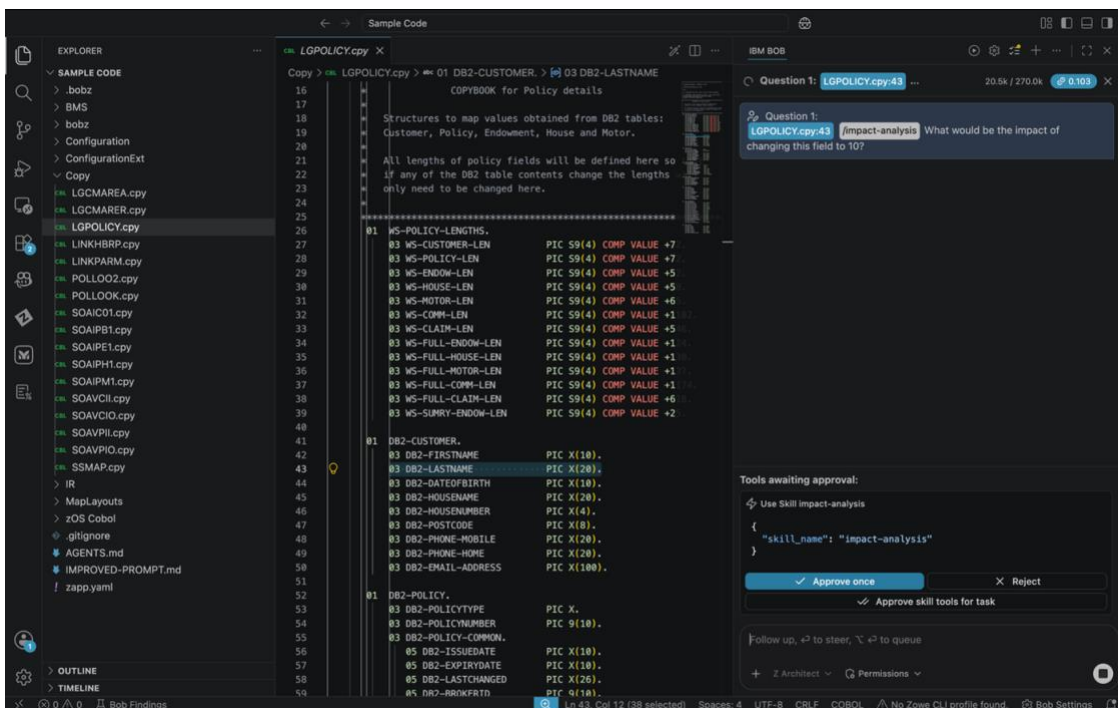
LGPOLICY.cpy:43 `/impact-analysis` What would be the impact of changing this field to 10?



/impact-analysis command entered in the chat

4. Approve the Request

If auto-approvals are off, click **Approve** for each step or turn some/all approvals on.



Approval request for impact analysis tool

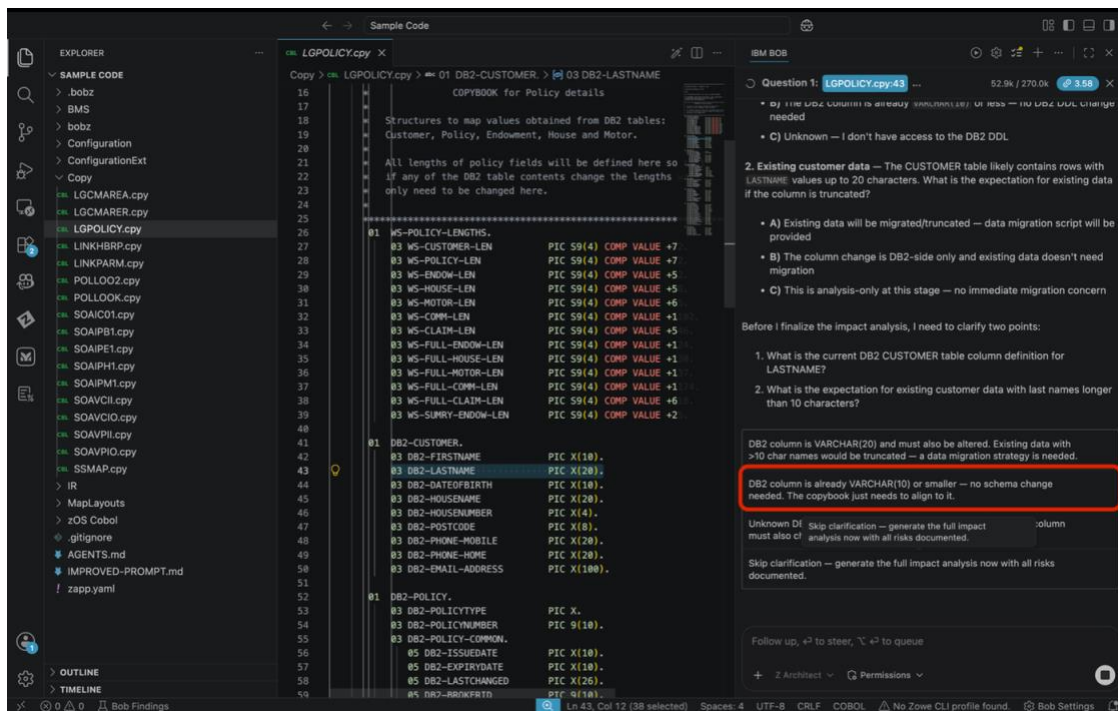
5. Bob Asks for Clarification

Bob will begin the flow of reviewing data and then prompt you with a question to determine what type of impact analysis you want. For the purposes of this lab, select the last one:

DB2 column is already VARCHAR(10) or smaller - no schema change needed.
The copybook just needs to align to it.

Tip

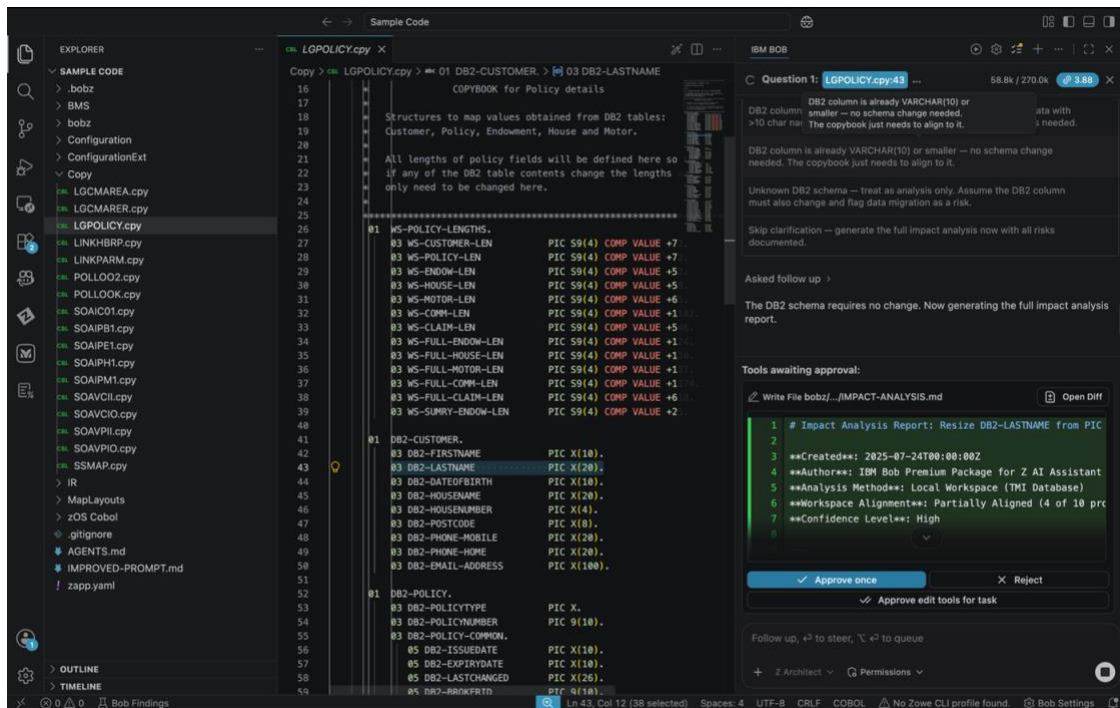
If you do not see this option, select one similar — or you can type a response instead of selecting an option if ever needed.



Bob clarification prompt with option selection

6. Bob Generates the Impact Analysis

As Bob finishes its analysis, you will see an `Impact-Analysis.md` file being generated in real time. Please select save or approve.



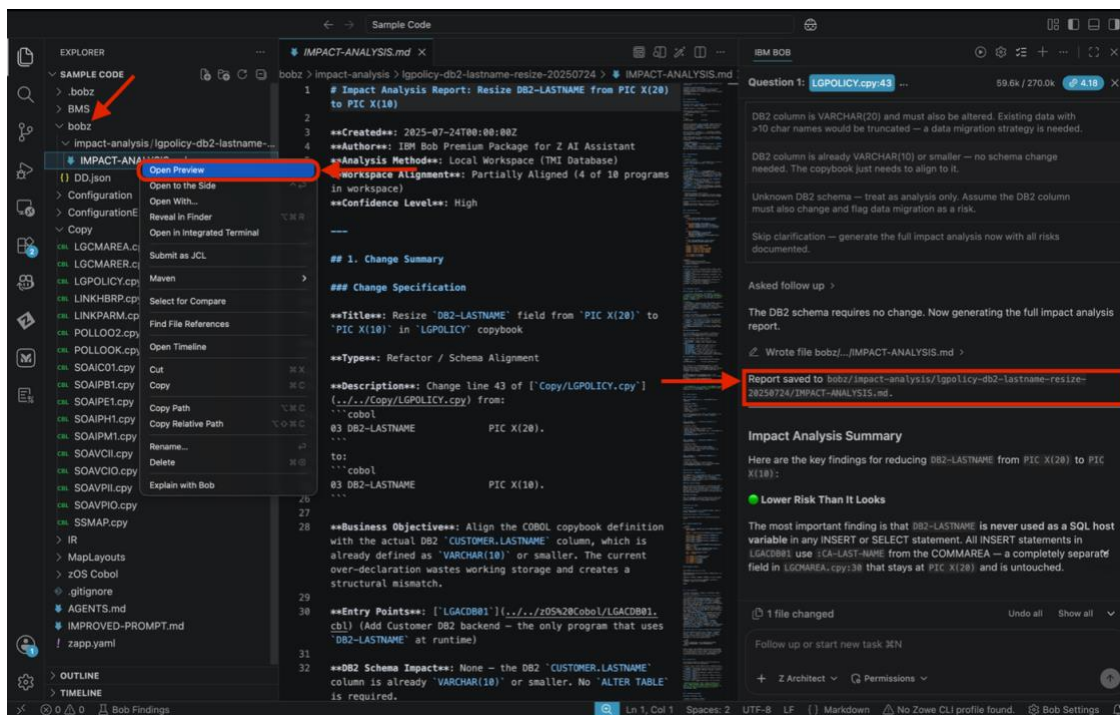
Impact-Analysis.md being generated in real time

7. Preview the Impact Analysis

Once the analysis is complete, you can right-click the Impact-Analysis.md file in the file explorer and select **Preview** to view the generated markdown document.

Tip

Within the chat window, the location of where the report was saved to may vary. For instance, in the following example it was generated in bobz/impact-analysis/IMPACT-ANALYSIS.md.



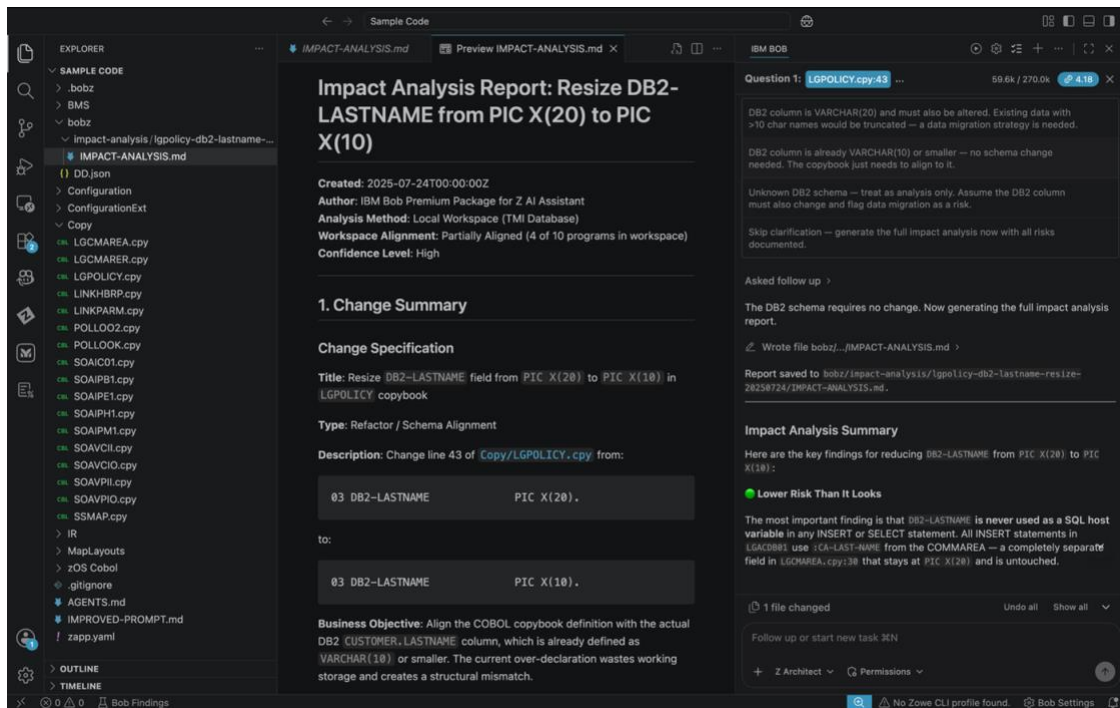
Preview of the generated Impact Analysis document

Expected Results of Exercise 1

Once the **Impact-Analysis.md** preview is open, take time to carefully review the full report before proceeding to implementation. The document is structured to show you exactly what is at risk when the field size is reduced from 20 to 10 characters. Pay particular attention to the following sections within the report:

- **Scope:** the total number of programs and copybooks affected by the change
- **Risk Level:** Bob's assessment of how significant the change is (e.g., High, Medium, Low)
- **Affected Components:** a list of every program that references DB2-LASTNAME either directly or via the LGPOLICY copybook
- **Recommended Implementation Sequence:** the suggested order in which files should be updated to minimize the chance of breaking dependencies during the change

Keep this document open or note the affected file paths, as you will reference them in the next exercise when implementing the change.



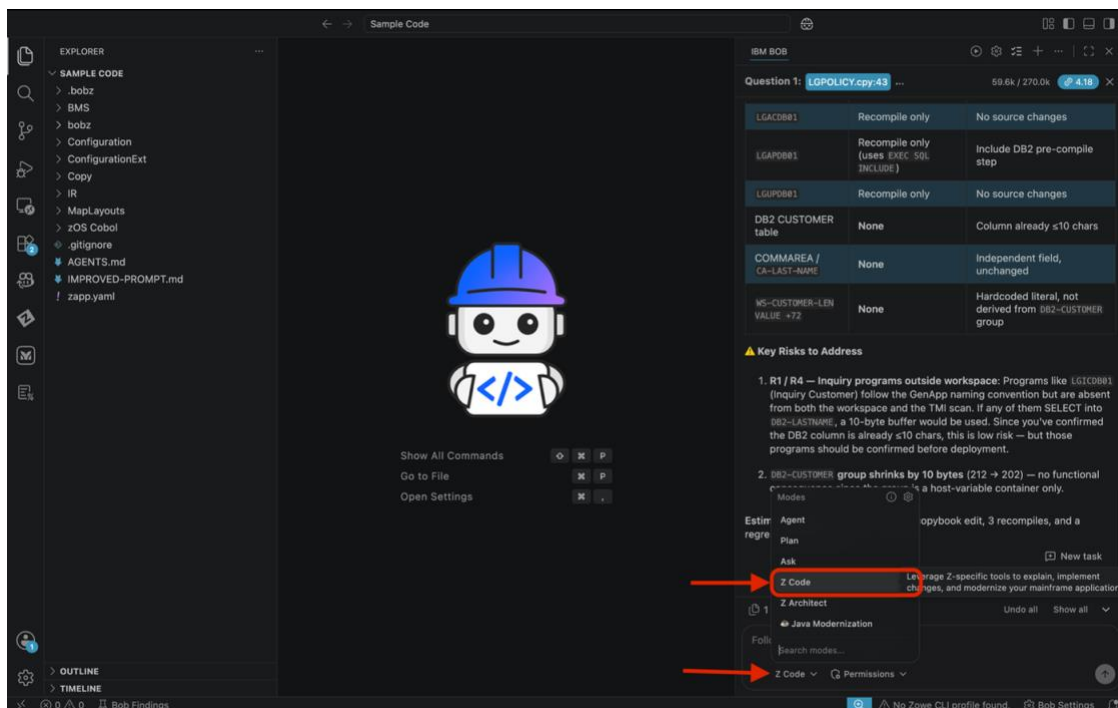
Impact Analysis report showing scope and affected components

Exercise 2: Implement the Change

Take the Impact-Analysis.md report Bob generated and use it to drive real code changes — having Bob apply every edit across all affected programs and produce a DB2 migration script, all from a single prompt.

1. Switch to Z Code Mode

Click the mode selector and switch from **Z Architect** to **Z Code** mode. Z Code mode is optimized for making and reviewing direct code changes.



Switching from Z Architect to Z Code mode

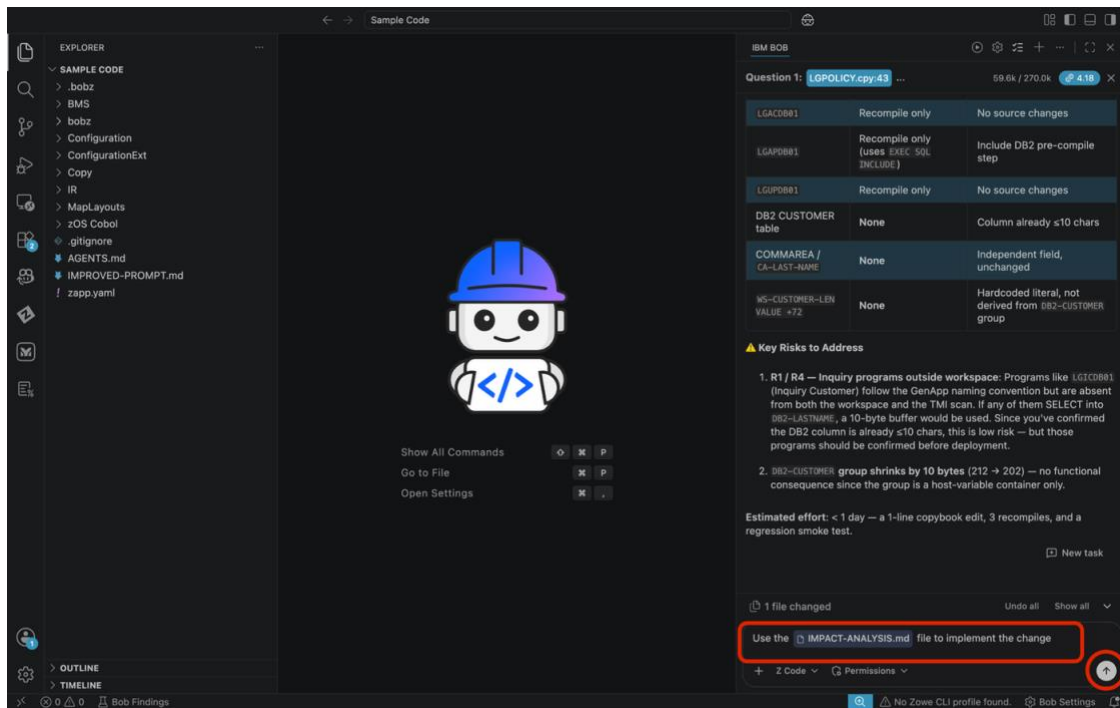
2. Reference the Impact Report

In the chat, enter the following prompt — replacing `<path to file>` with the actual file path of the `Impact-Analysis.md` generated in the previous exercise. You can copy the path by right-clicking the file in the file explorer and selecting **Copy Path**:

Use the `<path to file>` file to implement the change

Tip

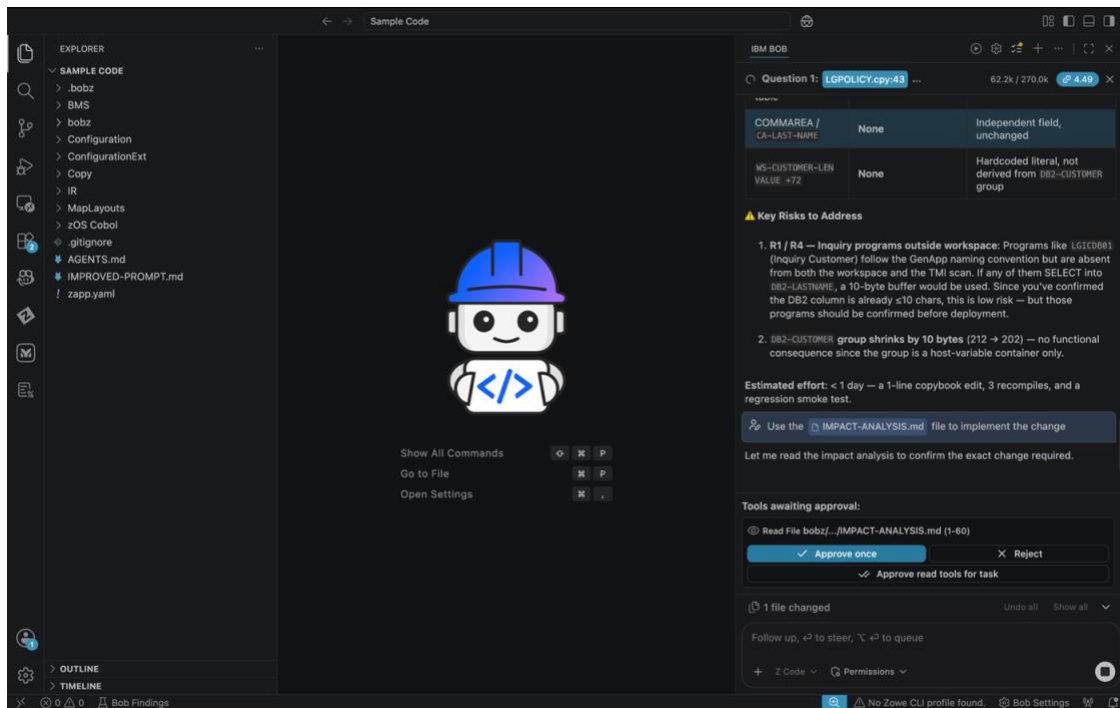
You can also type `@` in the chat bar and select the file directly from the explorer — this inserts the correct path without copying it manually.



Impact report path entered in the chat prompt

3. Approve Each Change

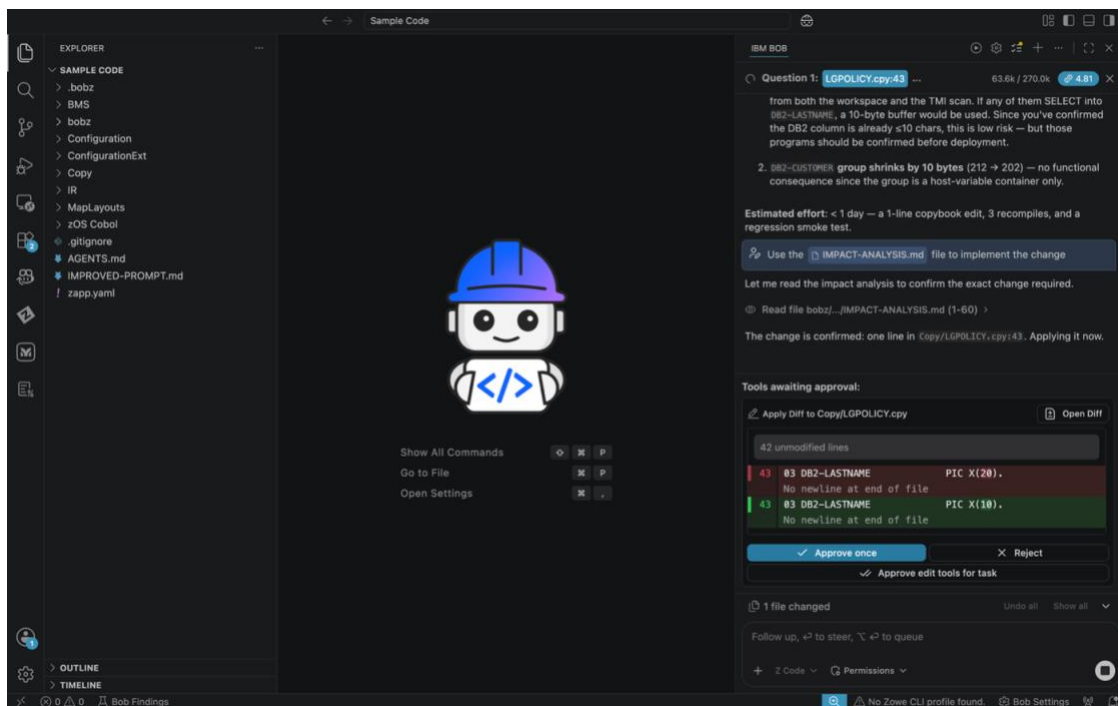
If Auto Approve is off, you will be prompted to approve each individual change as Bob works through the affected files. Click **Approve** for each request.



Approval requests as Bob applies changes

As Bob applies each change, you will see a diff view of the code in the chat window. This allows you to review every change before it is applied to the codebase. You can also click on the file name in the diff view to open the file in the editor and see the changes in context.

- **Green lines** — additions or new content
- **Red lines** — removals or replaced content



Diff view showing green additions and red removals

Expected Results of Exercise 2

Once all changes have been applied, Bob will generate database migration scripts. Within the chat you will see a summary of the changes made. Please review to see what changes were made.

- ☒ All affected COBOL programs updated
- ☒ Copybook field definition changed
- ☒ Validation logic updated in affected programs
- ☒ DB2 migration scripts generated
- ☒ Summary of all changes produced

Tip

After Bob completes the task you can select **files changed** and select a code file to see the changes made too.

Key Takeaways

- How to select fields for impact analysis
- Generating comprehensive impact reports
- Understanding change propagation
- Identifying all affected components
- Creating safe implementation plans

- Using the impact report to drive actual code changes with Bob

Start a New Chat

Please select the + sign at the top of the chat window to start a new session before moving to the next lab.

Lab 4: Refactoring & Service Extraction

Prerequisites: Complete Lab 1 (Lab Setup) before starting this lab.

Code Set: Any COBOL workspace — recommended code: Sample Code

Duration: 60 minutes

Difficulty: Intermediate

Overview

In this use case, you'll identify business rules in monolithic COBOL programs, check dependencies, and extract business services for modernization — transforming legacy mainframe applications toward service-oriented architectures.

Learning Objectives

By the end of this use case, you will be able to:

- Identify business rules functionality in monolithic COBOL programs
 - Check dependencies before refactoring code
 - Extract business services from COBOL programs
 - Create REST API wrappers for extracted services
 - Integrate refactored services with main programs
 - Understand service-oriented architecture patterns for mainframe modernization
-

Exercise: Identify Business Rules Functionality

Learn to identify which parts of a COBOL program contain pure business logic that can be extracted into services, versus infrastructure code that should remain in the main program.

Before You Begin

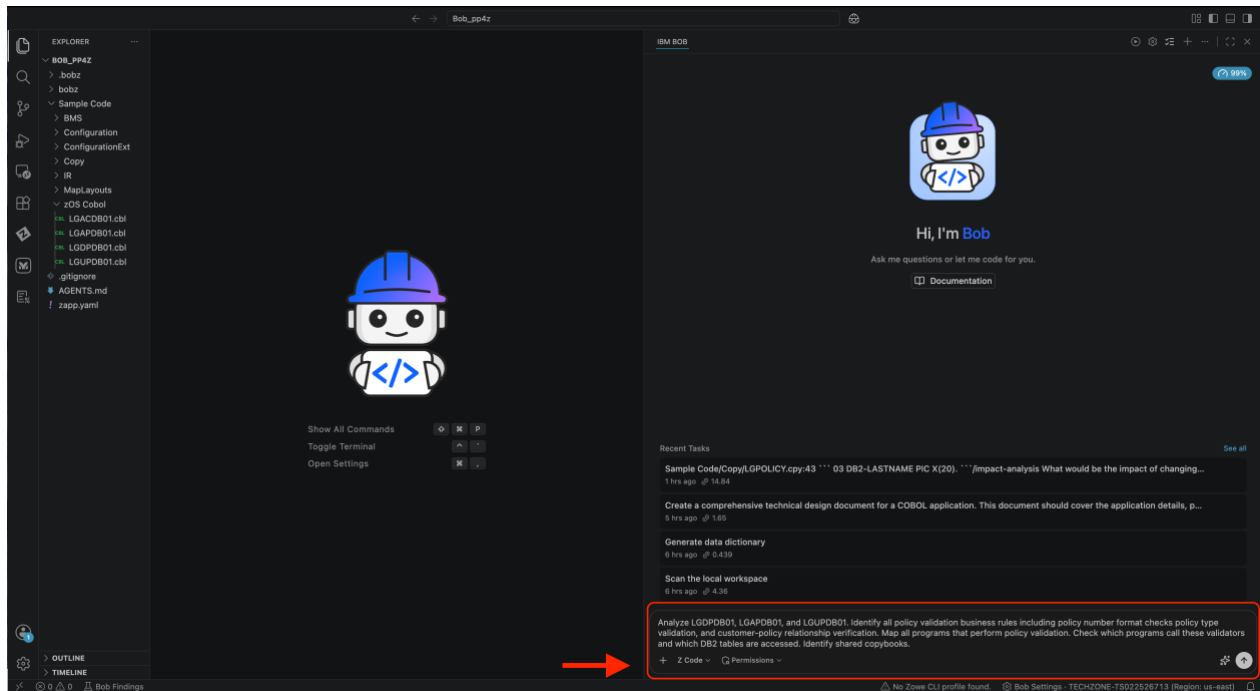
Ensure you are in **Z Code** mode before starting this exercise.

Refactor workflow selection

1. Analyze for Refactor Candidates

ACTION: Enter the following prompt into the chat window:

Analyze LGDPDB01, LGAPDB01, LGACDB01, and LGUPDB01. Identify all policy validation business rules including policy number format checks policy type validation, and customer-policy relationship verification. Map all programs that perform policy validation. Check which programs call these validators and which DB2 tables are accessed. Identify shared copybooks.

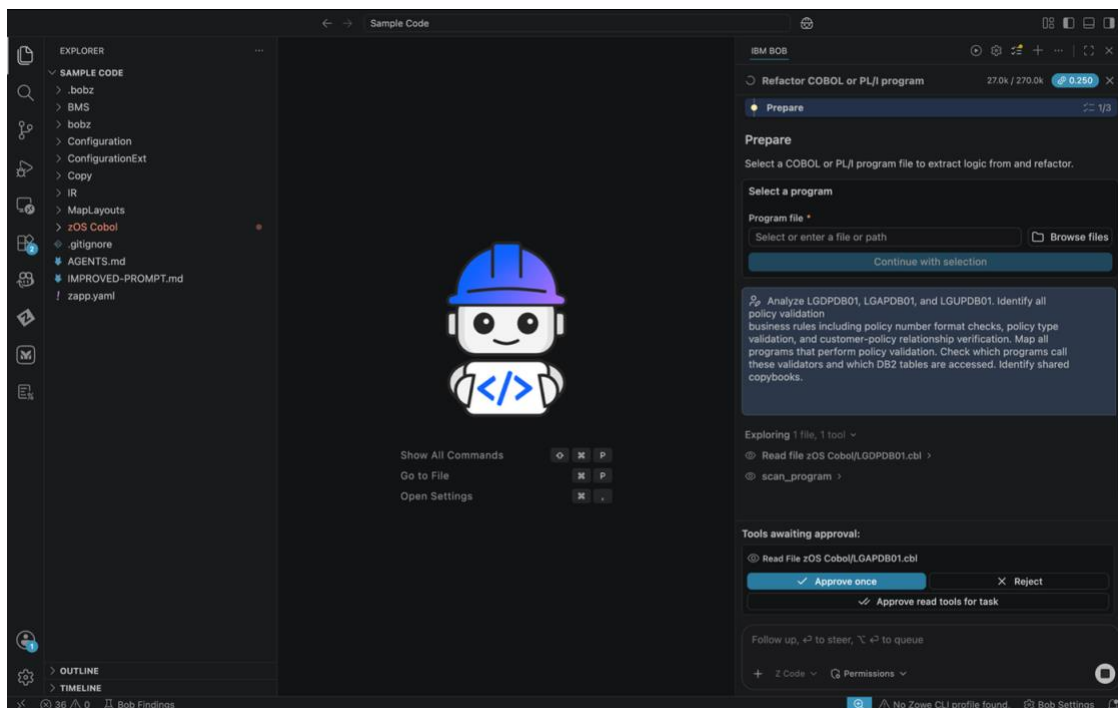


Refactor workflow prompt entry

3. Review the Analysis Results

Bob will analyze multiple COBOL programs and automatically generating a structured task list. The tool will identify key business rule areas such as policy validation, customer relationships, and data integrity checks. Users can review the generated tasks and approve them to proceed with deeper analysis, ensuring alignment before execution.

ACTION: Please continue to approve any requests for skills or various requests unless otherwise noted.



Analysis results and approval

Expected Results

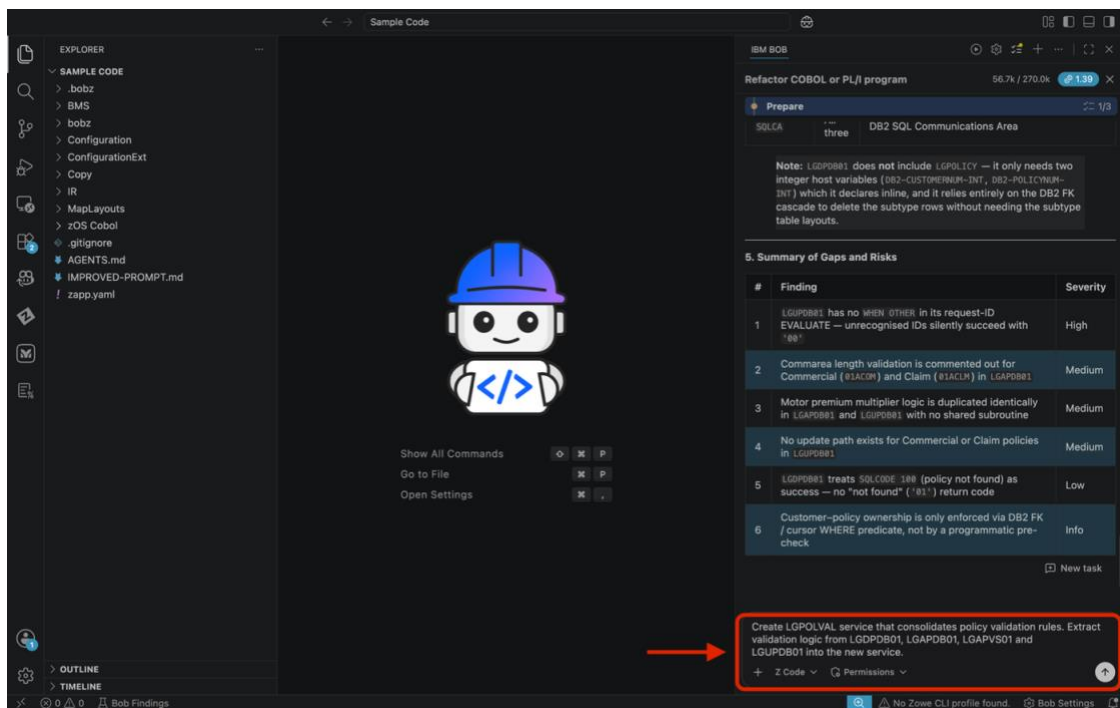
This example shows Bob completing the analysis and generating structured business rule documentation. Based on multiple COBOL programs, it consolidates findings into a clear, organized output, highlighting validation logic, data formats, and program relationships. This allows users to quickly understand core business rules and supports downstream tasks like modernization and impact analysis. You may review through a generated markdown or within the chat window on the right.

Exercise: Refactor Business Services

Before refactoring, it's critical to understand what dependencies exist. This prevents breaking changes and helps plan the extraction strategy.

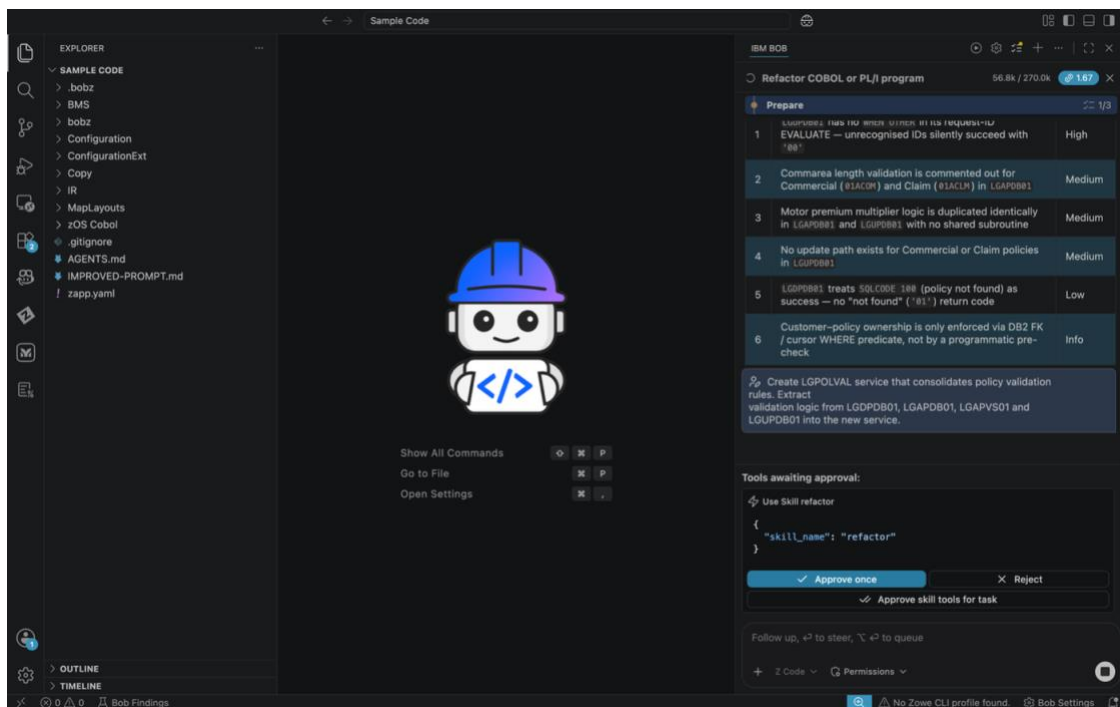
1. **ACTION:** Take the following prompt and paste it into the chat window:

Create LGPOLVAL service that consolidates policy validation rules. Extract validation logic from LGDPDB01, LGAPDB01, LGACDB01, and LGUPDB01 in to the new service.



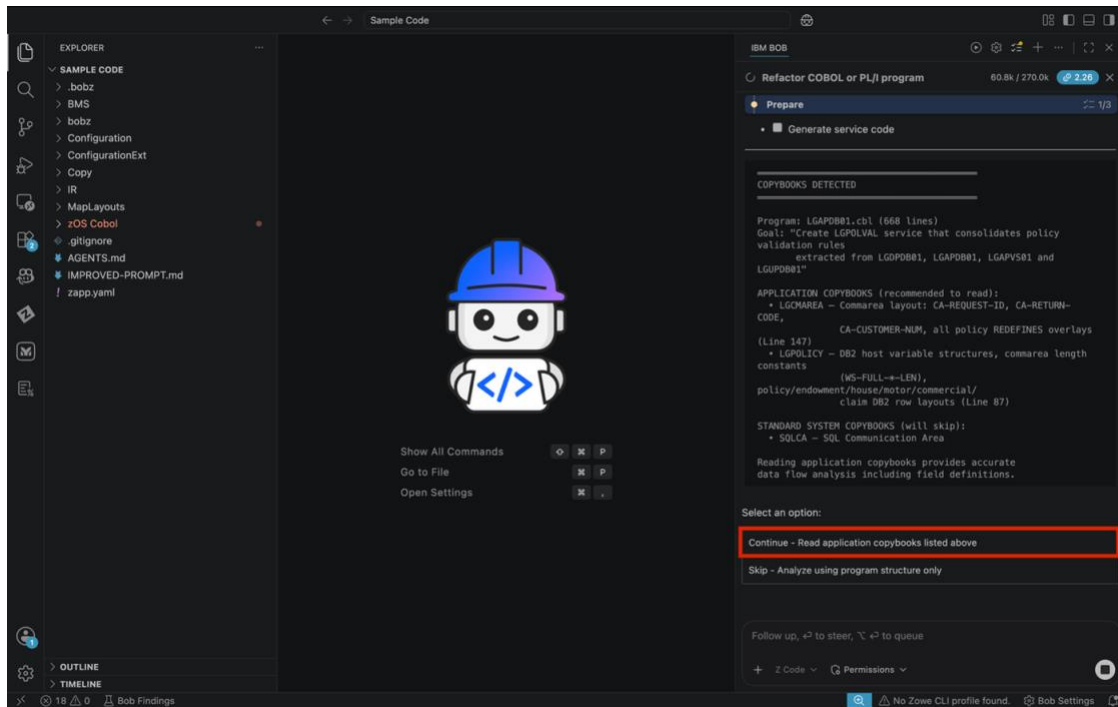
Refactor service prompt

2. **ACTION:** You will be prompted to approve the request. Please click **Approve** to continue, or you can select **Auto Approve** to approve all requests automatically.



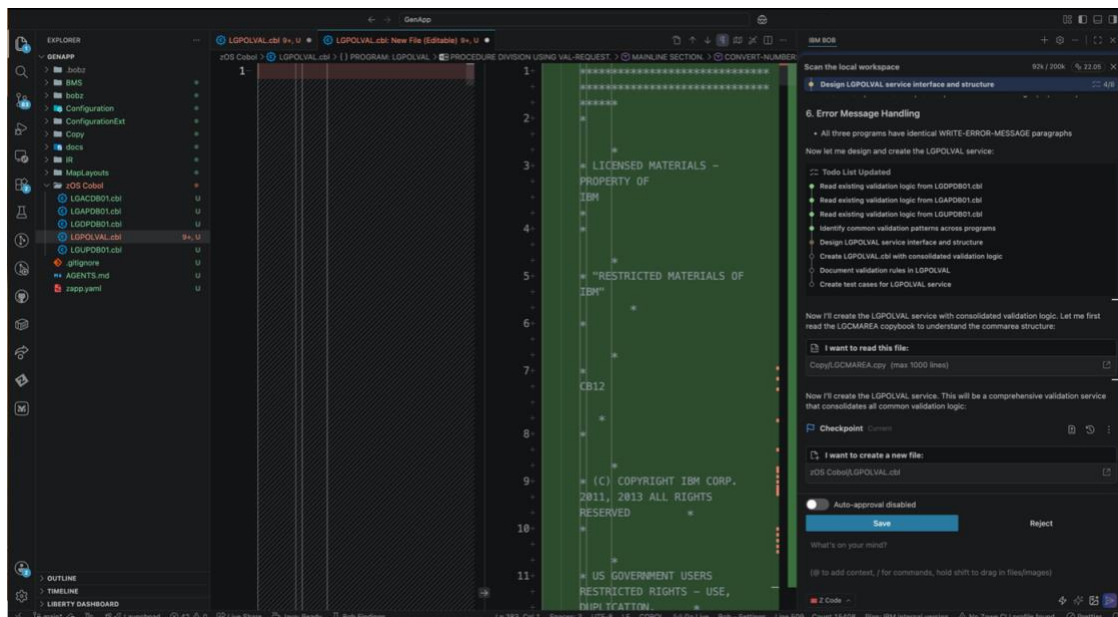
Approve refactor request

3. **ACTION:** If prompted please select “Continue – Read application copybooks listed above”.



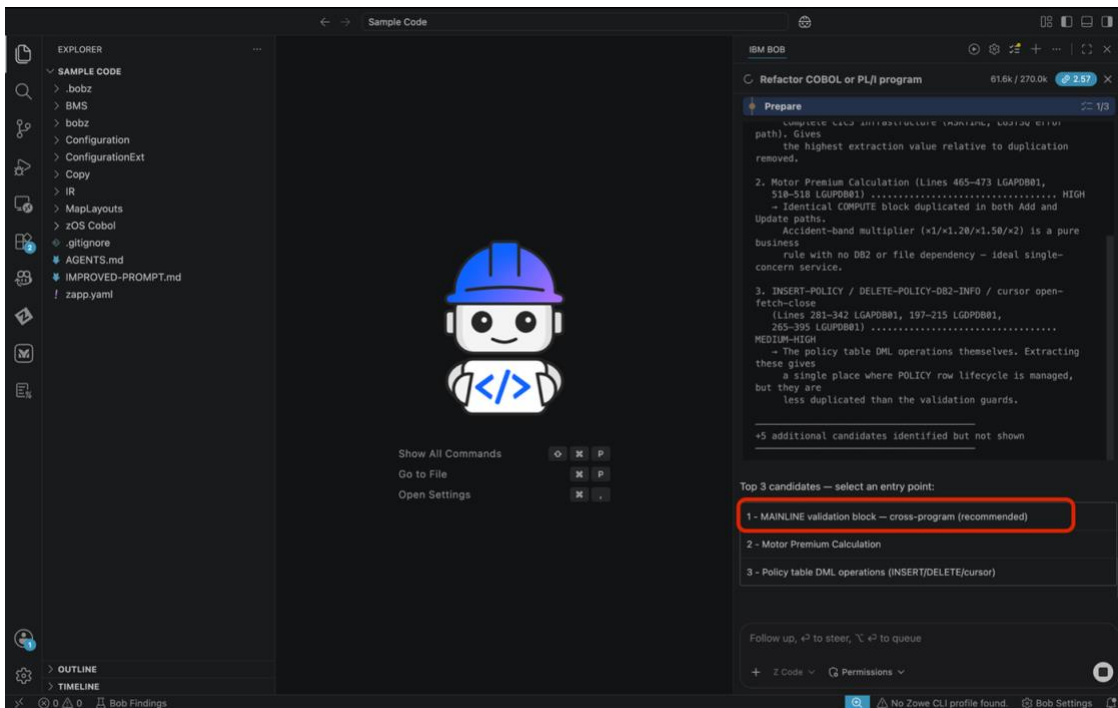
Continue with copybooks

4. The generation of the program will begin. Please wait for this to complete.



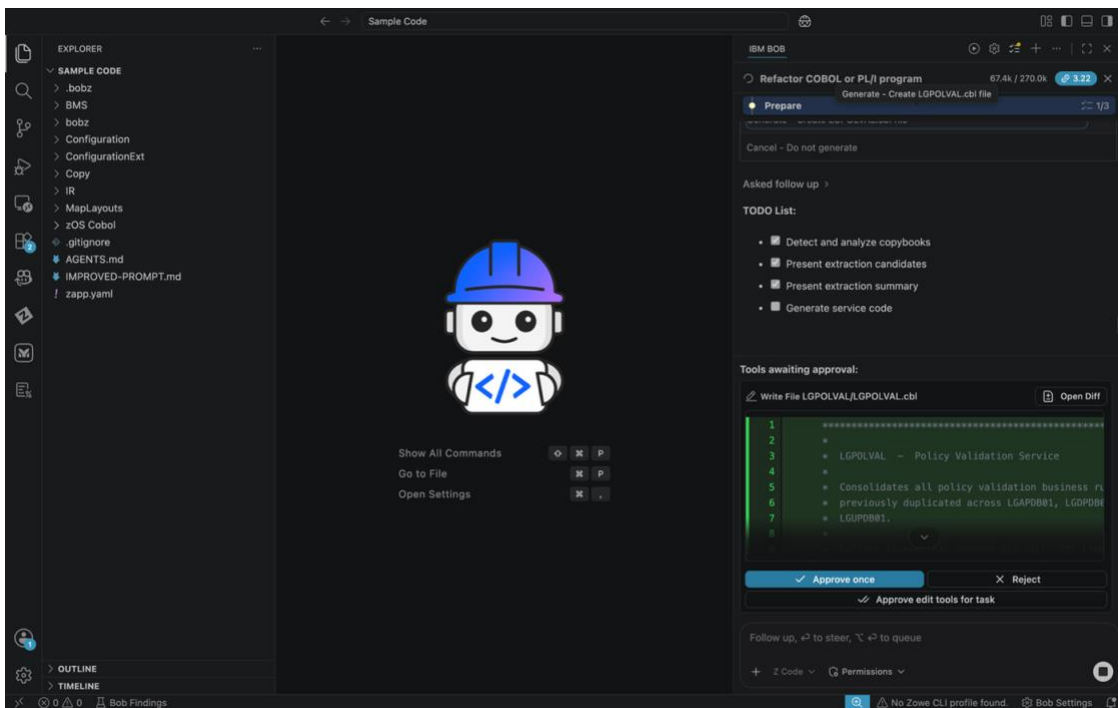
Program generation in progress

5. **ACTION:** Select the “1 – Mainline validation block – cross- program (recommended). If prompted please also select the”Generate – Create LGPOLVAL.cbl”



Select mainline validation block

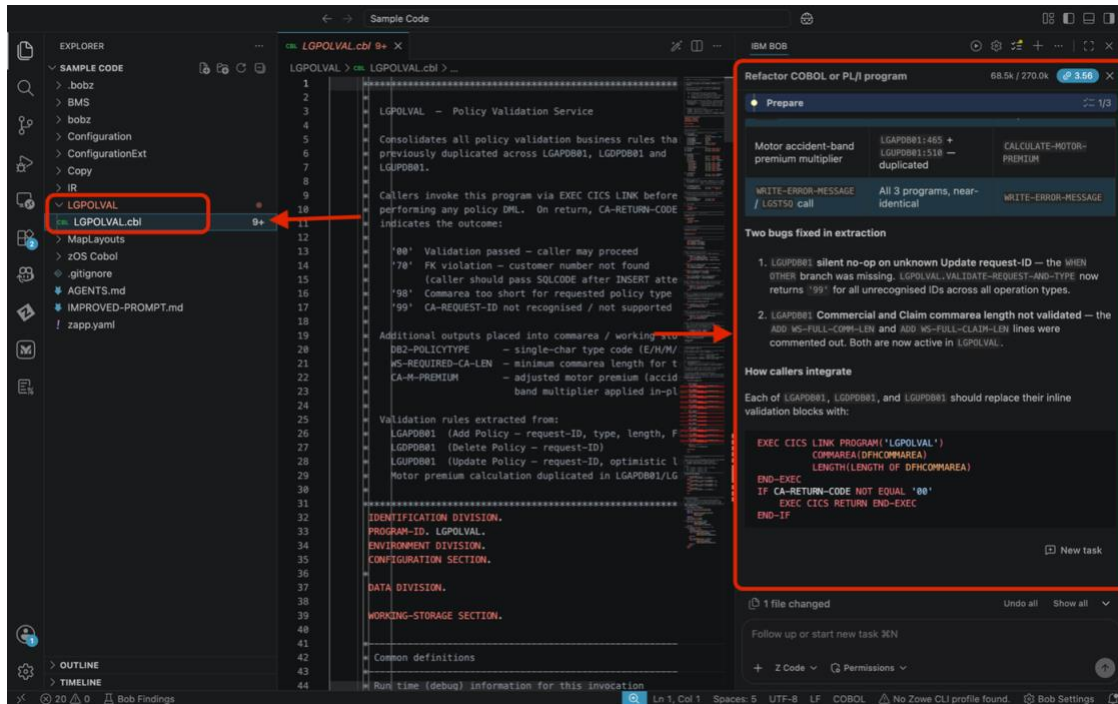
6. **ACTION:** Please select the approve option during generation



Approve during generation

7. Please read through the generated information in the chat window. Locate the generated code for review.

Example: C:/Users/<YourUserName>/Documents/Bob/Sample Code/zOS
Cobol/LGPOLVAL.cb1



Generated LGPOLVAL code

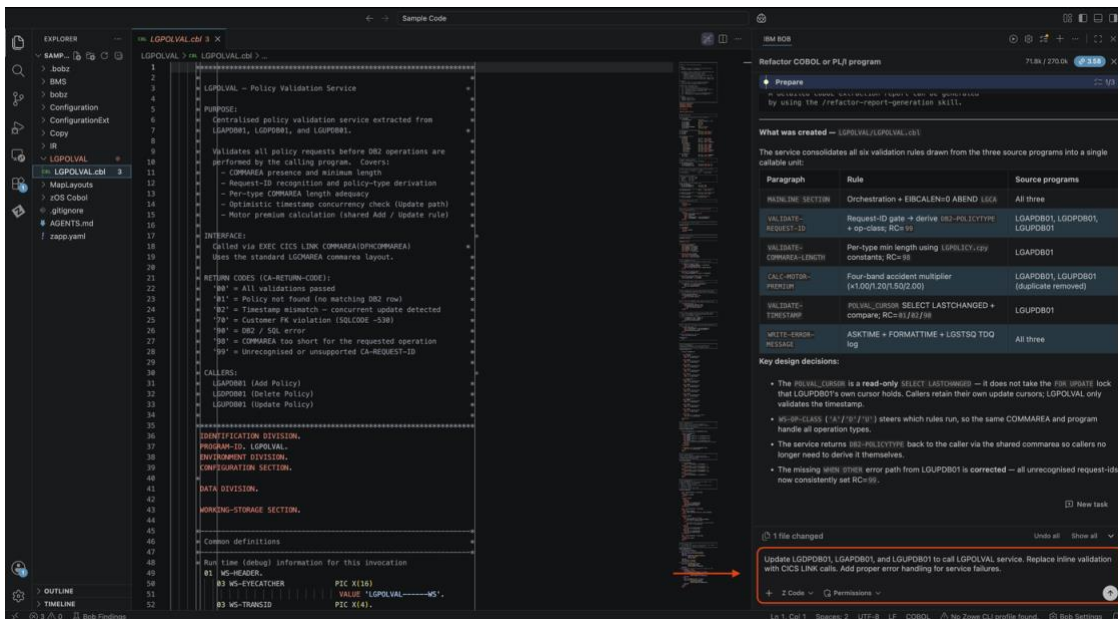
Expected Results

- ☒ Business rules identified and documented
- ☒ Dependencies checked and mapped
- ☒ COBOL service program generated with extracted business logic

Exercise: Add Link Code to Main Module

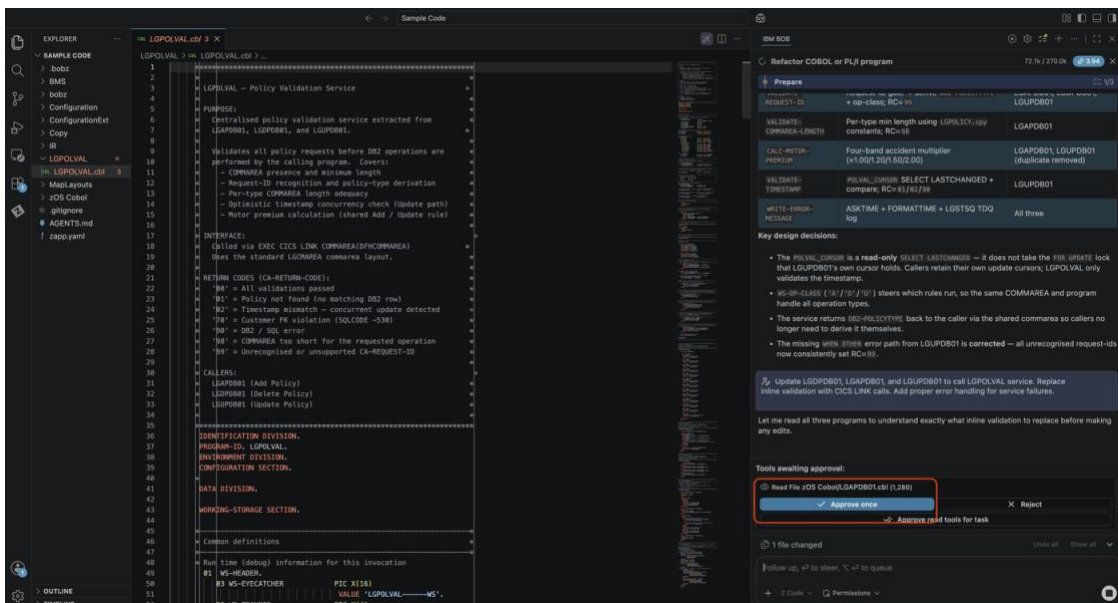
1. **ACTION:** Please paste the following prompt on the chat window on the right as shown in the image below.

Update LGDPDB01, LGAPDB01, and LGUPDB01 to call LGPOLVAL service. Replace inline validation with CICS LINK calls. Add proper error handling for service failures.



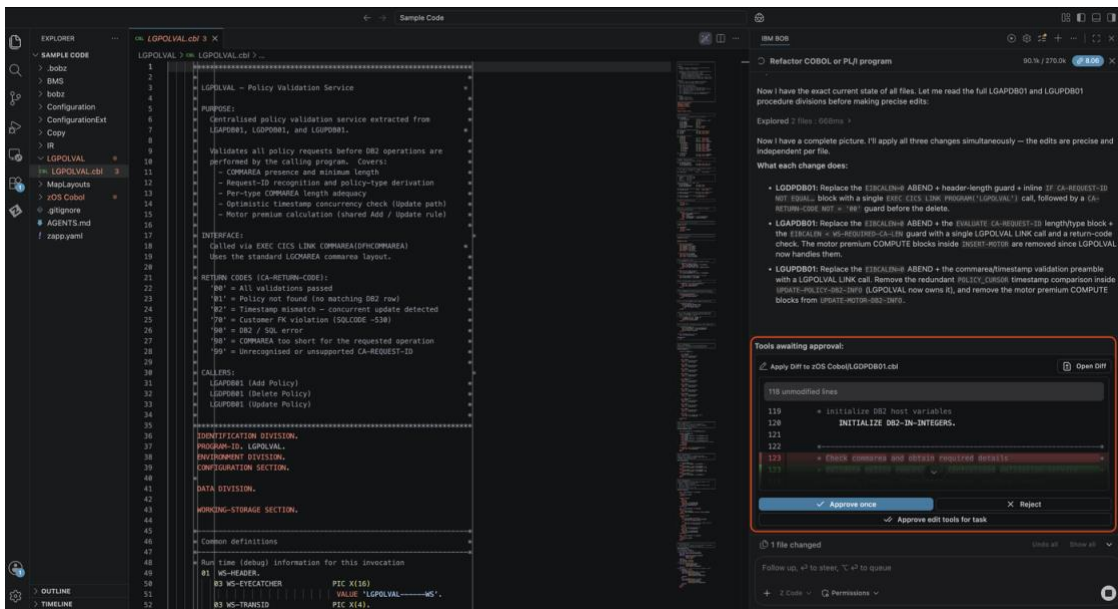
Link code prompt

2. ACTION: When prompted please continue to select approve unless otherwise noted.



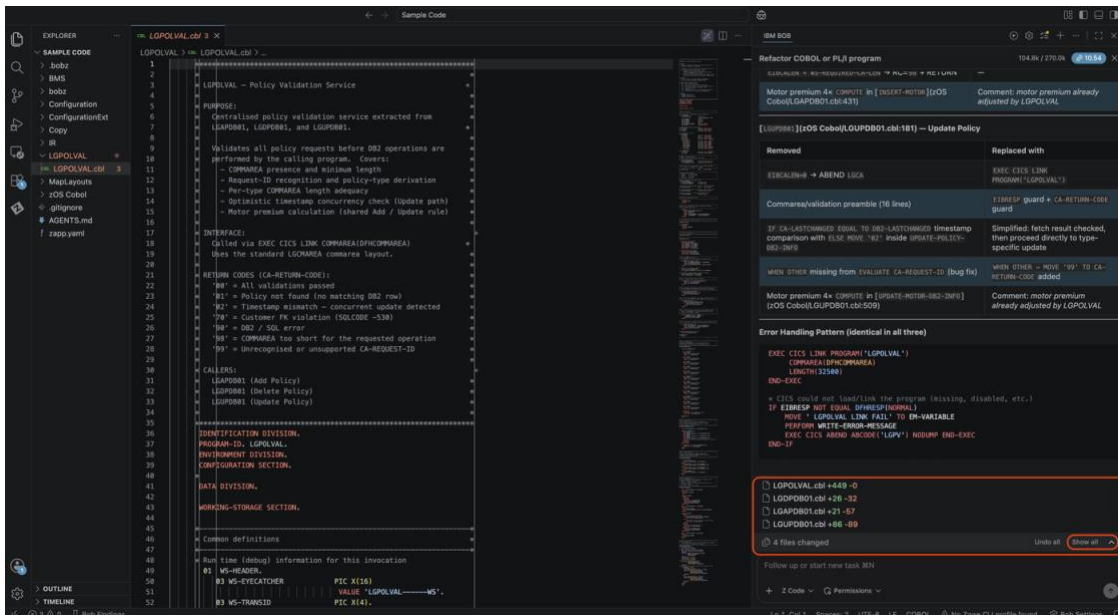
Approve link code changes

3. You'll be able to see the exact changes while the to do list completes. Please continue to select the approve option while Bob works through the To Do List.



Changes in progress

4. Please review the completed changes!



Completed refactoring changes

Expected Results

- ✓ Main programs updated to call service
- ✓ Original business logic removed
- ✓ Parameter passing implemented correctly
- ✓ Error handling maintained

Key Takeaways

- How to identify extractable business rules in COBOL
- Checking dependencies before refactoring
- Extracting business services from monolithic code
- Creating REST API wrappers for COBOL services
- Integrating refactored services with main programs
- Service-oriented architecture patterns for mainframes

Start a New Chat

Please select the + sign at the top of the chat window to start a new session before moving to the next lab.

Lab 5: Spec-Driven COBOL Development

Prerequisites: Complete Lab 1 (Lab Setup) before starting this lab

Code Set: Any COBOL workspace — recommended code: `Sample Code`

Spec Sheet: `LGRENPOL-spec.md`

Duration: 45 minutes

Difficulty: Intermediate

Overview

By the end of this lab, you will be able to:

- Use a program specification document to prompt COBOL code generation
- Instruct Bob to derive application coding standards from the existing codebase before generating
- Generate a new CICS COBOL program that integrates with existing copybooks and called programs
- Verify generated code against a specification's requirements
- Validate syntax using Z Open Editor
- Identify and correct any gaps between the spec and the generated output

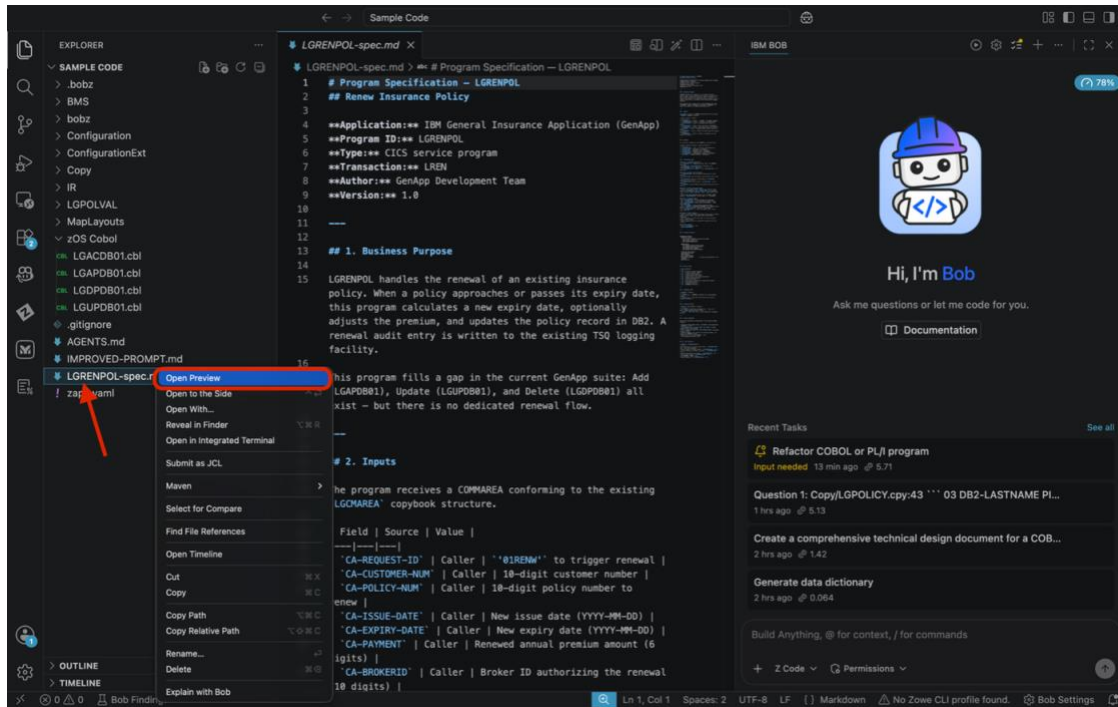
Before You Begin

ACTION: Open `LGRENPOL-spec.md` in the editor using preview

Example of where it would be located: `/Users/<username>/Downloads/PPFZ BOBATHON/Sample Code/LGRENPOL-spec.md`

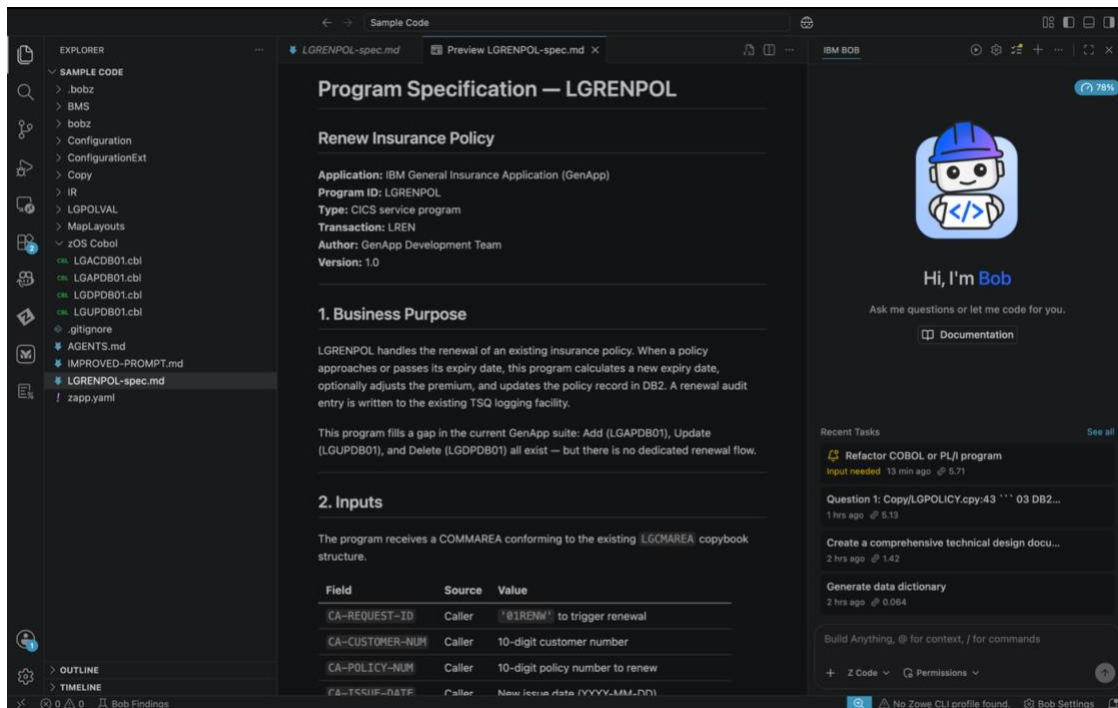
Note the key elements Bob will need to honor:

- Program ID: LGRENPOL, Transaction: LREN
- Uses the existing LGCMAREA.cpy and LGPOLICY.cpy copybooks
- Calls three existing programs: LGIPDB01, LGUPDB01, LGSTSQ
- 6-step processing flow with specific return codes
- Must follow GenApp naming and error-handling conventions



Spec document in preview

2. Review the document to understand the requirements and constraints.

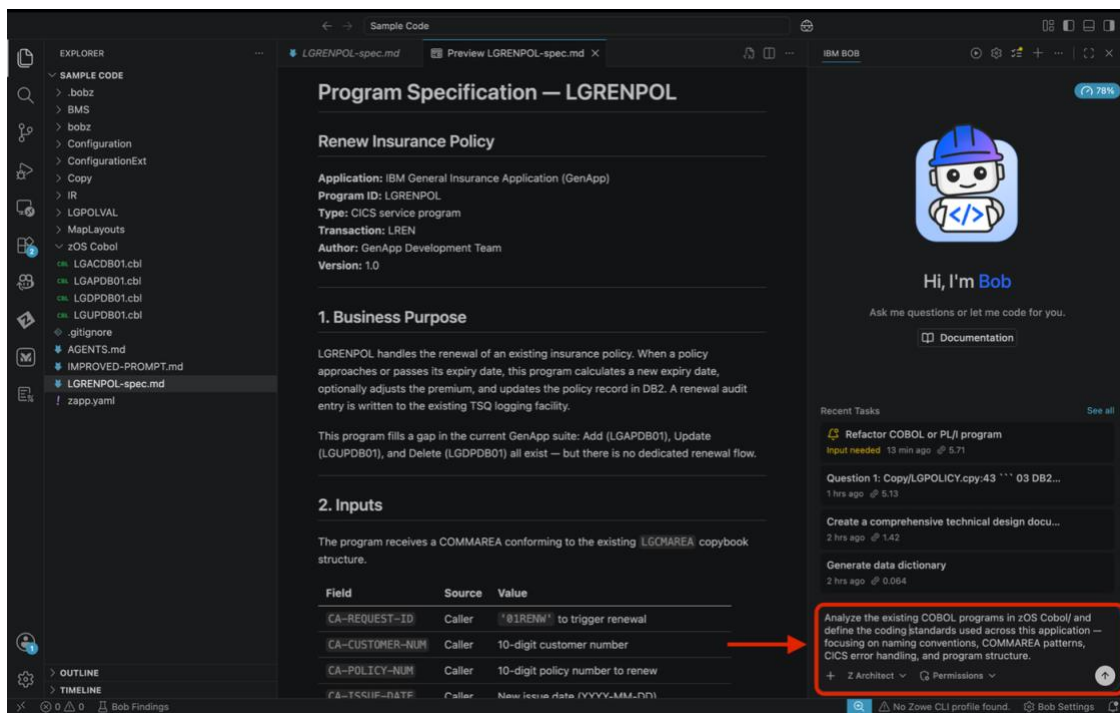


Spec document requirements

Exercise: Coding Standards

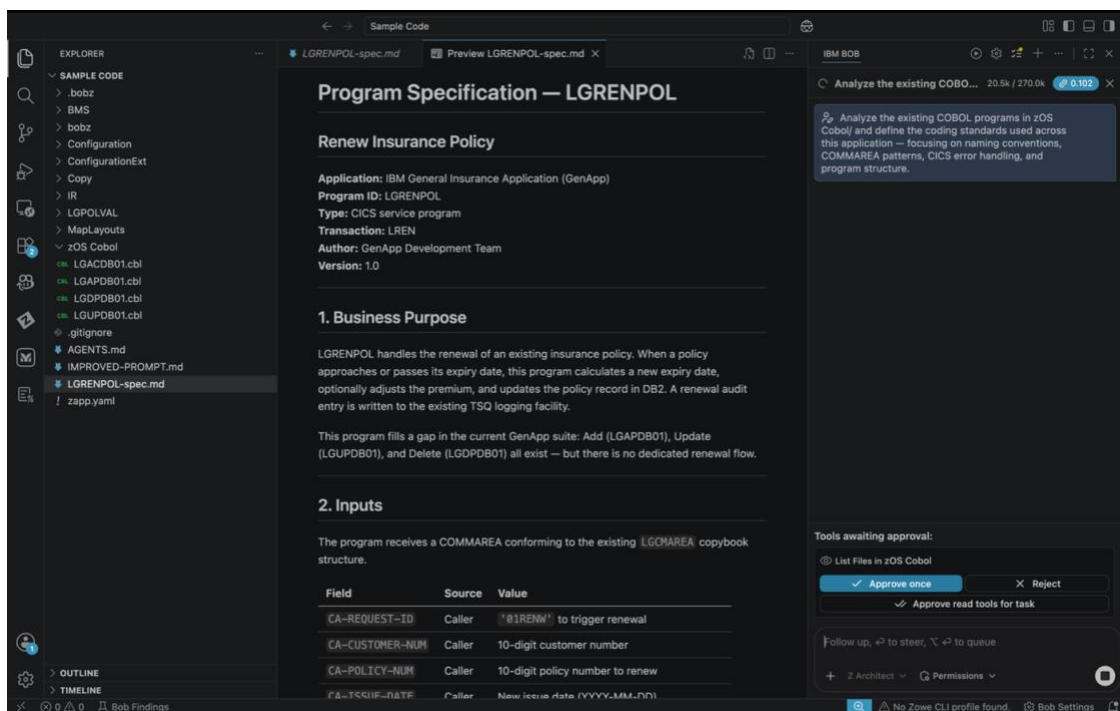
1. **ACTION:** Ensure you are in **Z Architect** mode, and enter the following prompt In the chat:

Analyze the existing COBOL programs in zOS Cobol/ and define the coding standards used across this application – focusing on naming conventions ,
COMMAREA patterns, CICS error handling, and program structure.



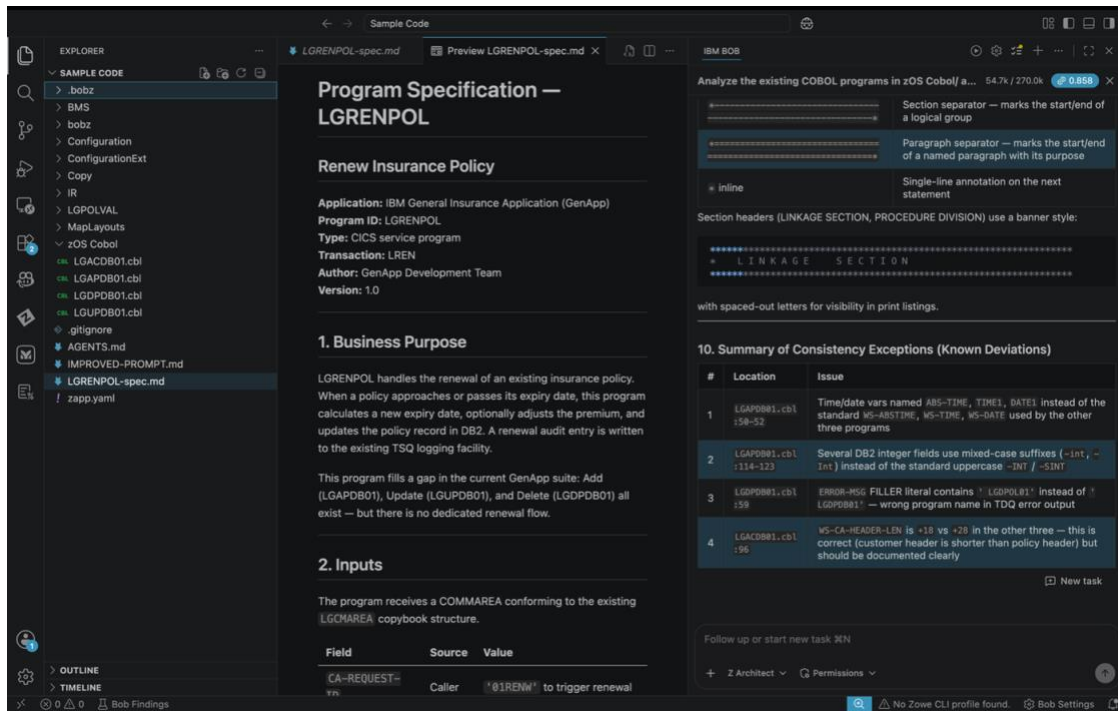
Coding standards prompt

- ACTION:** Please select the **Approve** option when prompted or you can select **Approve All** in permissions to auto approve all requests.



Approve coding standards analysis

3. Please review the generated chat window on the right side of the screen.



Coding standards results

Expected Results

- ✅ Bob summarizes GenApp's coding conventions

Exercise: Generate the Program from the Spec

Point Bob at the spec sheet and the existing codebase together — let it generate the full COBOL program in one prompt. The spec defines *what* to build; the existing programs define *how* to build it.

1. **ACTION:** Ensure you are now in Z Code mode, in the chat, enter:

Using the program specification in LGRENPOL-spec.md, generate the complete

CICS COBOL program LGRENPOL and save it as zOS Cobol/LGRENPOL.cbl.

Follow the structure and conventions of zOS Cobol/LGAPDB01.cbl

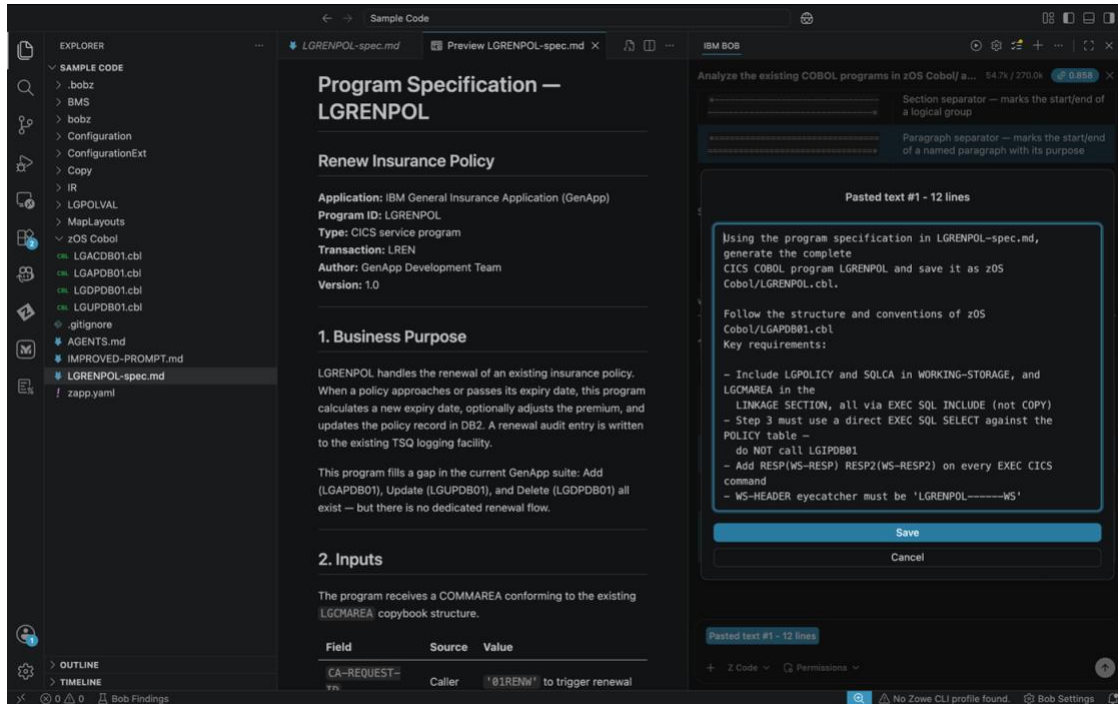
Key requirements:

- Include LGPOLICY and SQLCA in WORKING-STORAGE, and LGCMAREA in the LINKAGE SECTION, all via EXEC SQL INCLUDE (not COPY)
- Step 3 must use a direct EXEC SQL SELECT against the POLICY table — do NOT call LGIPDB01

- Add RESP(W5-RESP) RESP2(W5-RESP2) on every EXEC CICS command
- WS-HEADER eyecatcher must be 'LGRENPOL-----WS'

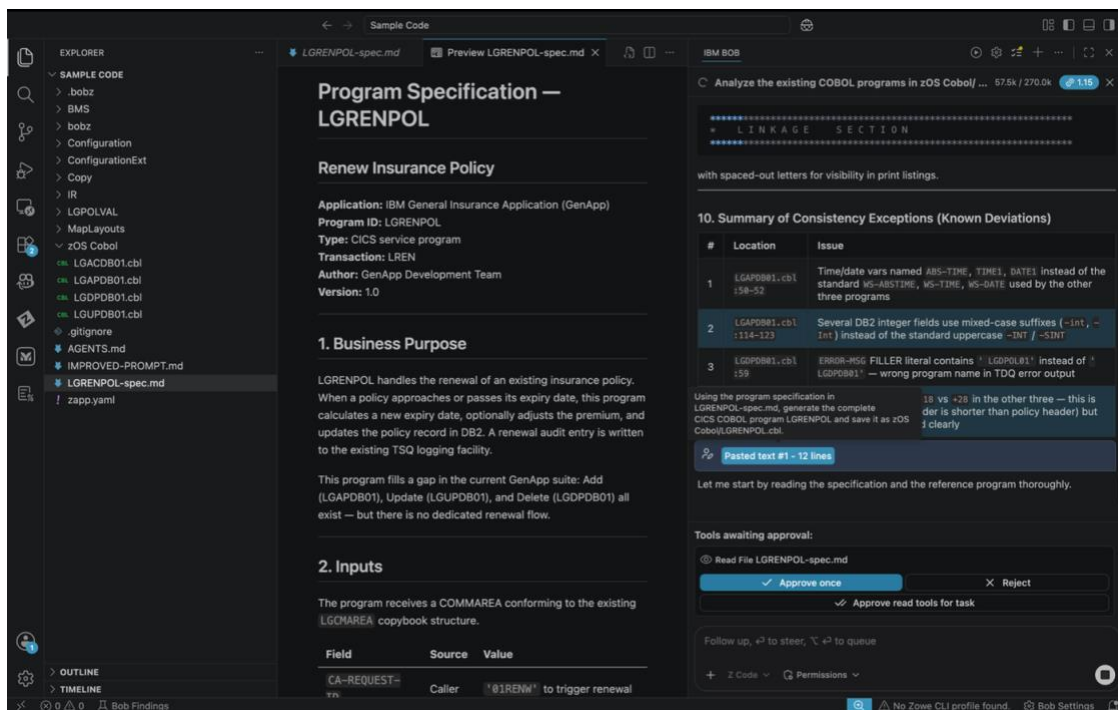
Tip

When you paste it may show as “Pasted Text”, if you click on it, it will appear like before. Press enter once completed.



Code generation prompt

2. **ACTION:** You may see a couple of approvals, please continue to click approve unless otherwise noted. This will include the creation of the program.

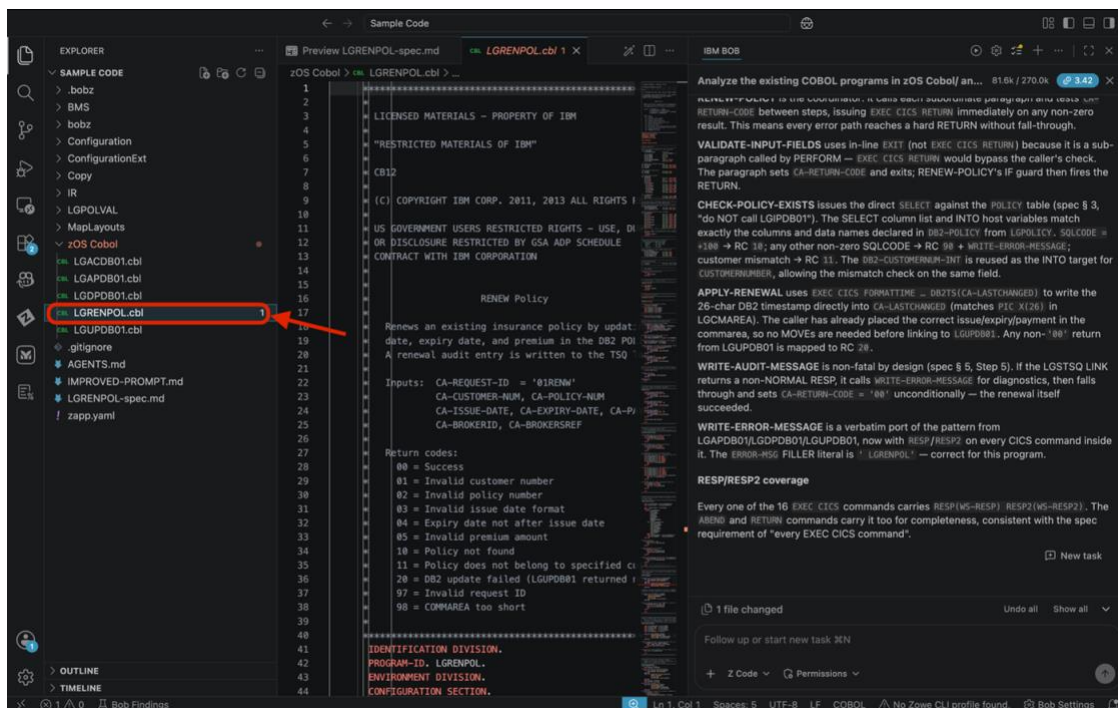


Approve code generation

- When complete, open z/OS Cobol/LGRENPOL.cbl in the editor and review.

Expected Results

- ✓ z/OS Cobol/LGRENPOL.cbl created
- ✓ PROGRAM-ID. LGRENPOL in the Identification Division
- ✓ WS-HEADER block present with eyecatcher ' LGRENPOL - - - - -WS '
- ✓ COPY LGCMAREA in the Linkage Section
- ✓ EXEC CICS LINK PROGRAM(LGUPDB01) for the DB2 update
- ✓ All return codes from the spec (00, 01-05, 10, 11, 20, 97, 98) present
- ✓ All paragraphs end with EXIT



Generated LGRENPOL program

Exercise: Verify Against the Spec

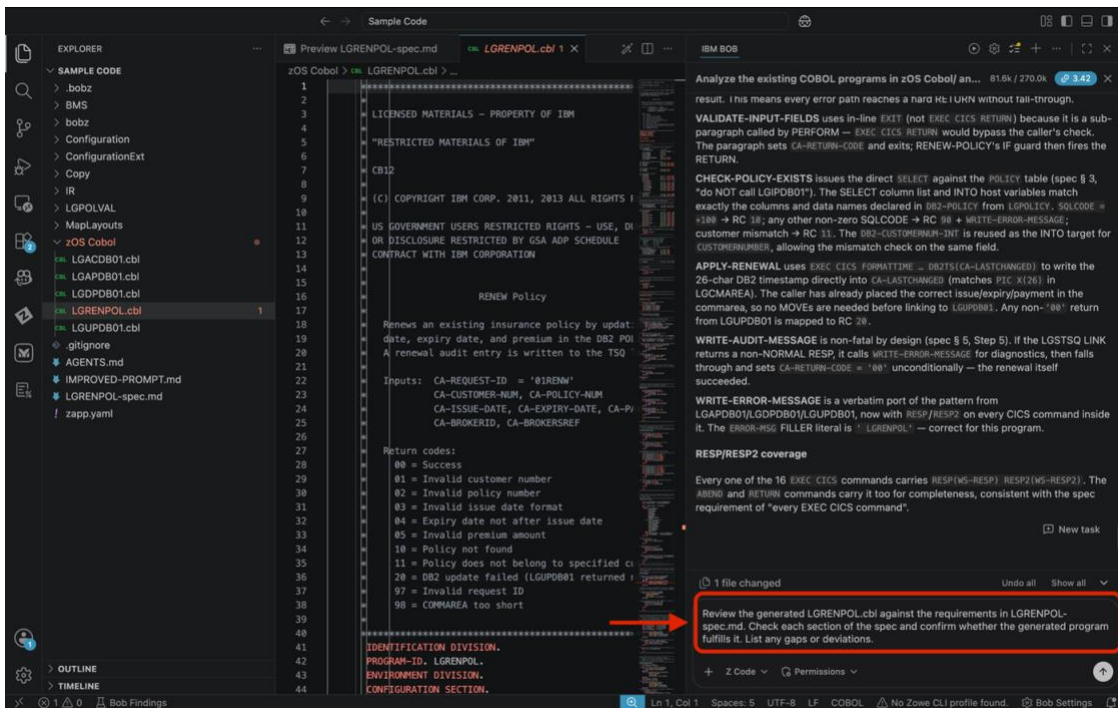
Systematically check the generated program against the spec — section by section. This is the review step a lead developer would perform before approving a generated program for testing.

1. **ACTION:** In the chat, enter the following prompt:

Review the generated LGRENPOL.cbl against the requirements in labs/z-spec-driven-development/LGRENPOL-spec.md.

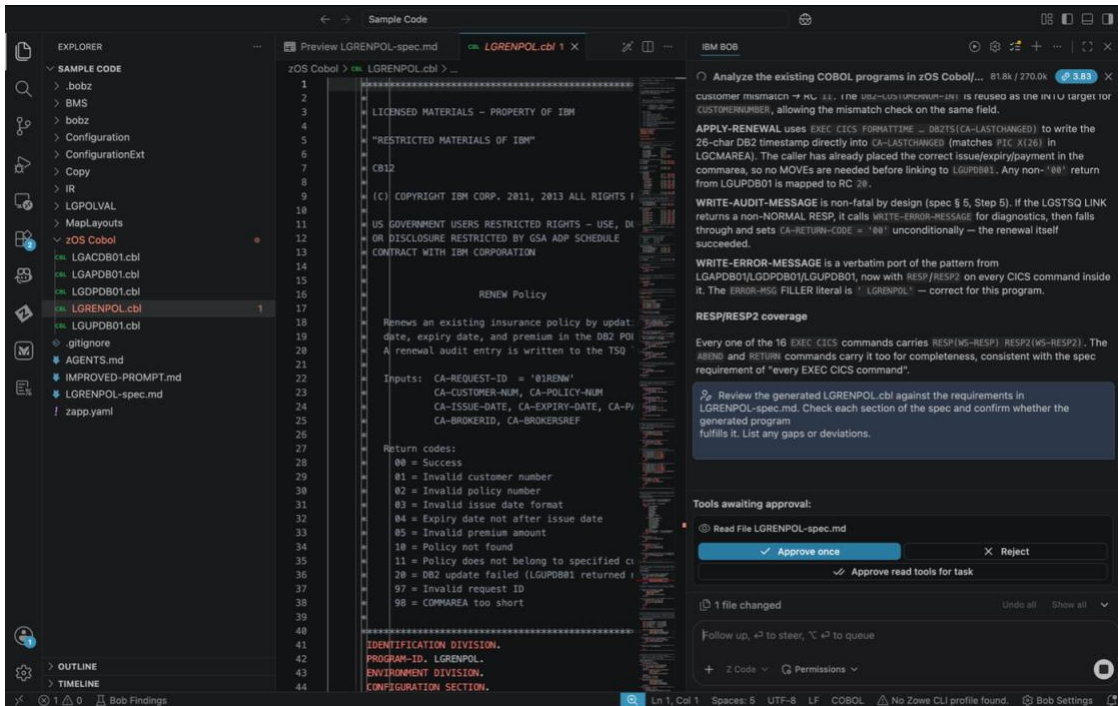
Check each section of the spec and confirm whether the generated program

fulfills it. List any gaps or deviations.



Verification prompt

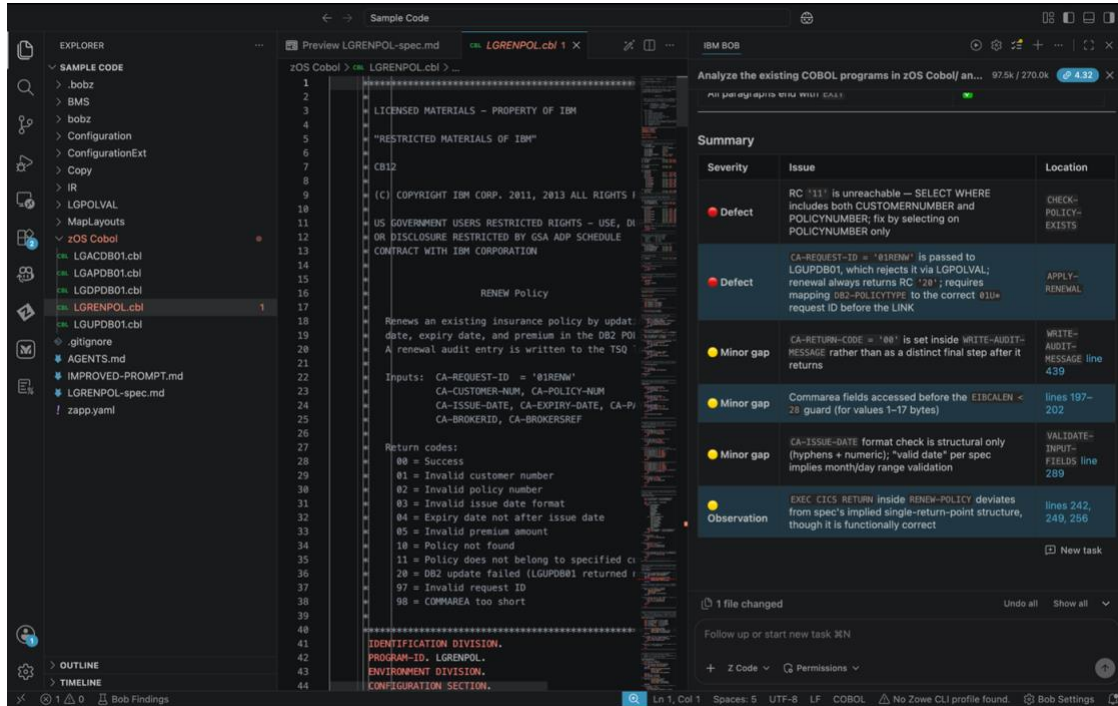
2. ACTION: Select the approve option unless otherwise noted.



Approve verification

- Review the generated chat window on the right side of the screen to understand any gaps or deviations from the spec.

Optional Feel free to ask Bob to fix any gaps or deviations found in the review.



Verification results

Key Takeaways

- How to write a program spec that gives Bob enough context to generate correctly
- How to use a spec sheet as the primary generation prompt
- How to systematically verify generated code against a specification

Start a New Chat

Please select the + sign at the top of the chat window to start a new session before moving to the next lab.

Lab 6: UI Modernization — CICS to Modern Web

Prerequisites: Complete Lab 1 (Lab Setup) before starting this lab.

Code Set: Any COBOL workspace — recommended code: Sample Code

Duration: 45 minutes

Difficulty: Intermediate

Overview

In this use case, you'll transform CICS green screen applications into modern web interfaces — analyzing BMS maps, understanding screen flow, and generating modern UI components with REST APIs.

Exercise: Understand & Analyze Insurance Application

Learn to analyze CICS screens and understand the business flow, data requirements, and user interactions.

1. **ACTION:** Ensure you are in **Z Architect** mode, and copy and paste the following into the chat:

Create an analysis document of this application that helps analyze the conversion of greenscreens to a modern UI

1. Identify all screens and their purposes
2. Map screen fields to data entities
3. Identify user workflows and navigation patterns
4. Determine which screens should become:
 - Separate pages
 - Modal dialogs
 - Inline forms
 - Dashboard widgets
5. Identify data relationships between screens
6. Suggest modern UI/UX improvements

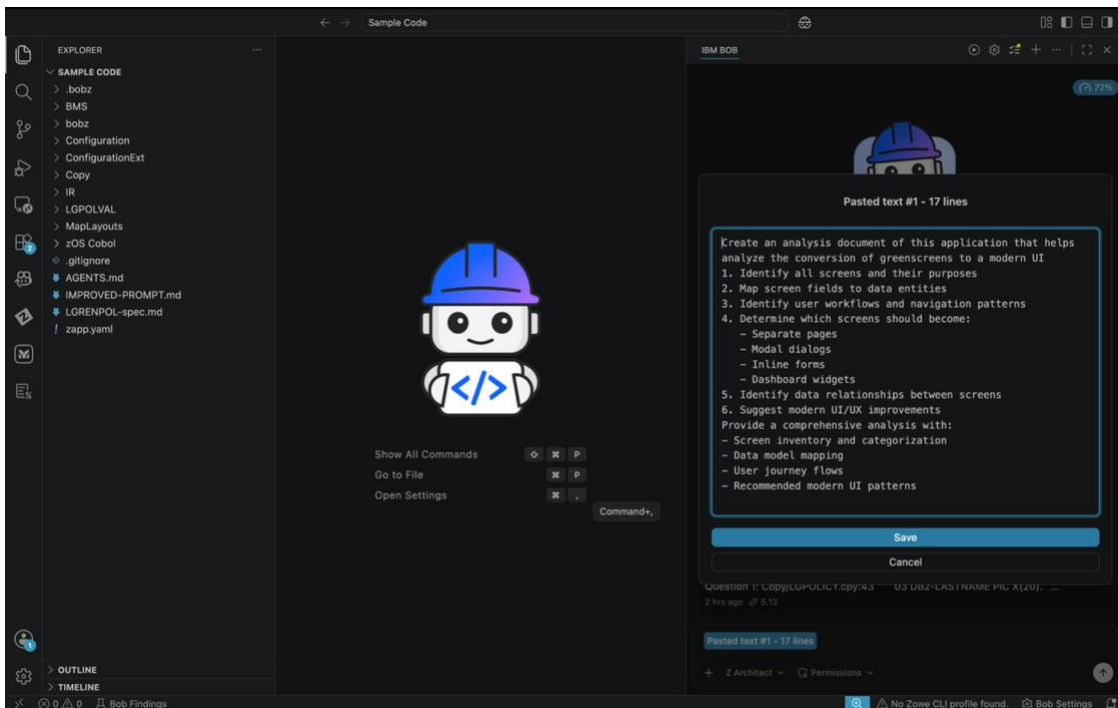
Provide a comprehensive analysis with:

- Screen inventory and categorization
- Data model mapping
- User journey flows
- Recommended modern UI patterns

Tip

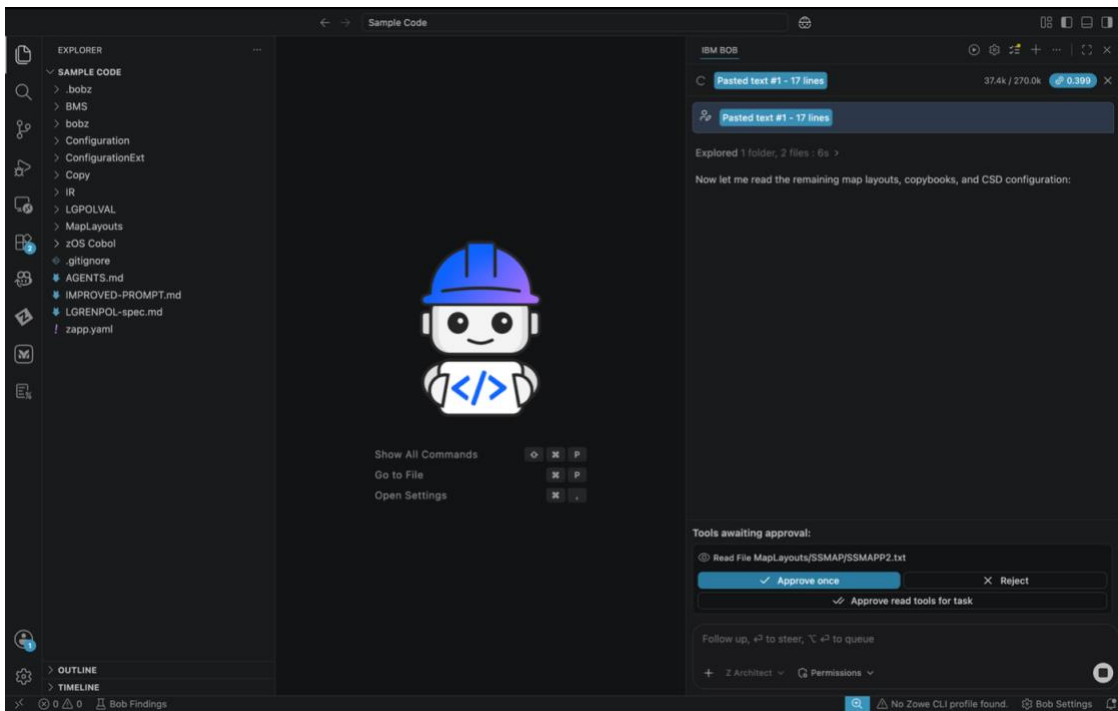
When you paste it may show as “Pasted Text”, if you click on it, it will appear like before. Please press enter once completed.

Recommendation Turn Auto approve on during this section!



UI analysis prompt

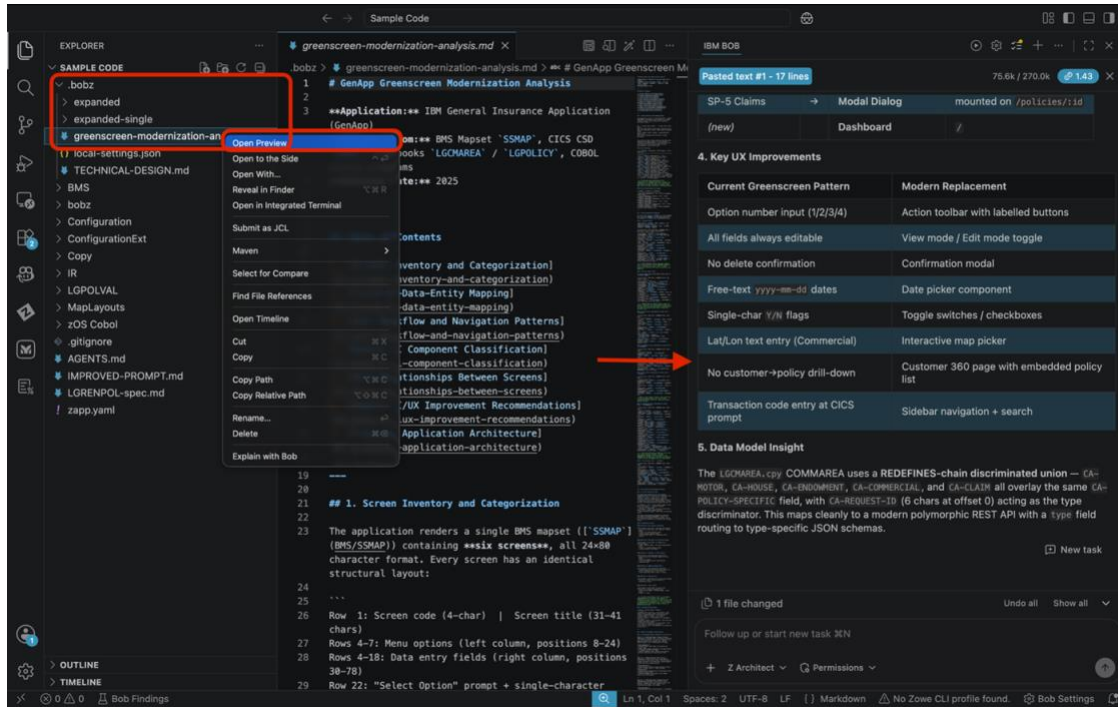
2. In the chat box, you'll notice Bob will begin to generate information, such as reading from the various files to understand the ask. Please continue to select approve, including the generation of the analysis file.



Bob generating analysis

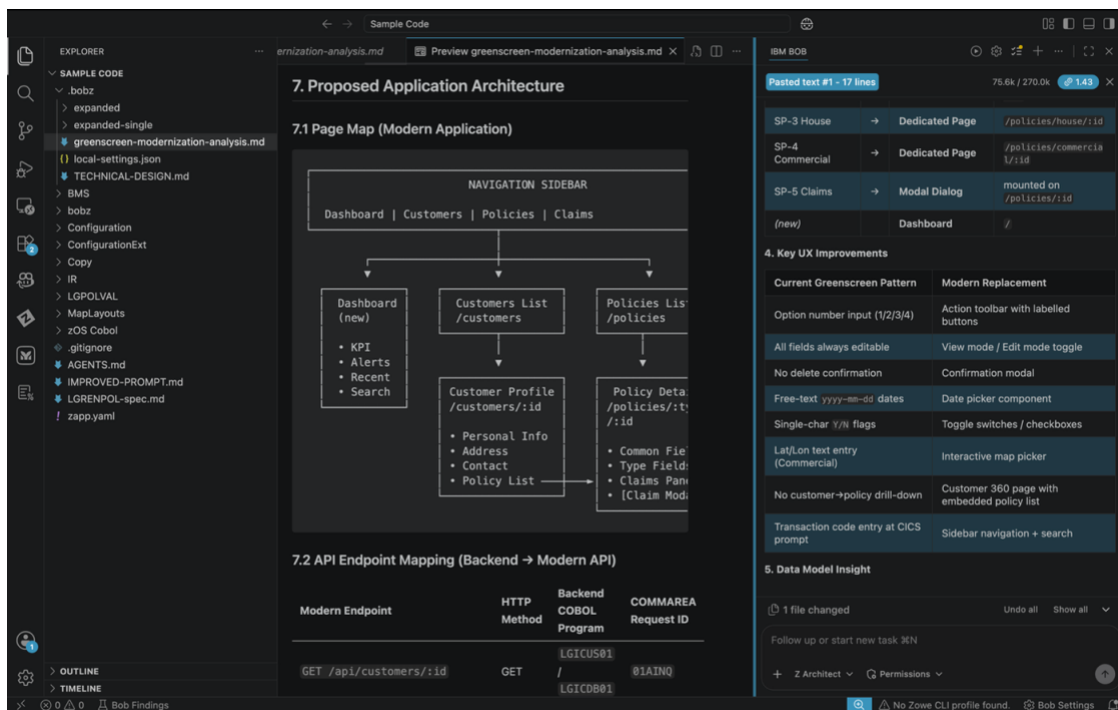
3. In the chat box, review the generated content to find the location where the documents have been generated. Please open this as a preview.

Example: For example, this document is located in .bobz/greenscreen-modernization-analysis.md.



Analysis document location

4. Please scroll down the generated output to find information such as modern UX/UI Recommendations, Rest API Endpoints etc.



UI recommendations output

Expected Results

- ✓ Complete screen inventory documented
- ✓ Screen flow understood
- ✓ Data fields cataloged
- ✓ Validation rules identified
- ✓ Navigation paths mapped

Exercise 2: Leverage prompt within chat to implement UI

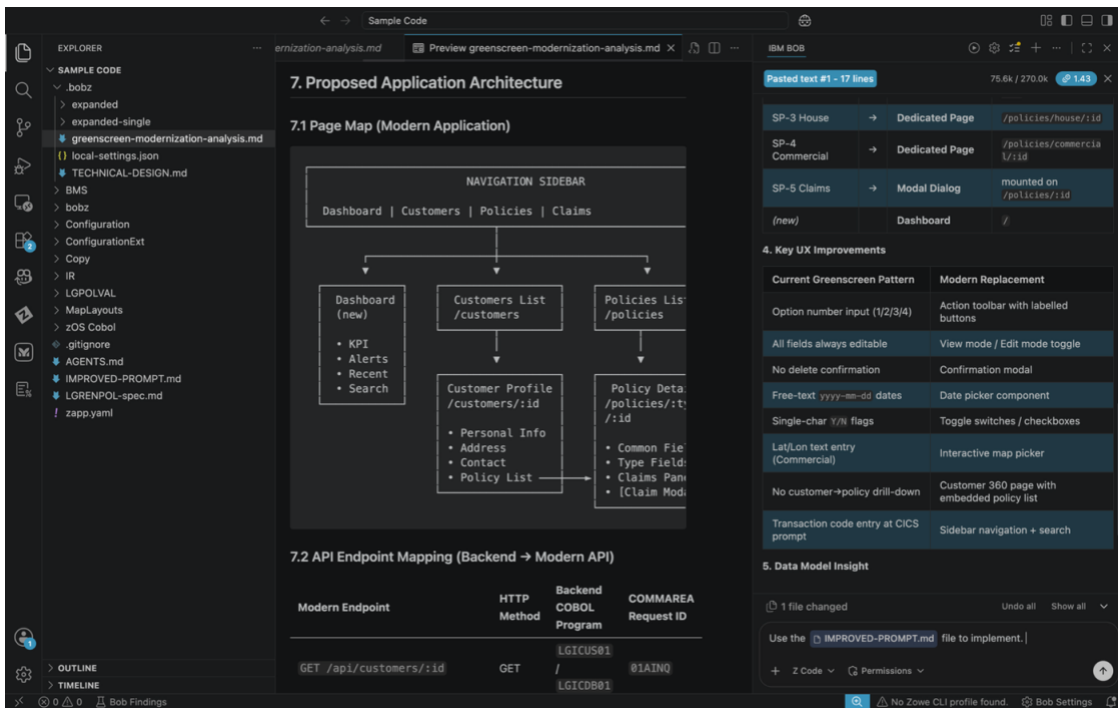
Transform green screen specifications into modern, user-friendly web interface designs.

- ACTION:** Please switch to Z Code Mode and copy the path from the Improved_Prompt.md to paste into the chat session. In the chat window, enter the following prompt, replacing <path to file> with the actual file path of the Improved_Prompt.md file. You can copy the path by right clicking the file in the explorer panel and selecting Copy Path:

Use the <path to file> file to implement.

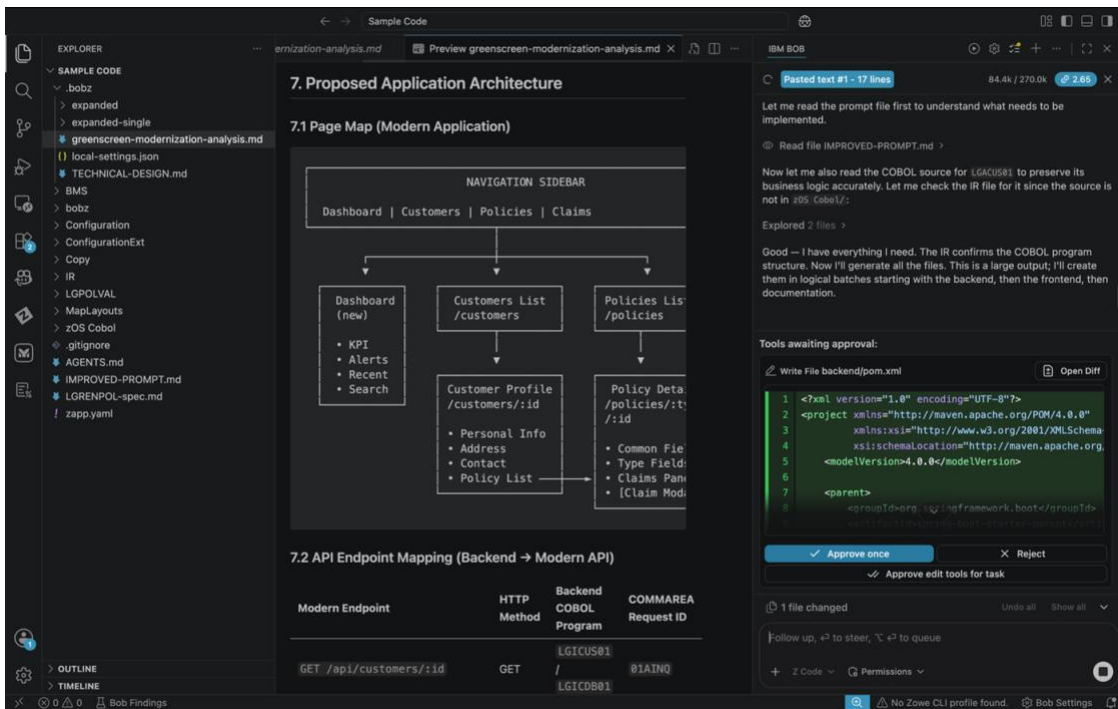
Tip

You can also use the @ symbol in the chat bar to reference files directly from the explorer, which automatically inserts the correct path without needing to copy it manually.



Implement UI prompt

- You'll notice that Bob creates an action plan as it moves through the creation of these features. You will be asked to approve some actions if auto approve is turned off. Recommendation is to turn it on for this portion.

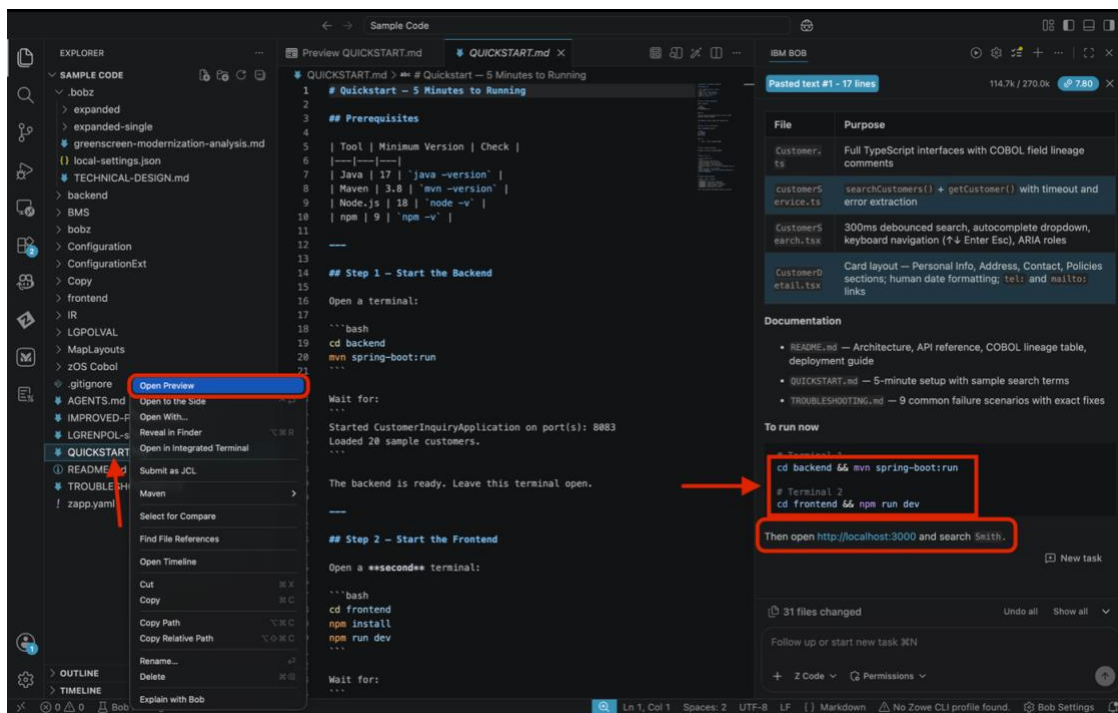


Bob action plan

- During generation you will notice that multiple files have been generated. The modernization code for the front-end and the back-end will get generated first, afterwards you will see a quick start guide gets generated as the one of the last steps of running the long prompt. Please open the Quickstart.md as a preview.

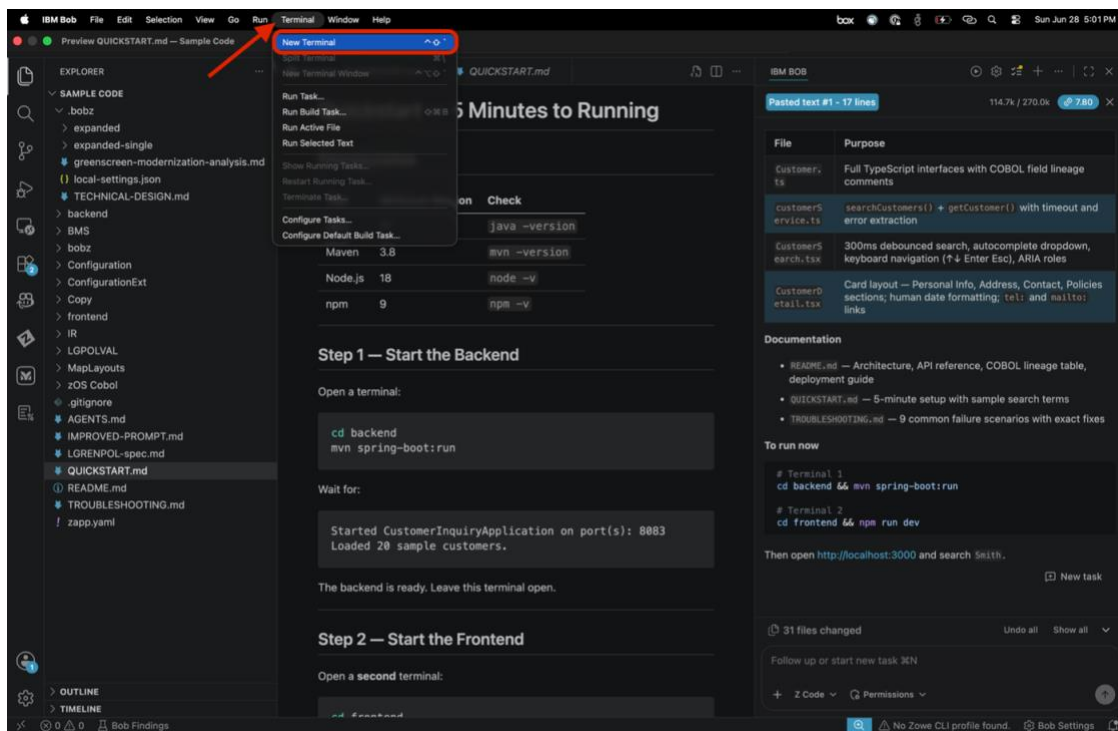
Sometimes it doesn't complete and you don't have your QUICKSTART.md - in that case you will need to do an additional prompt to get it finished.(Please finish your work on this till you have the QUICKSTART.md file generated)

- You will see some commands to start the services in the final outputs of the chat. These commands may be slightly different from the screenshots below. If you receive an error message in starting a service, please flag an instructor down.



Service start commands

- ACTION:** Bring up the terminal in VS Code



Open terminal in VS Code

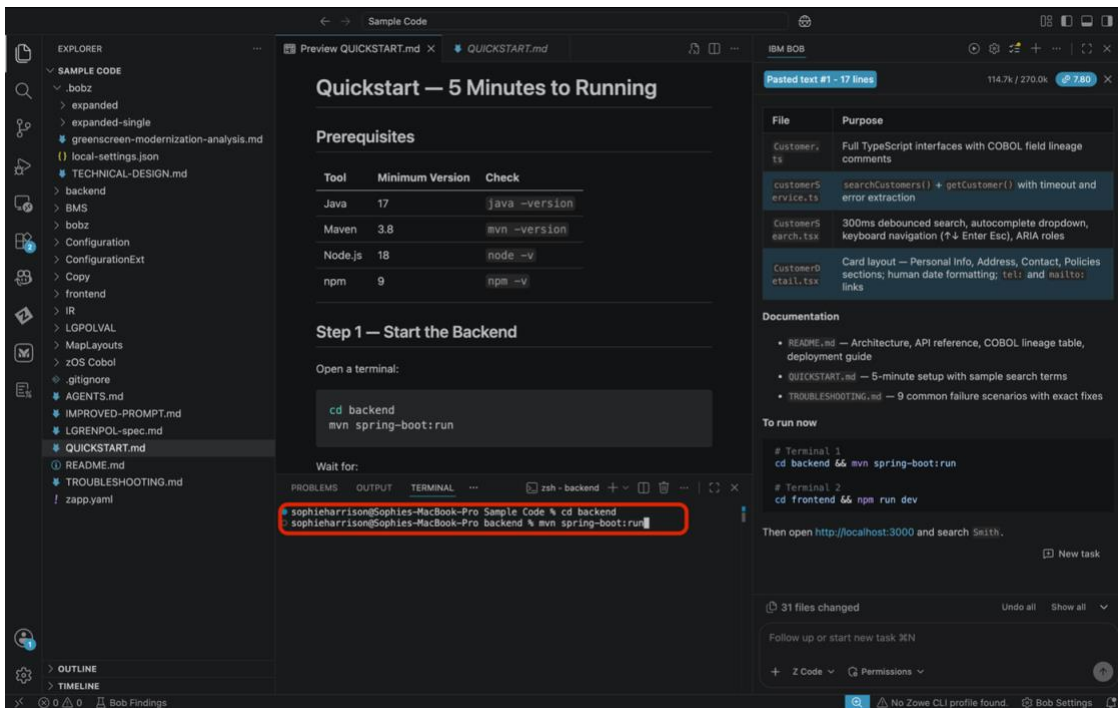
STOP

Take a moment and ensure you have all the necessary items installed, if you do not - see your QUICKSTART.md for links to install in your system.

6. **ACTION:** Issue the following commands that are in your QUICKSTART.md to get to the backend and then run command. These may be different than what you see below. If you have any trouble please alert an instructor!

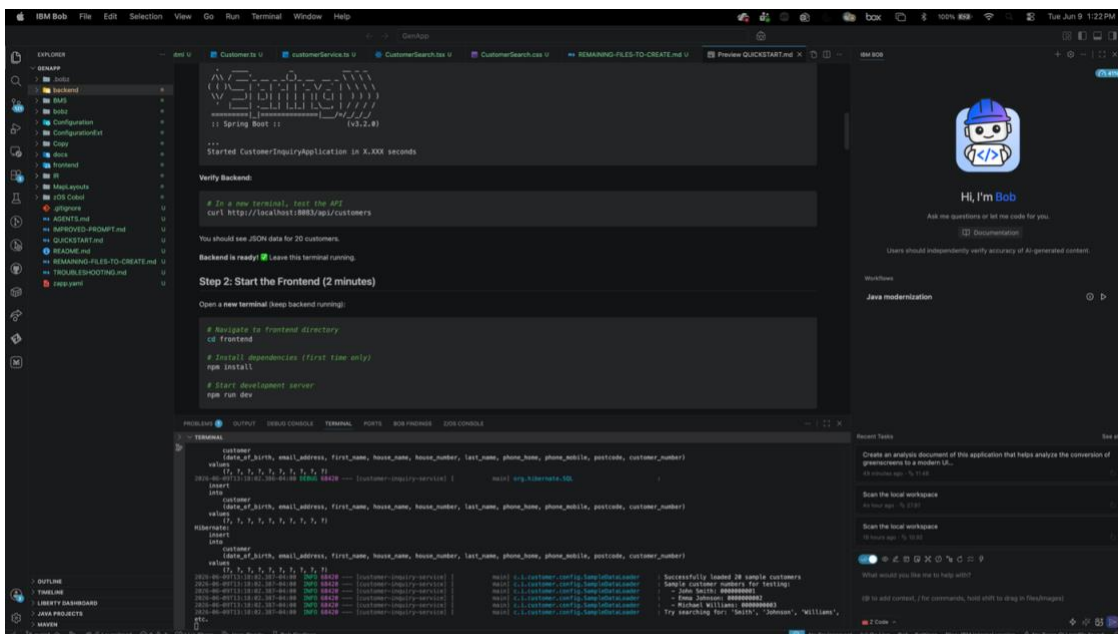
For example, the commands may look like this:

```
cd <path to backend>
mvn spring-boot:run
```

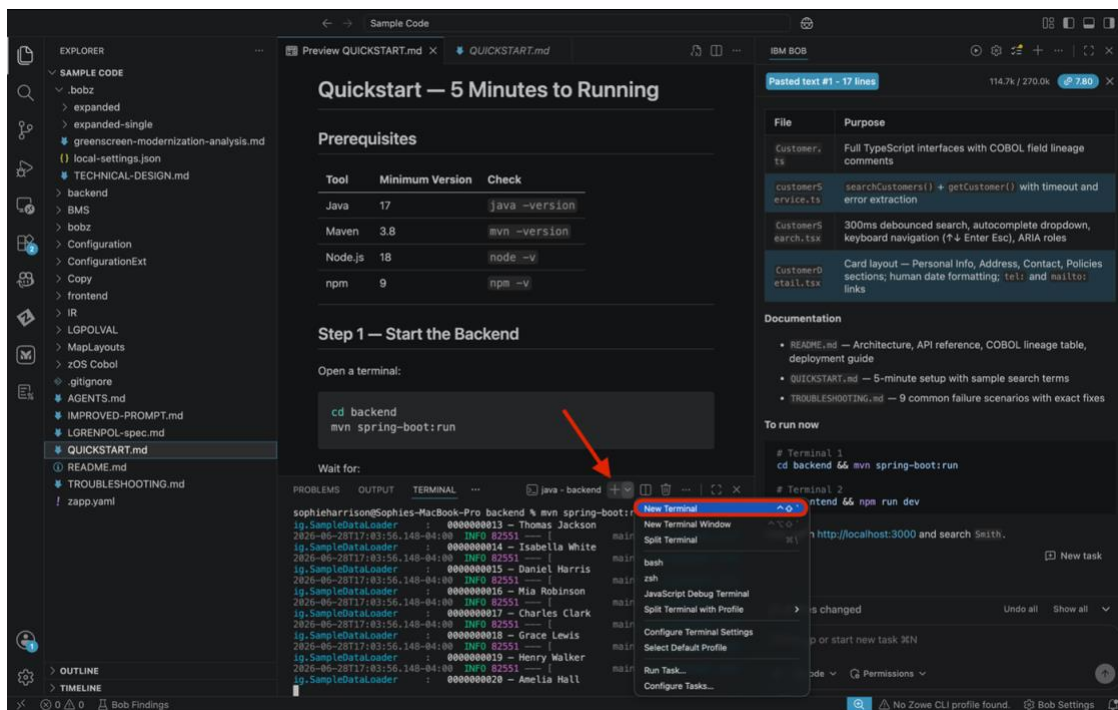
Backend startup

- You should see where it mentions having loaded 20 sample customers. Leave this terminal running.



Backend loaded customers

- ACTION:** Open a second terminal



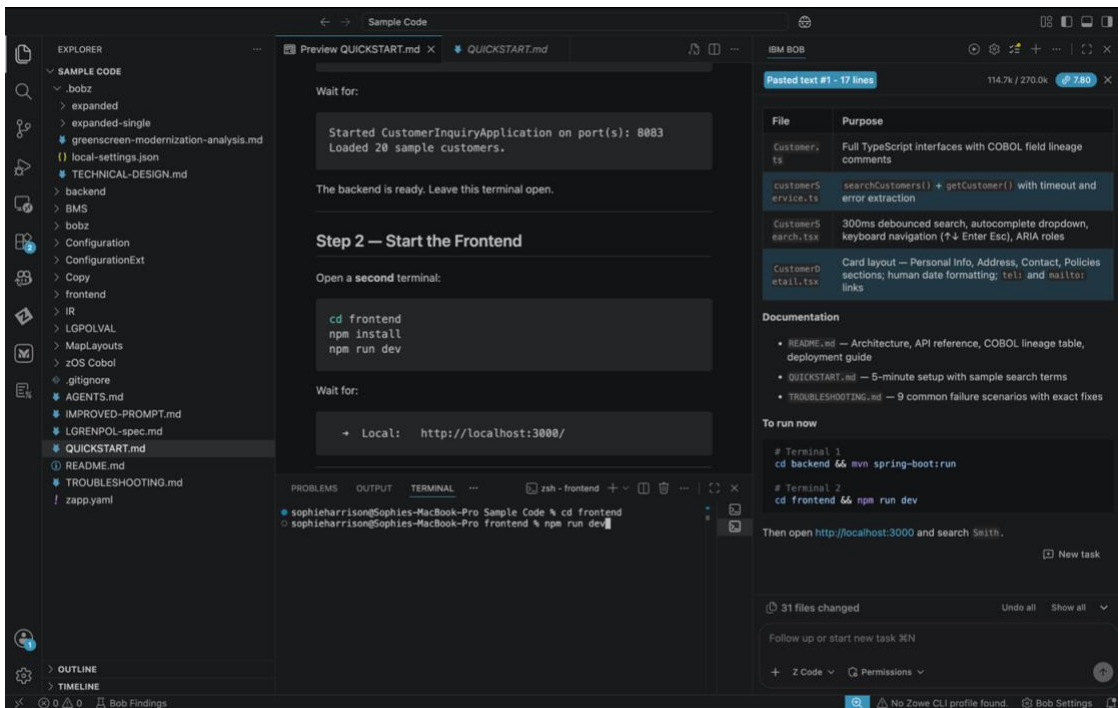
Second terminal

9. **ACTION:** Issue the following commands:

`Cd <path to frontend>`

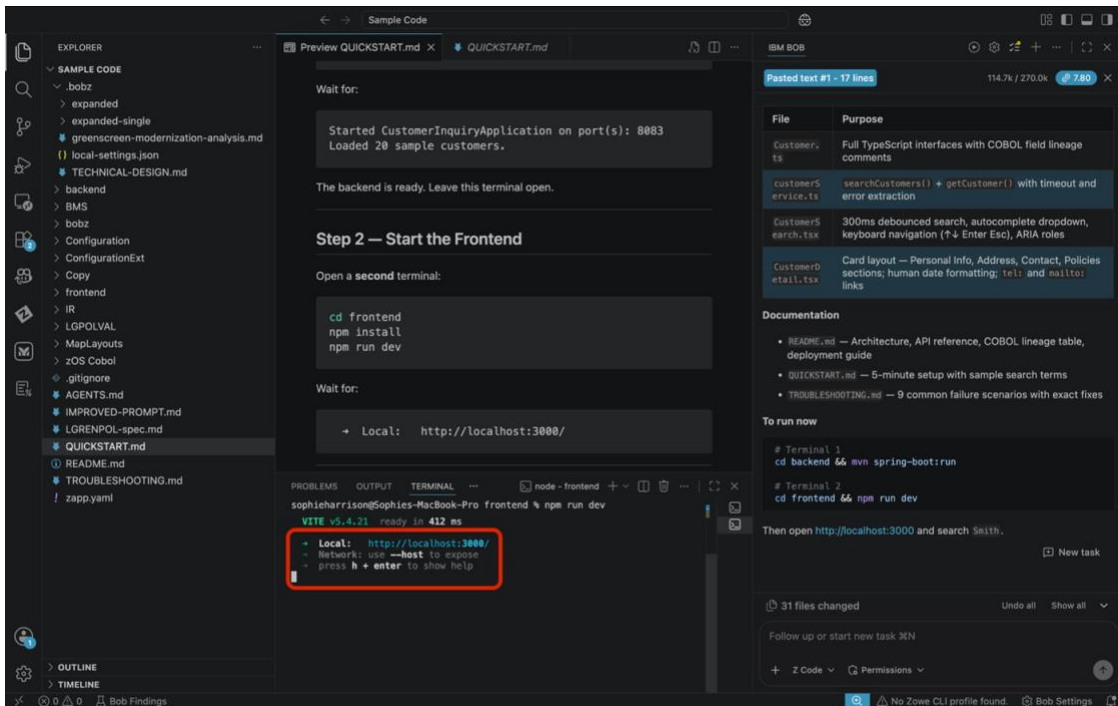
`Npm install && npm run dev`

Note In the following example picture npm install is not issued as it has been previously installed.



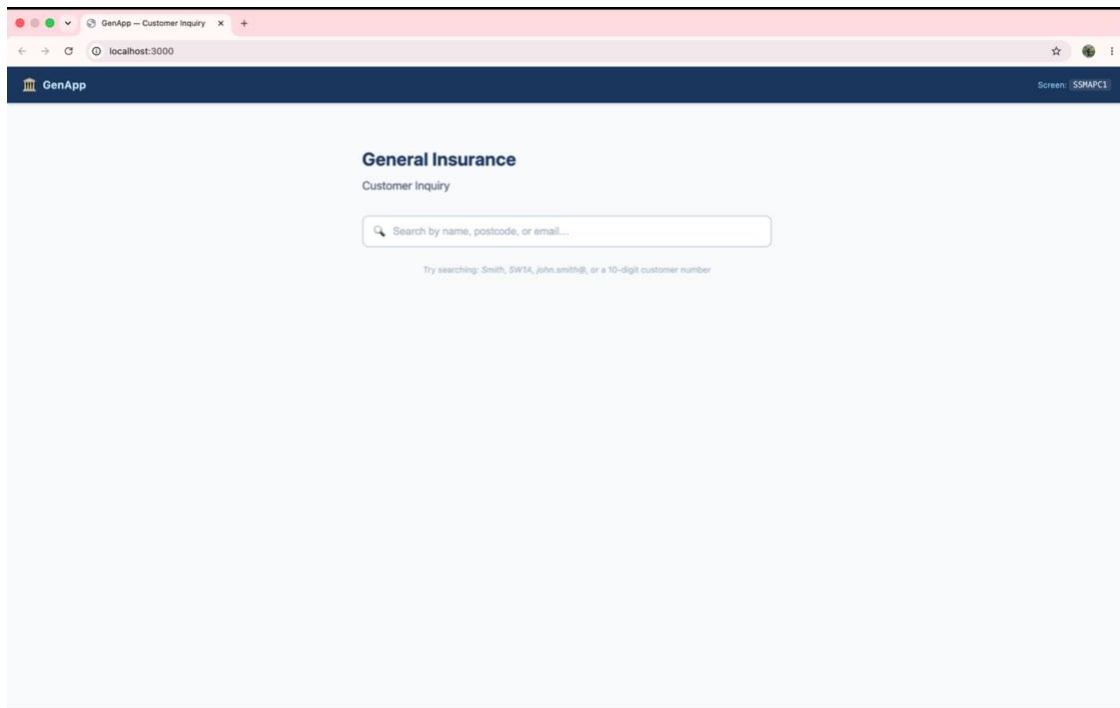
Frontend startup

10. **ACTION:** Click “follow link” on the local address listed.



Follow link to local address

11. Explore UI

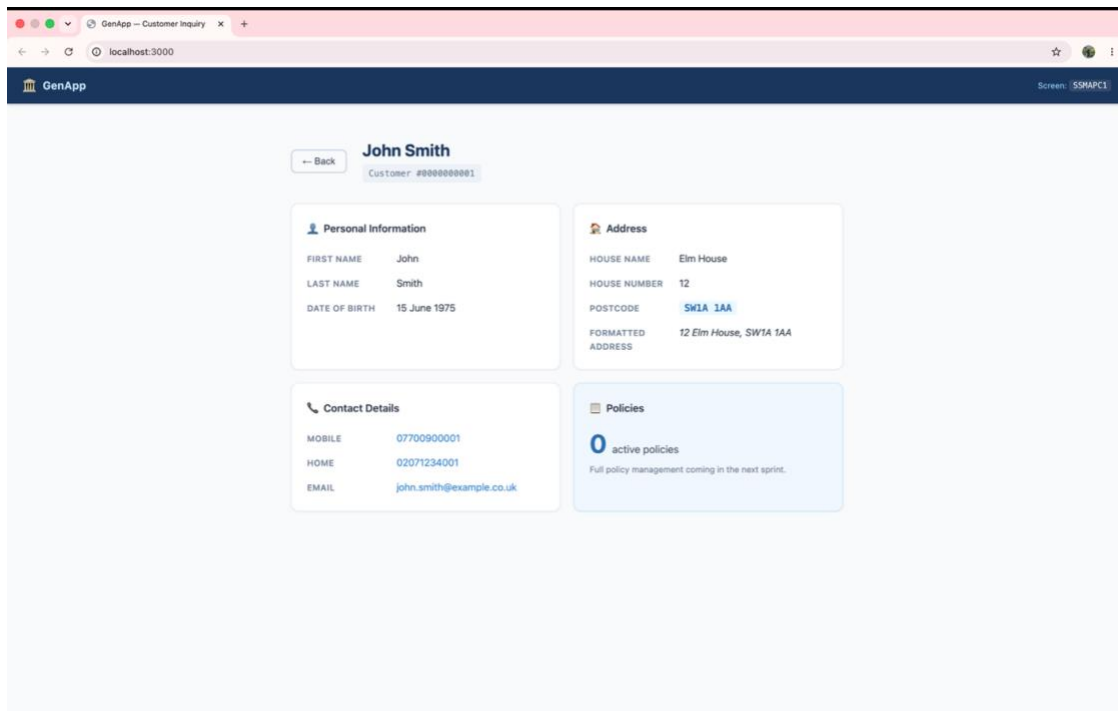


Modern UI interface

12. Try typing Smith into the search bar, here we can see that personal information has been added into customer details.

Tip

If your UI doesn't look like the one in the picture you can grab a screenshot and prompt Bob in code mode to make your frontend to look like the picture attached and add the picture and he will match it.



Search results in modern UI

Expected Results

- ☒ Modern UI design created
- ☒ Component structure defined
- ☒ React/Angular code generated
- ☒ Responsive layout implemented
- ☒ Form validation included
- ☒ Error handling comprehensive

Key Takeaways

- How to analyze CICS green screens for modernization
- Generating modern UI components from legacy specifications
- Creating REST API wrappers for mainframe services
- Building responsive web interfaces from green screen workflows